

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**CẢI TIẾN MÔ HÌNH GOLD-YOLO
CHO HỆ THỐNG NHẬN DIỆN ĐỐI TƯỢNG**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: PGS-TS. VÕ MINH HUÂN

SVTH: ĐẶNG VĂN NGUYỄN

MSSV: 22119202

HUỲNH KIM PHÁT

MSSV: 22119209

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

CẢI TIẾN MÔ HÌNH GOLD-YOLO CHO HỆ THỐNG NHẬN DIỆN ĐỐI TƯỢNG

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

GVHD: PGS-TS. VÕ MINH HUÂN

SVTH: ĐẶNG VĂN NGUYỄN

MSSV: 22119202

HUỲNH KIM PHÁT

MSSV: 22119209

TP. HỒ CHÍ MINH – 06/2026



TP. Hồ Chí Minh, ngày 26 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN

Họ và tên sinh viên: Đặng Văn Nguyên - **MSSV:** 22119202

Huỳnh Kim Phát - **MSSV:** 22119209

Ngành: Công nghệ kỹ thuật máy tính

Lớp: 221191A-221191B

Giảng viên hướng dẫn: PGS-TS.Võ Minh Huân

Ngày nhận đề tài: 14/03/2026

Ngày nộp đề tài: 26/06/2026

1. Tên đề tài: Cải tiến mô hình gold-yolo cho hệ thống nhận diện đối tượng

2. Các số liệu, tài liệu ban đầu: Kiến thức cơ bản về các môn Học sâu (Deep Learning), Thị giác máy tính (Computer Vision) và Hệ thống nhúng / Trí tuệ nhân tạo nhúng (Edge AI),...

3. Nội dung thực hiện đề tài

Đề tài " Cải tiến mô hình Gold-YOLO cho hệ thống nhận diện đối tượng " được triển khai bằng Python và PyTorch. Nhóm đã tùy biến Backbone và Head với khối FasterBlock (tối ưu thiết bị Edge), module SimAM, hàm kích hoạt Mish và hàm mất mát Loss WIoU. Mô hình sau đó được huấn luyện và đánh giá toàn diện qua độ chính xác, số tham số, FLOPs và Inference Time nhằm phân tích ưu nhược điểm và đề xuất hướng phát triển.

TRƯỞNG BỘ MÔN
(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN
(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Cải tiến mô hình Gold-YOLO cho hệ thống nhận diện đối tượng

Sinh viên: + Đặng Văn Nguyên
+ Huỳnh Kim Phát

MSSV: 22119202
MSSV: 22119209

Hướng dẫn: PGS-TS. Võ Minh Huân

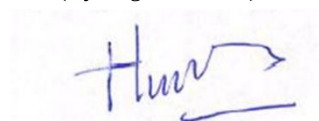
Nhận xét bao gồm các nội dung sau đây:

- Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:**
Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.
Đề tài đã xác định và đặt vấn đề một cách rõ ràng, bám sát với thực tiễn. Mục tiêu nghiên cứu được khoanh vùng cụ thể. Khung giải quyết vấn đề đi đúng hướng và đáp ứng được các yêu cầu cơ bản đã đề ra.
 - Phương pháp thực hiện/ phân tích/ thiết kế:**
Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.
Phương pháp có độ tin cậy dựa trên cơ sở lý thuyết và mô hình của các bài báo uy tín. Quá trình thiết kế, phân tích hệ thống được thực hiện bài bản, logic, phù hợp mục tiêu đã đề ra của đề án.
 - Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:**
Phù hợp với mục tiêu đề tài; phân tích và đánh giá / kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.
Các kết quả thực nghiệm thu được bám sát và phù hợp với mục tiêu ban đầu của đề tài. Quá trình kiểm thử, đánh giá mô hình được thực hiện ở mức độ tương đối đầy đủ, đảm bảo được độ tin cậy của các số liệu báo cáo.
 - Kết luận và đề xuất:**
Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.
Phân kết luận tổng kết đúng những gì đề tài đã thực hiện được, logic với cách đặt vấn đề. Các hướng đề xuất cải tiến có tính khả thi, mang tính thực tiễn và tạo tiền đề tốt cho các nghiên cứu mở rộng sau này.
 - Hình thức trình bày và bố cục báo cáo:**
Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.
Báo cáo được trình bày hợp lý. Bố cục các chương mục phân chia mạch lạc, rõ ràng và tuân thủ đúng định dạng chuẩn của nhà trường.
 - Kỹ năng chuyên nghiệp và tính sáng tạo:**
Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, và các kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.
Có trao đổi và tham khảo ý kiến của GVHD, có kỹ năng làm việc nhóm, cũng như tinh thần làm việc nghiêm túc trong suốt quá trình thực hiện đề án.
 - Tài liệu trích dẫn**
Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.
Có tính trung và nguồn tài liệu tham khảo phong phú, phù hợp với lĩnh vực nghiên cứu và được trích dẫn tương đối chuẩn xác theo quy định.
 - Đánh giá về sự trùng lặp của đề tài**
Cần khẳng định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.
5% đạo văn
 - Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa***
Cần cải thiện Inference Time
 - Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên**
Sinh viên có tinh thần ham học, cầu tiến, chủ động tham khảo ý kiến GVHD để có hướng cải thiện
- Đề nghị của giảng viên hướng dẫn**
Ghi rõ: “Báo cáo đạt/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép/ không được phép bảo vệ khóa luận tốt nghiệp”

Báo cáo đạt yêu cầu và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 26 tháng 06 năm 2026

Người nhận xét
(Ký và ghi rõ họ tên)



Võ Minh Huân

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện khóa luận xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Võ Minh Huân. Thầy đã luôn tận tình hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những lời góp ý, chỉ bảo và đánh giá của thầy qua các buổi báo cáo đã giúp nhóm nhìn nhận ra những khuyết điểm, từ đó vượt qua các khó khăn về mặt kỹ thuật, nội dung cũng như hoàn thiện hình thức của bài báo cáo.

Tiếp theo, nhóm xin gửi lời tri ân đến quý thầy cô trong khoa Điện – Điện tử của Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh. Cảm ơn quý thầy cô đã truyền đạt những kiến thức quý báu từ cơ bản đến nâng cao, tạo tiền đề và nền tảng vững chắc cho việc thực hiện khóa luận này. Do kiến thức và kinh nghiệm thực tế còn hạn hẹp, nội dung của bài khóa luận chắc chắn khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ quý thầy cô để có thể tiếp thu và hoàn thiện đề tài hơn nữa.

Sau cùng, nhóm thực hiện xin kính chúc quý Thầy/Cô thật nhiều sức khỏe, luôn tràn đầy nhiệt huyết và gặt hái được nhiều thành công trong sự nghiệp giáo dục.

Nhóm thực hiện đề tài

Đặng Văn Nguyên

Huỳnh Kim Phát

TÓM TẮT

Trong bối cảnh công nghệ trí tuệ nhân tạo và hệ thống nhúng ngày càng phát triển mạnh mẽ, bài toán nhận diện đối tượng theo thời gian thực trên các thiết bị phần cứng Edge đòi hỏi các mô hình mạng nơ-ron không chỉ đạt độ chính xác cao mà còn phải tối ưu hóa toàn diện về mặt tốc độ lẫn tài nguyên tính toán. Tuy nhiên, các kiến trúc phân cấp kết hợp đặc trưng truyền thống thường gặp phải hiện tượng nghẽn cổ chai và thất thoát thông tin, trong khi các khối trích xuất tiêu chuẩn lại tiêu tốn nhiều năng lực xử lý. Sự ra đời của cơ chế Thu thập và Phân phối (Gather-and-Distribute) trong Gold-YOLO cùng giải pháp tinh gọn mạng của FasterNet đã mở ra một hướng tiếp cận đột phá để giải quyết thách thức này.

Đề tài “Cải tiến mô hình Gold-YOLO cho hệ thống nhận diện đối tượng” được thực hiện nhằm nghiên cứu, thiết kế và hiện thực hóa một mô hình thị giác máy tính tiên tiến, có tính ứng dụng cao trên các nền tảng phần cứng giới hạn tài nguyên. Hệ thống đề xuất thực hiện cải tiến khối C2f truyền thống thành khối Fast-C2f thông qua việc tích hợp tích chập một phần (Partial Convolution - PConv), giúp loại bỏ các tính toán dư thừa và tăng tốc độ truy cập bộ nhớ. Đồng thời, cơ chế chú ý ba chiều không tham số SimAM được đan cài linh hoạt nhằm tối ưu khả năng gán trọng số đặc trưng theo cả kênh và không gian, giúp mô hình tập trung chính xác vào đối tượng mục tiêu mà không làm gia tăng dung lượng hay làm chậm hệ thống.

Báo cáo luận án được trình bày theo cấu trúc chặt chẽ từ cơ sở lý thuyết về mạng nơ-ron tích chập và các cơ chế chú ý, thiết kế chi tiết các khối chức năng cải tiến, đến triển khai thực nghiệm và đánh giá hiệu năng. Quá trình thực nghiệm được triển khai trên nền tảng PyTorch thuộc hệ sinh thái Ultralytics, thực hiện huấn luyện và kiểm thử đối sánh toàn diện dựa trên các chỉ số học sâu cốt lõi bao gồm độ chính xác (mAP), số lượng tham số (Parameters), khối lượng tính toán (FLOPs) và tốc độ xử lý thực tế (FPS) nhằm chứng minh tính đúng đắn cùng sự vượt trội của giải pháp đề xuất.

MỤC LỤC

LỜI CẢM ƠN	v
TÓM TẮT.....	vi
DANH MỤC HÌNH ẢNH	xi
DANH MỤC BẢNG.....	xii
CÁC TỪ VIẾT TẮT	xiii
CHƯƠNG 1: GIỚI THIỆU	1
1.1. Đặt vấn đề và lí do chọn đề tài.....	1
1.2. Mục tiêu nghiên cứu.....	2
1.3. Đối tượng và phạm vi nghiên cứu.....	2
1.4. Cấu trúc báo cáo.....	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	5
2.1. Tổng quan về bài toán phát hiện đối tượng.....	5
2.1.1. Định nghĩa và các thách thức.....	5
2.1.2. Các hướng tiếp cận trong phát hiện đối tượng	6
2.1.3. Feature Pyramid Network (FPN) và đa tầng đặc trưng	8
2.2. Kiến trúc YOLOv8 chuẩn	10
2.2.1. Tổng quan kiến trúc	10
2.2.2. Backbone: Conv + C2f + SPPF	12
2.2.3. Mạng kết hợp (Neck) và Đầu phát hiện (Head) truyền thống	15
2.2.4. Hạn chế của YOLOv8 chuẩn.....	17

2.3. Kiến trúc YOLOv8-Gold	17
2.3.1. Tổng quan Gold-YOLO và động lực ra đời	17
2.3.2. Mạng trích xuất đặc trưng Backbone: Giữ nguyên Conv + C2f + SPPF	19
2.3.3. Gold-YOLO Head — các module đặc trưng.....	20
2.3.4. Detect head tại P3, P4, P5.....	25
2.3.5. Ưu điểm so với FPN chuẩn và hạn chế còn tồn tại	27
2.4. Các module nghiên cứu tích hợp.....	29
2.4.1. Module Fast-C2f.....	29
2.4.2. Cơ chế chú ý SimAM	32
2.4.3. Module Fast-C2f_SimAM	35
2.4.4. Tầng đặc trưng P2 trong phát hiện vật thể nhỏ.....	36
2.4.5. Cơ chế hàm kích hoạt Mish	38
2.4.6. Hàm mất mát WIoU.....	39
CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT.....	39
3.1. Phân tích hạn chế của mô hình Gold-YOLO	39
3.1.1. Khối C2f trong Backbone: Chi phí tính toán cao, chưa có cơ chế chú ý (Attention).....	40
3.1.2. Khối C2f trong Head: Nghẽn cổ chai tốc độ sau các bước Injection/Fusion	40
3.1.3. Thiếu tầng đặc trưng P2: Khó phát hiện vật thể nhỏ	40
3.1.4. Tổng hợp: Động lực (Motivation) cho từng cải tiến	41
3.2. Kiến trúc đề xuất GPFS.....	42

3.2.1. Tổng quan kiến trúc GPFS	42
3.2.2. Cải tiến 1 — Thay thế C2f $\rightarrow$ Fast-C2f_SimAM trong Backbone	45
3.2.3. Cải tiến 2 — Thay thế C2f $\rightarrow$ Fast-C2f trong Head.....	48
3.2.4. Cải tiến 3 — Bổ sung nhánh phát hiện vật thể nhỏ P2	50
3.2.5. Cải tiến 4 – Thay thế hàm kích hạt SiLU và ReLU sang Mish, thay đổi cơ chế tính toán hàm mất mát CIOU sang WIoUv3 với $\alpha = 1.9, \beta = 3$	52
3.3. So sánh kiến trúc 3 model	54
CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ.....	56
4.1. Dữ liệu thực nghiệm.....	56
4.1.1. Tổng quan về bộ dữ liệu gốc và tính phù hợp	56
4.1.2. Quá trình lọc bỏ nhiễu và tái cấu trúc hệ thống nhãn (Data Preprocessing) .	56
4.1.3. Phân chia dữ liệu và giải quyết thách thức mất cân bằng nhãn (Dataset Partitioning)	57
4.2. Thông số và môi trường của thiết bị thử nghiệm.....	58
4.2.1. Cấu hình phần cứng	58
4.2.2. Môi trường huấn luyện	58
4.2.3. Tham số huấn luyện.....	59
4.2.4. Thông số của thiết bị nhúng.....	59
4.3. Kết quả và so sánh theo baseline	60
4.3.1. Kết quả thực nghiệm.....	60
4.3.2. Phân tích các đường cong đánh giá hiệu năng mô hình	64
4.3.3. Ma trận nhầm lẫn (Confusion Matrix).....	68

4.3.4. Đánh giá kết quả phát hiện trên tập kiểm thử.....	72
4.3.5. So sánh mô hình đề xuất với mô hình cơ sở (Baseline)	73
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	75
5.1. Tóm tắt đóng góp của đề tài.....	75
5.2. Hạn chế của đề tài	76
5.3. Hướng phát triển tiếp theo	77
TÀI LIỆU THAM KHẢO.....	79

DANH MỤC HÌNH ẢNH

Hình 1:Sơ đồ khối của Yolov8	11
Hình 2: Sơ đồ khối của kiến trúc Gold-YOLO.....	19
Hình 3:Sơ đồ khối SimFusion_4in	22
Hình 4:Sơ đồ khối cơ chế hoạt động của InjectionMultiSum_Auto_pool	23
Hình 5: Sơ đồ khối AdvPoolFusion.....	25
Hình 6: C2f module	30
Hình 7:C2f-Faster module	30
Hình 8:Kiến trúc FasterNet và cơ chế Partial Convolution (PConv)	31
Hình 9:So sánh cơ chế phân bổ trọng số chú ý 1D, 2D và 3D	33
Hình 10:Sơ đồ khối của Yolov8	43
Hình 11:Sơ đồ khối của kiến trúc Gold-YOLO.....	43
Hình 12:Sơ đồ kiến trúc GPFS_YOLO	43
Hình 13:Biểu đồ Điểm F1 – Độ tin cậy	64
Hình 14:Biểu đồ Độ chuẩn xác – Độ tin cậy	65
Hình 15:Biểu đồ Độ chuẩn xác – Độ thu hồi.....	65
Hình 16:Biểu đồ Độ thu hồi – Độ tin cậy	66
Hình 17:Ma trận nhầm lẫn	69
Hình 18:Ma trận nhầm lẫn chuẩn hóa.....	69
Hình 19:Kết quả phát hiện đối tượng trên tập kiểm thử	72

DANH MỤC BẢNG

Bảng 1: Chi tiết cấu trúc mạng Backbone của Gold-YOLO	20
Bảng 2: Vai trò của các module trong Gold-YOLO Head	20
Bảng 3: So sánh kiến trúc 3 model	54
Bảng 4: Thống kê phân phối số lượng nhãn đối tượng trong tập dữ liệu thực nghiệm	58
Bảng 5: Cấu hình phần cứng.....	58
Bảng 6: Môi trường huấn luyện.....	59
Bảng 7: Tham số truyền trong quá trình huấn luyện	59
Bảng 8: Thông số của Raspberry Pi 4	59
Bảng 9: Kết quả thử nghiệm các cải tiến	60
Bảng 10: Kết quả đánh giá mô hình cải tiến.....	61
Bảng 11: Kết quả đánh giá mô hình YOLOv8m	61
Bảng 12: Kết quả đánh giá mô hình YOLOgold	61
Bảng 13: Kết quả đánh giá mô hình cải tiến trên thiết bị nhúng Raspberry Pi4	63
Bảng 14: So sánh mô hình cải tiến với mô hình cơ sở (baseline). ML: Máy huấn luyện, Pi4: Raspberry Pi4	74

CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Tên đầy đủ	Nghĩa tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	AP	Average Precision	Độ chính xác trung bình
3	ARM	Advanced RISC Machine	Cấu trúc vi xử lý RISC tiên tiến (phổ biến trên thiết bị nhúng)
4	C2f	Cross Stage Partial Bottleneck with Two Convolutions	Khối phân nhánh chéo một phần với hai phép tích chập (trong YOLOv8)
5	CPU	Central Processing Unit	Bộ xử lý trung tâm
6	FPN	Feature Pyramid Network	Mạng kim tự tháp đặc trưng
7	FPS	Frames Per Second	Số khung hình xử lý trên mỗi giây
8	GD-Neck	Gather-and-Distribute Neck	Cấu trúc mạng cổ kết hợp cơ chế Thu thập và Phân phối
9	GFLOPs	Giga Floating-point Operations per Second	Tỷ phép tính toán dấu phẩy động trên giây (đo độ phức tạp mô hình)
10	GPU	Graphics Processing Unit	Bộ xử lý đồ họa
11	IFM	Information Fusion Module	Module dung hợp thông tin (trong mạng Gold-YOLO)
12	INT8	8-bit Integer	Định dạng số nguyên kích thước 8-bit
13	KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute	Tập dữ liệu chuẩn hóa về giao thông của Viện Công nghệ Karlsruhe và Viện Công nghệ Toyota
14	MAC	Memory Access Cost / Multiply-Accumulate	Chi phí truy cập bộ nhớ / Phép nhân-tích lũy toán học
15	mAP	mean Average Precision	Độ chính xác trung bình toàn cục
16	NPU	Neural Processing Unit	Bộ xử lý nơ-ron chuyên dụng (tăng tốc AI)
17	ONNX	Open Neural Network Exchange	Định dạng chuyển đổi và trao đổi mạng nơ-ron mở
18	PConv	Partial Convolution	Phép tích chập một phần
19	PTQ	Post-Training Quantization	Lượng tử hóa mô hình sau khi huấn luyện
20	QAT	Quantization-Aware Training	Huấn luyện nhận thức lượng tử hóa
21	RAM	Random Access Memory	MemoryBộ nhớ truy cập ngẫu nhiên
22	SimAM	Simple, Parameter-Free Attention Module	Module chú ý ba chiều đơn giản không chứa tham số

23	TPU	Tensor Processing Unit	Bộ xử lý Tensor chuyên dụng cho học sâu
24	UAV	Unmanned Aerial Vehicle	Phương tiện bay không người lái (Thiết bị bay flycam/drone)
25	YOLO	You Only Look Once	Thuật toán nhận diện đối tượng "Bạn chỉ nhìn một lần"

CHƯƠNG 1: GIỚI THIỆU

1.1. Đặt vấn đề và lí do chọn đề tài

Trong những năm gần đây, thị giác máy tính (Computer Vision) đã trở thành một trong những lĩnh vực phát triển nhanh nhất của trí tuệ nhân tạo, với nhiều ứng dụng thực tiễn như giám sát an ninh, xe tự lái, kiểm tra chất lượng sản phẩm trong công nghiệp và hỗ trợ y tế [9, 10]. Trong đó, bài toán phát hiện đối tượng (Object Detection) đóng vai trò nền tảng, đòi hỏi hệ thống không chỉ nhận diện được đối tượng trong ảnh mà còn phải xác định chính xác vị trí và kích thước của chúng trong thời gian thực [9, 10].

Họ mô hình YOLO (You Only Look Once) từ khi ra đời đến nay luôn là lựa chọn hàng đầu trong các bài toán phát hiện đối tượng nhờ khả năng cân bằng giữa độ chính xác và tốc độ xử lý [1]. Phiên bản YOLOv8, được Ultralytics giới thiệu [2], đã mang lại nhiều cải tiến đáng kể về kiến trúc so với các phiên bản trước, đặc biệt là module C2f (Cross Stage Partial with 2 convolutions) kế thừa từ tư tưởng của CSPNet [3] thay thế cho C3, và cơ chế phát hiện đầu tách rời (decoupled head). Tuy nhiên, mô hình YOLOv8 gốc vẫn tồn tại một số hạn chế trong các tình huống thực tế:

Thứ nhất, module C2f trong backbone mặc dù hiệu quả nhưng chưa được tối ưu về tốc độ tính toán, đặc biệt khi triển khai trên các thiết bị có tài nguyên hạn chế.

Thứ hai, kiến trúc gốc chưa tích hợp cơ chế chú ý (attention mechanism) để giúp mô hình tập trung vào các vùng quan trọng trong ảnh, dẫn đến khả năng phân biệt đặc trưng chưa tối ưu trong các cảnh có nền phức tạp.

Thứ ba, cấu hình phát hiện mặc định chỉ sử dụng các tầng đặc trưng P3, P4, P5 tương ứng với stride 8, 16, 32 pixel, khiến mô hình gặp khó khăn khi phát hiện các đối tượng có kích thước rất nhỏ so với toàn bộ ảnh đầu vào [9].

Xuất phát từ những hạn chế trên, đề tài này tập trung nghiên cứu và đề xuất cải tiến kiến trúc YOLOv8 theo ba hướng chính. Đầu tiên, thay thế module C2f bằng Fast-C2f — một biến thể được thiết kế để giảm chi phí tính toán trong khi vẫn duy trì khả năng trích xuất đặc trưng [6]. Tiếp theo, tích hợp cơ chế chú ý SimAM (Simple, Parameter-Free Attention Module) [4] vào backbone thông qua module Fast-C2f_SimAM, giúp mô hình học được các đặc trưng phân biệt tốt hơn mà không làm tăng đáng kể số lượng tham số. Sau đó bổ sung nhánh phát hiện P2 (stride 4) và lược bỏ phát hiện nhánh P5 nhằm tăng cường khả năng phát hiện đối tượng nhỏ, đáp ứng yêu cầu của các bài toán thực tế với dữ liệu có tính đặc thù cao [9]. Sau cùng thay thế hàm kích hoạt trong các khối tích chập của

toàn bộ mô hình thành hàm kích hoạt Mish [27] và hàm tính toán thay đổi từ CIoU bằng hàm tính toán mới là WioU [25,26].

Mô hình đề xuất được xây dựng và đánh giá trên tập dữ liệu gồm 5 lớp đối tượng, sử dụng thang đo medium (scale m) làm cấu hình chuẩn, nhằm đảm bảo sự cân bằng giữa độ chính xác, tốc độ và tài nguyên tính toán. Kết quả nghiên cứu kỳ vọng đóng góp một hướng tiếp cận hiệu quả cho việc cải tiến các mô hình phát hiện đối tượng dựa trên nền tảng YOLO, đặc biệt trong các tình huống yêu cầu phát hiện vật thể nhỏ với hiệu năng thời gian thực.

1.2. Mục tiêu nghiên cứu

Mục tiêu của nghiên cứu là thiết kế và hiện thực hóa một kiến trúc mạng nơ-ron nhận diện đối tượng tiên tiến dựa trên nền tảng YOLOv8 [2] và kiến trúc phân phối đặc trưng Gold-YOLO [5], nhằm tối ưu hóa sự cân bằng giữa độ chính xác và chi phí tính toán (số lượng tham số, GFLOPs, tốc độ xử lý). Qua đó, đề tài hướng tới việc tạo ra một mô hình tinh gọn, hiệu năng cao, đặc biệt phù hợp để triển khai trên các thiết bị phần cứng ở biên (Edge Devices) [10] phục vụ cho bài toán giám sát giao thông thực tế.

Để hiện thực hóa mục tiêu này, nghiên cứu đi sâu tìm hiểu cơ sở lý thuyết về kiến trúc YOLOv8 [2], cơ chế Thu thập và Phân phối (Gather-and-Distribute) [5], nguyên lý Tích chập một phần (PConv) của FasterNet [6] và cơ chế chú ý ba chiều không chứa tham số SimAM [4]. Dựa trên nền tảng đó, đề tài tiến hành tùy biến kiến trúc mạng bằng cách thay thế khối C2f truyền thống bằng khối Fast-C2f tinh gọn, đồng thời đan cài cơ chế chú ý SimAM, thay đổi hàm kích hoạt thành Mish và hàm tính toán mất mát WioU để đưa ra mô hình hoàn chỉnh. Hệ thống sau đó được cài đặt bằng framework PyTorch [8] và tiến hành huấn luyện trực tiếp trên tập dữ liệu giao thông phức tạp KITTI [7]. Cuối cùng, hiệu năng của mô hình sẽ được đo lường và phân tích đối sánh toàn diện với mô hình gốc thông qua các chỉ số cốt lõi (mAP, Parameters, GFLOPs và FPS) nhằm chứng minh tính đúng đắn cùng sự vượt trội của giải pháp cải tiến.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu trọng tâm của đề án là các kiến trúc mạng nơ-ron tích chập (CNN) tiêu biểu trong bài toán phát hiện đối tượng một giai đoạn (One-stage Object Detection) [9], với nền tảng cốt lõi là cấu trúc mạng YOLOv8 [2] và cơ chế phân phối đặc trưng đa tỷ lệ Gold-YOLO (GD-FPN) [5]. Bên cạnh đó, đề tài đi sâu nghiên cứu phương pháp tối ưu hóa tính toán thông qua phép Tích chập một phần (PConv) ứng dụng

trong kiến trúc FasterNet [6], cùng cơ chế chú ý ba chiều không chứa tham số SimAM [4]. Kế thừa các cơ sở đó, đối tượng thiết kế cốt lõi của nghiên cứu là kiến trúc mô hình nhận diện đối tượng cải tiến mang tên GPFS_YOLO. Mô hình này nổi bật với việc tích hợp các khối trích xuất Fast-C2f_SimAM và sự dịch chuyển thang đo phát hiện chiến lược (loại bỏ nhánh độ phân giải thấp P5, bổ sung nhánh độ phân giải siêu cao P2) [11], cùng với cơ chế hàm kích hoạt Mish [29] và hàm mất mát WIoU [27, 28].

Về phạm vi nghiên cứu, đề tài tập trung giải quyết bài toán nhận diện các đối tượng nhỏ, ở khoảng cách xa (đặc thù của dữ liệu camera giao thông) [11] nhằm hướng tới việc triển khai trên các thiết bị phần cứng ở biên (Edge Devices) [12]. Quá trình huấn luyện, xác thực và kiểm thử được giới hạn thực hiện độc quyền trên tập dữ liệu giao thông KITTI [7], tập trung vào ba lớp đối tượng tham gia giao thông phổ biến nhất: ô tô (Car), người đi bộ (Pedestrian) và xe đạp/xe máy (Cyclist). Toàn bộ việc tùy biến cấu trúc mạng lưới và mô phỏng được lập trình bằng ngôn ngữ Python, sử dụng framework PyTorch [8] thuộc hệ sinh thái Ultralytics [2]. Cuối cùng, hiệu năng của mô hình GPFS_YOLO sẽ được phân tích và đối sánh trực tiếp với các mô hình nền tảng (YOLOv8 và Gold-YOLO gốc) dựa trên các tiêu chí: Độ chính xác ($mAP@0.5$ và $mAP@0.5:0.95$), Dung lượng lưu trữ (Parameters), Khối lượng tính toán (GFLOPs) và Tốc độ suy luận (FPS).

1.4. Cấu trúc báo cáo

Chương 1: Giới thiệu. Trình bày bối cảnh, đặt vấn đề, động lực nghiên cứu, từ đó xác định mục tiêu, đối tượng và phạm vi thực hiện của đề tài nhằm hướng tới tối ưu hóa mô hình nhận diện đối tượng trên thiết bị biên.

Chương 2: Cơ sở lý thuyết. Cung cấp nền tảng kiến thức tổng quan về bài toán phát hiện đối tượng (Object Detection). Phân tích chuyên sâu về kiến trúc nền tảng YOLOv8, cấu trúc phân phối đặc trưng của Gold-YOLO, cùng cơ sở lý thuyết của các module cải tiến hiện đại sẽ được áp dụng như khối Tích chập một phần (Fast-C2f), cơ chế chú ý ba chiều không chứa tham số (SimAM), hàm kích hoạt Mish và hàm mất mát WIoU.

Chương 3: Phương pháp đề xuất. Đây là chương trọng tâm trình bày chi tiết về kiến trúc mô hình cải tiến GPFS_YOLO do nhóm nghiên cứu thiết kế. Nội dung bao gồm việc phân tích các hạn chế của mô hình gốc, sơ đồ thay thế toàn diện các khối C2f bằng Fast-C2f_SimAM ở mạng xương sống (Backbone) và Fast-C2f ở mạng cổ (Neck/Head). Đồng thời, chương này giải thích chiến lược dịch chuyển thang đo phát hiện: loại bỏ nhánh P5

và bổ sung nhánh độ phân giải cao P2 nhằm tăng cường khả năng phát hiện vật thể nhỏ, cùng với thay thế hàm kích hoạt và hàm mất mát.

Chương 4: Thực nghiệm và kết quả. Mô tả chi tiết môi trường cài đặt, tham số cấu hình và tập dữ liệu thực nghiệm (5 lớp đối tượng). Trình bày các kết quả đánh giá đối sánh toàn diện giữa mô hình GPFS_YOLO và các mô hình nền tảng dựa trên các chỉ số cốt lõi: độ chính xác (mAP), số lượng tham số, khối lượng tính toán (GFLOPs) và tốc độ suy luận (FPS), từ đó đưa ra những phân tích và đánh giá sâu sắc về hiệu năng hệ thống.

Chương 5: Kết luận và hướng phát triển. Tổng kết lại những đóng góp khoa học và giá trị thực tiễn mà đề tài đã đạt được. Đồng thời, chỉ ra những hạn chế còn tồn đọng của hệ thống trong quá trình thực nghiệm và đề xuất các hướng nghiên cứu, phát triển tiếp theo trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về bài toán phát hiện đối tượng

2.1.1. Định nghĩa và các thách thức

a. Định nghĩa về bài toán phát hiện đối tượng

Phát hiện đối tượng (Object Detection) là một trong những lĩnh vực nghiên cứu nền tảng và mang tính ứng dụng cao nhất của Thị giác máy tính (Computer Vision) [9, 10]. Mục tiêu của bài toán này là thiết kế các thuật toán hoặc mạng nơ-ron có khả năng quét qua một bức ảnh hoặc video để tự động tìm kiếm, khoanh vùng và nhận diện các đối tượng mục tiêu. Về bản chất, phát hiện đối tượng là sự kết hợp chặt chẽ của ba nhiệm vụ thị giác cốt lõi [11, 12]:

- Phân loại hình ảnh (Image Classification): Trả lời cho câu hỏi "Đối tượng trong vùng nhìn thấy là gì?". Thuật toán sẽ trích xuất các đặc trưng trực quan và gán một nhãn phân lớp (class label) cụ thể cho đối tượng (ví dụ: ô tô, người đi bộ, xe đạp) kèm theo độ tin cậy (confidence score) xác định mức độ chắc chắn của dự đoán.
- Định vị đối tượng (Object Localization): Trả lời cho câu hỏi "Đối tượng đó nằm ở đâu?". Nhiệm vụ này yêu cầu hệ thống phải xác định chính xác tọa độ không gian của đối tượng trong bức ảnh, thường được biểu diễn dưới dạng một hộp giới hạn (Bounding Box) với 4 giá trị cơ bản: tọa độ tâm (x, y), chiều rộng (w) và chiều cao (h).
- Phát hiện đa đối tượng (Multi-object Detection): Trong các ngữ cảnh thực tế (như môi trường giao thông), một khung hình hiếm khi chỉ chứa một đối tượng duy nhất. Do đó, hệ thống cần có khả năng thực hiện đồng thời việc phân loại và định vị cho hàng chục, thậm chí hàng trăm đối tượng khác nhau (thuộc nhiều nhãn lớp khác nhau) xuất hiện cùng lúc trên cùng một khung hình.

b. Các thách thức hiện nay trong phát hiện đối tượng

Mặc dù sự ra đời của các mạng nơ-ron tích chập (CNN) và học sâu (Deep Learning) đã đưa hiệu năng nhận diện lên một tầm cao mới, bài toán phát hiện đối tượng trong môi

trường thực tế (đặc biệt là lĩnh vực giám sát giao thông) vẫn phải đối mặt với nhiều thách thức lớn:

- Sự biến thiên về kích thước (Scale Variation) và đối tượng nhỏ: Các vật thể trong không gian 3D khi chiếu lên mặt phẳng ảnh 2D sẽ có kích thước thay đổi liên tục phụ thuộc vào khoảng cách đến camera. Các đối tượng ở xa (như xe cộ ở cuối đường) thường có kích thước rất nhỏ, mang ít đặc trưng không gian (spatial features). Các mạng CNN truyền thống thường làm mất đi các đặc trưng vi mô này sau nhiều lớp gộp (pooling), dẫn đến tỷ lệ bỏ sót (false negative) cao [11].
- Hiện tượng che khuất (Occlusion) và điều kiện ánh sáng: Trong môi trường giao thông thực tế, các phương tiện và người đi bộ thường xuyên bị che khuất một phần bởi các vật thể khác, hoặc bị ảnh hưởng bởi điều kiện thời tiết (mưa, sương mù), ánh sáng (chói sáng, thiếu sáng), làm nhiễu loạn hình thái của đối tượng và gây nhầm lẫn cho việc trích xuất đặc trưng [7, 28].
- Sự đánh đổi giữa Độ chính xác (Accuracy) và Tốc độ suy luận (Inference Speed): Để tăng khả năng nhận diện các vật thể nhỏ và khó, các mô hình học sâu thường có xu hướng tăng cường chiều sâu và chiều rộng của mạng lưới. Điều này làm gia tăng hàm lượng tham số và khối lượng tính toán (GFLOPs). Tuy nhiên, đối với các ứng dụng yêu cầu thời gian thực như xe tự lái hay hệ thống giao thông thông minh được triển khai trên phần cứng ở biên (Edge Devices) có tài nguyên hạn hẹp, việc sử dụng các mô hình quá lớn sẽ gây ra độ trễ (latency) không thể chấp nhận được [12].

2.1.2. Các hướng tiếp cận trong phát hiện đối tượng

Sự phát triển vượt bậc của mạng nơ-ron tích chập (CNN) đã định hình lại hoàn toàn các phương pháp giải quyết bài toán phát hiện đối tượng [9, 10]. Hiện nay, các thuật toán học sâu trong lĩnh vực này được phân chia thành hai khuynh hướng thiết kế chủ đạo: Mô hình hai giai đoạn (Two-stage Detectors) và Mô hình một giai đoạn (One-stage Detectors).

a. Mô hình hai giai đoạn (Two-stage Detectors)

Mô hình hai giai đoạn tiếp cận bài toán bằng cách chia luồng xử lý thành hai bước tuần tự và biệt lập:

- Trích xuất vùng đề xuất (Region Proposal): Thuật toán sẽ quét qua toàn bộ bức ảnh để tìm ra hàng ngàn "vùng ứng viên" (Region of Interest - RoI) có khả năng chứa đối tượng, thông qua các kỹ thuật như Selective Search hoặc Mạng đề xuất vùng (Region Proposal Network - RPN).
- Phân loại và tinh chỉnh hộp giới hạn: Các vùng ứng viên này sau đó được trích xuất đặc trưng độc lập và đưa vào một mạng CNN thứ hai để thực hiện việc phân loại nhãn đối tượng (Classification) và hồi quy tinh chỉnh lại tọa độ hộp giới hạn (Bounding Box Regression).
- Các mô hình tiêu biểu: R-CNN (Region-based CNN) [13] là kiến trúc tiên phong cho trường phái này, sau đó được tối ưu hóa qua các phiên bản Fast R-CNN, Faster R-CNN [14] và Mask R-CNN.
- Đặc điểm: Nhờ cơ chế quét và tinh chỉnh kỹ lưỡng từng vùng đề xuất, mô hình hai giai đoạn đạt được độ chính xác (Accuracy) rất cao, đặc biệt tốt trong việc xử lý các vật thể nhỏ hoặc bị che khuất. Tuy nhiên, kiến trúc phức tạp và tính toán lặp lại khiến chúng có tốc độ suy luận chậm và tốn nhiều tài nguyên bộ nhớ.

b. Mô hình một giai đoạn (One-stage Detectors)

Khắc phục nhược điểm về tốc độ của mô hình hai giai đoạn, trường phái một giai đoạn loại bỏ hoàn toàn bước trích xuất vùng ứng viên. Hệ thống coi việc phát hiện đối tượng là một bài toán hồi quy (regression problem) duy nhất.

- Cơ chế hoạt động: Bức ảnh đầu vào sẽ được đưa qua mạng nơ-ron đúng một lần (single pass / end-to-end). Mạng lưới phân chia bức ảnh thành một ma trận lưới (grid) và tiến hành dự đoán trực tiếp đồng thời cả tọa độ hộp giới hạn lẫn xác suất phân lớp cho từng ô không gian đó.

- Tiêu biểu nhất là họ mô hình YOLO (You Only Look Once) [1], SSD (Single Shot MultiBox Detector) [15] và RetinaNet [16].
- Đặc điểm: Tốc độ suy luận của mô hình một giai đoạn cực kỳ nhanh, kiến trúc mạng tinh gọn, rất lý tưởng cho các ứng dụng đòi hỏi xử lý thời gian thực (real-time). Trong quá khứ, các mô hình này thường yếu thế hơn khi nhận diện vật thể nhỏ. Tuy nhiên, sự ra đời của các kiến trúc hiện đại (như YOLOv8 [2] với các module FPN [17], PANet [18] hay cơ chế Attention [4, 25]) đã thu hẹp đáng kể khoảng cách này.

c. So sánh sự đánh đổi giữa độ chính xác và tốc độ (Accuracy vs. Speed Trade-off)

Sự khác biệt cốt lõi giữa hai hướng tiếp cận chính là sự đánh đổi (trade-off) giữa Độ chính xác (mAP) và Tốc độ xử lý khung hình (FPS):

- Về độ chính xác: Two-stage Detectors (như Faster R-CNN [14]) thường nhỉnh hơn về chỉ số mAP, ít gặp hiện tượng định vị sai lệch nhờ cơ chế sàng lọc ứng viên hai bước. Ngược lại, One-stage Detectors dễ gặp tình trạng mất cân bằng dữ liệu giữa vùng có đối tượng và vùng nền (hiện tượng class imbalance) [16], dẫn đến độ chính xác tổng thể thấp hơn một chút.
- Về tốc độ và độ trễ: One-stage Detectors (như YOLO [1]) chiếm ưu thế tuyệt đối. Nhờ tính toán đồng thời trên toàn bộ bức ảnh thay vì từng vùng cắt lẻ tẻ, YOLO có thể đạt ngưỡng hàng chục đến hàng trăm FPS, trong khi Faster R-CNN [14] thường chỉ đạt mức dưới 10 FPS trên cùng một thiết lập phần cứng.

Đối với bài toán giám sát giao thông thực tế, nơi hệ thống phải đối mặt với dòng phương tiện di chuyển liên tục và thường được triển khai trên các thiết bị biên (Edge Devices) giới hạn tài nguyên [12], tốc độ suy luận thời gian thực là yêu cầu tiên quyết. Do đó, hướng tiếp cận One-stage Detector (cụ thể là nền tảng YOLOv8 [2] / Gold-YOLO [5]) đã được lựa chọn làm cơ sở phát triển cho mô hình GPFS_YOLO. Bằng cách tích hợp các cải tiến về mặt cấu trúc (Fast-C2f [6], SimAM [4], nhánh P2 [11]), đề tài hướng tới việc giữ nguyên lợi thế tốc độ của mô hình một giai đoạn, đồng thời bứt phá về độ chính xác nhằm giải quyết triệt để nhược điểm nhận diện vật thể nhỏ.

2.1.3. Feature Pyramid Network (FPN) và đa tầng đặc trưng

a. Tại sao cần phát hiện đa tỷ lệ (Multi-scale detection)?

Sự biến thiên về kích thước (Scale Variation) của đối tượng là một trong những thách thức kinh điển nhất trong thị giác máy tính [9]. Trong mạng nơ-ron tích chập (CNN), khi dữ liệu ảnh đi qua các lớp sâu hơn, các phép gộp (pooling) hoặc tích chập có bước trượt (strided convolution) sẽ làm giảm dần độ phân giải không gian của bản đồ đặc trưng (feature map). Điều này dẫn đến một nghịch lý trong thiết kế mạng CNN truyền thống [11]:

- Các tầng nông (Shallow layers): Có độ phân giải cao, giữ được nhiều chi tiết không gian chính xác, ranh giới rõ ràng (rất tốt để định vị) nhưng lại thiếu thông tin ngữ nghĩa (tức là mạng chưa hiểu rõ vật thể đó là gì).
- Các tầng sâu (Deep layers): Có độ phân giải rất thấp nhưng mang thông tin ngữ nghĩa cực kỳ phong phú và trường tiếp nhận (receptive field) rộng (rất tốt để phân loại). Tuy nhiên, do kích thước quá nhỏ, các đặc trưng của những vật thể bé thường bị "xóa sổ" hoàn toàn ở các tầng này [11].

Để giải quyết bài toán này, Mạng kim tự tháp đặc trưng (Feature Pyramid Network - FPN) ra đời [17]. Cơ chế cốt lõi của FPN là kết hợp luồng thông tin từ dưới lên (bottom-up) với luồng thông tin từ trên xuống (top-down) thông qua các kết nối ngang (lateral connections). Nhờ đó, FPN truyền các đặc trưng ngữ nghĩa mạnh mẽ từ các tầng sâu ngược lên các tầng nông có độ phân giải cao. Quá trình này tạo ra một "kim tự tháp" các bản đồ đặc trưng, nơi mọi tầng (dù lớn hay nhỏ) đều sở hữu sự cân bằng hoàn hảo giữa độ chi tiết không gian và độ sâu ngữ nghĩa, cho phép hệ thống phát hiện đối tượng đa tỷ lệ một cách chính xác [17, 18].

b. Ý nghĩa của các tầng đặc trưng P2, P3, P4, P5

Trong cấu trúc của các mô hình phát hiện đối tượng hiện đại (như họ YOLO [1, 2]), các tầng đặc trưng được ký hiệu là P_i , trong đó i đại diện cho mức độ giảm lấy mẫu (downsampling) so với ảnh gốc theo công thức: $\text{Stride} = 2^i$. Cụ thể, ý nghĩa và vai trò của từng tầng như sau:

- Tầng P2 ($\text{Stride} = 2^2 = 4$): Bản đồ đặc trưng có kích thước bằng 1/4 so với ảnh gốc. Đây là tầng có độ phân giải siêu cao (High-resolution), chứa đựng những đặc trưng

vì mô sắc nét nhất như góc cạnh, ranh giới. Tầng này đặc biệt quan trọng để phát hiện các vật thể có kích thước cực nhỏ (như phương tiện ở rất xa, người đi bộ nhỏ bé trong tập dữ liệu KITTI [7]). Tuy nhiên, do kích thước ma trận lớn, việc tính toán trên P2 thường tiêu tốn rất nhiều tài nguyên.

- Tầng P3 (Stride = $2^3 = 8$): Bản đồ đặc trưng có kích thước bằng 1/8 ảnh gốc. Tầng này cung cấp sự cân bằng tốt giữa chi tiết và ngữ nghĩa, thường được thiết lập làm nhánh phát hiện mặc định nhỏ nhất trong các mô hình YOLO tiêu chuẩn để nhận diện vật thể nhỏ [2].
- Tầng P4 (Stride = $2^4 = 16$): Bản đồ đặc trưng có kích thước bằng 1/16 ảnh gốc. Với trường tiếp nhận vừa đủ lớn, nhánh P4 đảm nhiệm vai trò phát hiện các đối tượng có kích thước tầm trung.
- Tầng P5 (Stride = $2^5 = 32$): Bản đồ đặc trưng có kích thước bằng 1/32 ảnh gốc (tầng sâu nhất). Nó có độ phân giải thấp nhất nhưng bao quát toàn bộ bức ảnh, mang lại trường tiếp nhận không lờ. Nhánh P5 được sinh ra chuyên biệt để nhận diện các vật thể có kích thước rất lớn, chiếm phần lớn khung hình.

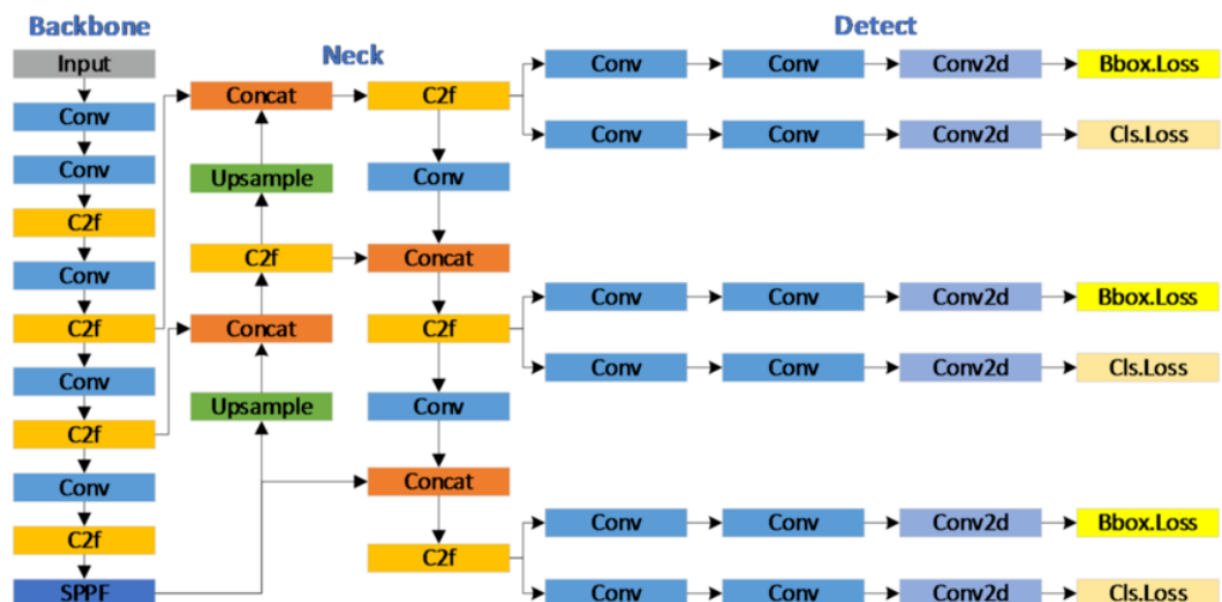
2.2. Kiến trúc YOLOv8 chuẩn

2.2.1. Tổng quan kiến trúc

YOLOv8 (You Only Look Once version 8) [2] là một trong những mạng nơ-ron phát hiện đối tượng một giai đoạn (one-stage detector) tiên tiến nhất, được thiết kế theo trường phái không sử dụng mỏ neo (Anchor-free). Kiến trúc tổng thể của mô hình được chia thành ba module chính: Mạng xương sống (Backbone), Mạng cổ (Neck) và Đầu phát hiện (Head).

a. Sơ đồ kiến trúc tổng thể

Để dễ hình dung luồng luân chuyển dữ liệu đa tỷ lệ từ đầu vào đến đầu ra, kiến trúc YOLOv8 có thể được mô hình hóa qua sơ đồ khối sau:



Hình 1: Sơ đồ khối của YOLOv8

b. Phân tích các thành phần kiến trúc

- **Mạng xương sống (Backbone):** Đóng vai trò là bộ trích xuất đặc trưng (Feature Extractor). Kế thừa triết lý của CSPNet [3], YOLOv8 sử dụng các lớp Tích chập tiêu chuẩn (Conv) để giảm độ phân giải không gian (downsampling) và áp dụng khối C2f (Cross Stage Partial Bottleneck) để trích xuất thông tin. Khối C2f tạo ra các kết nối tắt (skip connections) phong phú, giúp gradient lan truyền tốt hơn. Tầng sâu nhất của Backbone được chốt lại bằng khối SPPF (Spatial Pyramid Pooling - Fast) kế thừa từ kiến trúc SPP [19] giúp mở rộng trường tiếp nhận toàn cục mà không làm chậm tốc độ xử lý.
- **Mạng cổ (Neck):** Có nhiệm vụ hòa trộn các đặc trưng từ tầng nông đến tầng sâu. Cấu trúc Neck của YOLOv8 áp dụng tư tưởng của PANet (Path Aggregation Network) [18] nhưng được tinh giản. Mạng thực hiện phép nối (Concat) để hợp nhất đặc trưng từ luồng Top-down và Bottom-up, sau đó đưa qua các khối C2f để làm mịn thông tin.
- **Đầu phát hiện (Head):** YOLOv8 sử dụng cấu trúc Đầu phát hiện phân tách (Decoupled Head) [2], nghĩa là tách biệt hoàn toàn hai nhánh tính toán: một nhánh dự đoán nhãn lớp (Classification) và một nhánh dự đoán tọa độ hộp giới hạn

(Bounding Box Regression). Mạng dự đoán kết quả trên 3 tầng đặc trưng P3, P4 và P5.

c. Công thức toán học cốt lõi (Hàm mất mát)

Bằng việc loại bỏ các hộp mờ neo (Anchor-free), cơ chế tính toán tổn thất của YOLOv8 trở nên trực tiếp và linh hoạt hơn. Tổng hàm mất mát (Total Loss) trong quá trình huấn luyện được định nghĩa là sự tổng hòa của ba thành phần:

$$\mathcal{L}_{total} = \lambda_{cls}\mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{IoU} + \lambda_{dfl}\mathcal{L}_{DFL} \quad (2.1)$$

Trong đó:

$\mathcal{L}_{cls}$: (Classification Loss): Hàm mất mát phân loại, thường sử dụng Binary Cross-Entropy (BCE) để tính toán sai số giữa xác suất dự đoán nhãn lớp và nhãn thực tế (Ground Truth).

$\mathcal{L}_{IoU}$: (Bounding Box Regression Loss): Hàm mất mát hồi quy hộp giới hạn. YOLOv8 sử dụng Complete IoU (CIoU) [20] để tối ưu hóa sự chồng lấp giữa hộp dự đoán và hộp thực tế. CIoU không chỉ xét đến diện tích phần giao nhau mà còn xét đến khoảng cách tâm và tỷ lệ khung hình:

$$\mathcal{L}_{IoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v \quad (2.2)$$

(Với ρ là khoảng cách Euclid giữa hai tâm, c là đường chéo của hộp bao quanh nhỏ nhất, và v là tham số đo lường sự nhất quán của tỷ lệ khung hình).

$\mathcal{L}_{DFL}$: (Distribution Focal Loss): Hàm mất mát phân phối tiêu điểm [21]. Đây là một cải tiến toán học quan trọng của YOLOv8, giúp mô hình hóa các tọa độ giới hạn của Bounding Box dưới dạng một phân phối xác suất liên tục thay vì các giá trị rời rạc, làm mịn các dự đoán ở mức độ pixel và giúp hệ thống bắt viền đối tượng sắc nét hơn.

2.2.2. Backbone: Conv + C2f + SPPF

Mạng xương sống (Backbone) của YOLOv8 đảm nhận vai trò trích xuất đặc trưng cốt lõi từ hình ảnh đầu vào. Nó chuyển đổi không gian điểm ảnh (pixel) thô thành các bản đồ đặc trưng (feature maps) chứa thông tin từ các chi tiết cấp thấp (cạnh, góc) đến ngữ nghĩa cấp cao (đối tượng) [10]. Backbone của YOLOv8 [2] được cấu thành từ ba thành phần chính: Chuỗi giảm lấy mẫu P1 $\rightarrow$ P5, khối trích xuất C2f và khối gộp không gian SPPF.

a. Chuỗi trích xuất phân cấp (P1 $\rightarrow$ P5)

Để mô hình có khả năng nhận diện các đối tượng ở nhiều kích thước khác nhau, hình ảnh đầu vào sẽ được đưa qua một chuỗi các lớp tích chập tiêu chuẩn (Conv) có bước trượt (stride) $S = 2$. Kỹ thuật này giúp giảm lấy mẫu (downsampling) không gian hình ảnh theo

từng cấp độ. Giả sử đầu vào có kích thước (H_{in}, W_{in}) và sử dụng lớp tích chập có kích thước nhân (kernel) $K = 3$, padding $P = 1$, kích thước bản đồ đặc trưng đầu ra được tính theo công thức [10]:

$$H_{out} = \left\lfloor \frac{H_{in} - K + 2P}{s} \right\rfloor + 1 = \left\lfloor \frac{H_{in} - 3 + 2(1)}{2} \right\rfloor + 1 = \frac{H_{in}}{2} \quad (2.3)$$

$$W_{out} = \frac{W_{in}}{2} \quad (2.4)$$

Sau mỗi lớp Conv (với Stride=2), độ phân giải bị giảm đi một nửa trong khi số kênh (channels) đặc trưng được mở rộng. Quá trình này tạo ra 5 tầng đặc trưng chính thức, ký hiệu từ P1 đến P5:

- P1: Tỷ lệ 1/2 ảnh gốc.
- P2: Tỷ lệ 1/4 ảnh gốc.
- P3: Tỷ lệ 1/8 ảnh gốc (Bắt đầu được sử dụng để phát hiện vật thể nhỏ).
- P4: Tỷ lệ 1/16 ảnh gốc (Phát hiện vật thể trung bình).
- P5: Tỷ lệ 1/32 ảnh gốc (Tầng sâu nhất, phát hiện vật thể lớn).

b. Module C2f (Cross Stage Partial Bottleneck với 2 tích chập)

Sau các bước giảm lấy mẫu, YOLOv8 sử dụng khối C2f để tinh luyện và học các đặc trưng phức tạp. Đây là bản nâng cấp đáng giá từ khối C3 của các thế hệ trước, mang lại sự cân bằng tối ưu giữa tốc độ tính toán và luồng lan truyền đạo hàm (gradient flow) [2].

Cấu trúc và nguyên lý hoạt động:

Khối C2f dựa trên triết lý mạng CSPNet (Cross Stage Partial Network) [3]. Dữ liệu đầu vào X trước tiên đi qua một lớp tích chập 1×1 sau đó được tách (split/chunk) thành hai nhánh có số kênh bằng nhau, gọi là Y_0 và Y_1

Nhánh Y_1 tiếp tục được đưa qua một chuỗi các khối thắt cổ chai (Bottleneck), sinh ra các đầu ra trung gian $Y_2, Y_3, \dots, Y_{n+1}$. Điểm đột phá của C2f là nó ghép nối (Concat) nhánh Y_0 , nhánh Y_1 cùng tất cả các đầu ra trung gian của các khối Bottleneck lại với nhau trước khi qua lớp tích chập cuối cùng.

Công thức toán học của module C2f:

Gọi $\mathcal{B}_i$ là hàm biểu diễn hoạt động của khối Bottleneck thứ i , ta có:

$$Y_2 = \mathcal{B}_1(Y_1) \quad (2.5)$$

$$Y_3 = \mathcal{B}_2(Y_2) \quad (2.6)$$

...

Đầu ra cuối cùng của khối C2f được tính bằng:

$$Out_{C2f} = \text{Conv}_{1 \times 1}(\text{Concat}(Y_0, Y_1, Y_2, \dots, Y_{n+1})) \quad (2.7)$$

Vai trò: Nhờ việc ghép nối liên tục ở nhiều cấp độ, thông tin gradient khi lan truyền ngược (backpropagation) được bảo toàn phong phú [3]. Nó khắc phục hiện tượng suy biến đạo hàm (gradient vanishing) ở các mạng sâu, giúp mô hình hội tụ nhanh và đạt độ chính xác cao hơn mà không làm tăng quá nhiều tham số.

c. Module SPPF (Spatial Pyramid Pooling - Fast)

Nằm ở vị trí cuối cùng của mạng xương sống (ngay sau tầng P5), module SPPF có nhiệm vụ mở rộng trường tiếp nhận toàn cục (global receptive field) và thu thập ngữ cảnh không gian đa tỉ lệ.

Cấu trúc và vai trò:

Thay vì dùng các bộ gộp song song có kích thước nhân lớn (như SPP truyền thống [19] dùng 5×5 , 9×9 , 13×13), SPPF sử dụng 3 lớp Gộp cực đại (MaxPool2D) kích thước 5×5 mắc nối tiếp nhau. Đầu ra của khối là sự kết hợp của bản đồ đặc trưng gốc và 3 bản đồ sau khi gộp tuần tự.

Biểu diễn toán học:

Gọi X_{in} là đầu vào của SPPF sau khi đã qua một lớp Conv. Hoạt động của 3 khối MaxPool nối tiếp:

$$P_1 = \text{MaxPool}_{5 \times 5}(X_{in}) \quad (2.8)$$

$$P_2 = \text{MaxPool}_{5 \times 5}(P_1) \quad (2.9)$$

$$P_3 = \text{MaxPool}_{5 \times 5}(P_2) \quad (2.10)$$

Theo tính chất vùng đón nhận (receptive field), hai lớp gộp 5×5 nối tiếp tương đương với một lớp 9×9 , và ba lớp 5×5 nối tiếp tương đương một lớp 13×13 . Đầu ra cuối cùng là:

$$Out_{SPPF} = \text{Conv}_{1 \times 1}(\text{Concat}(X_{in}, P_1, P_2, P_3)) \quad (2.11)$$

Nhờ thiết kế nối tiếp này, SPPF tổng hợp được thông tin đa tỉ lệ một cách nhanh chóng (Fast) hơn rất nhiều so với kỹ thuật SPP cũ, tối ưu hóa đáng kể chi phí tính toán (FLOPs) cho mô hình trước khi chuyển dữ liệu sang phân hệ Neck.

2.2.3. Mạng kết hợp (Neck) và Đầu phát hiện (Head) truyền thống

Sau khi hình ảnh đi qua mạng xương sống (Backbone) và trích xuất được 3 tầng đặc trưng trọng tâm là P3, P4 và P5, dữ liệu sẽ được chuyển tiếp vào Mạng kết hợp đặc trưng (Neck) và cuối cùng là Đầu phát hiện (Head) để đưa ra dự đoán.

a. Mạng cổ (Neck): Luồng FPN truyền thống (Upsample + Concat từ P5 → P3)

Mục đích của mạng Neck là giải quyết sự chênh lệch về mặt thông tin giữa các tầng: tầng nông (P3) có độ phân giải cao nhưng thiếu ngữ nghĩa, trong khi tầng sâu (P5) giàu ngữ nghĩa nhưng lại mất đi các chi tiết định vị không gian. YOLOv8 chuẩn áp dụng cấu trúc kết hợp FPN (Feature Pyramid Network) [17] kết hợp với luồng từ dưới lên (Bottom-up) của PANet [18] để hòa trộn các đặc trưng này.

Quá trình truyền thông tin Top-Down (Từ trên xuống):

Luồng này mang các thông tin ngữ nghĩa mạnh mẽ từ tầng sâu nhất (P5) ngược lên các tầng nông hơn (P4, P3).

- Đặc trưng P5 đầu tiên được tăng kích thước (Upsample) lên gấp đôi thông qua phép nội suy lân cận gần nhất (Nearest Neighbor Interpolation) để có cùng độ phân giải không gian với P4.
- Bản đồ đặc trưng sau khi Upsample được ghép nối dọc theo trục kênh (Concat) với bản đồ đặc trưng P4 gốc từ Backbone.
- Hỗn hợp này tiếp tục được đưa qua một khối C2f để làm mịn và trộn lẫn thông tin, tạo ra đặc trưng lai F_4
- Tương tự, F_4 tiếp tục được Upsample và ghép nối với P3, đi qua C2f để tạo ra F_3

Biểu diễn toán học của luồng Top-Down:

Gọi $Up(\cdot)$ là hàm nội suy tăng kích thước gấp 2 lần, đặc trưng tổng hợp tại các tầng được tính như sau:

$$F_4 = C2f\left(\text{Concat}(P_4, Up(P_5))\right) \quad (2.12)$$

$$F_3 = \text{C2f}\left(\text{Concat}(P_3, \text{Up}(F_4))\right) \quad (2.13)$$

(Luồng Bottom-Up sau đó sẽ đưa thông tin từ F_3 ngược lên lại để tạo ra các đầu ra cuối cùng N_3, N_4, N_5 đưa vào lớp Detect).

b. Đầu phát hiện (Head): Decoupled Head tại P3, P4, P5

Ở các phiên bản YOLO cũ (như YOLOv3 đến YOLOv5), đầu phát hiện là dạng gộp (Coupled Head), nghĩa là chỉ dùng một lớp tích chập duy nhất để xuất ra đồng thời cả xác suất phân loại (Class) và tọa độ hộp giới hạn (Bounding Box). Tuy nhiên, bài toán phân loại và bài toán định vị có bản chất hoàn toàn khác nhau: phân loại cần đặc trưng ngữ nghĩa toàn cục, trong khi định vị cần chi tiết không gian chính xác ở viền đối tượng. Sự gộp chung này gây ra hiện tượng xung đột (misalignment) [22].

Nhận thức được điều này, YOLOv8 áp dụng cấu trúc Đầu phát hiện phân tách (Decoupled Head) [22]. Tại mỗi thang đo phát hiện (N_3, N_4, N_5 tương ứng với P3, P4, P5), bản đồ đặc trưng sẽ được chẻ làm hai nhánh song song và độc lập hoàn toàn:

- **Nhánh Phân loại (Classification Branch):** Sử dụng các lớp Conv để trích xuất đặc trưng ngữ nghĩa, cuối cùng dùng hàm kích hoạt Sigmoid để dự đoán xác suất đối tượng thuộc về C lớp khác nhau. Đầu ra có số kênh là C
- **Nhánh Hồi quy tọa độ (Regression Branch):** Tập trung dự đoán vị trí hộp giới hạn. Do YOLOv8 theo trường phái Anchor-free, nhánh này không dự đoán độ lệch so với hộp mỏ neo, mà dự đoán trực tiếp khoảng cách từ tâm đối tượng đến 4 cạnh (trái, trên, phải, dưới) của Bounding Box. Đầu ra có dạng $4 \times \text{reg_max}$ (phục vụ cho hàm mất mát DFL [21]).

Biểu diễn toán học của Decoupled Head:

Cho bản đồ đặc trưng đầu vào N_i tại thang đo i các dự đoán được tính bằng:

$$P_{cls}^{(i)} = \sigma(\text{Conv}_{cls}(N_i)) \quad (\text{Xác suất phân lớp}) \quad (2.14)$$

$$B_{reg}^{(i)} = \text{Conv}_{reg}(N_i) \quad (\text{Tọa độ hộp giới hạn}) \quad (2.15)$$

Trong đó σ là hàm Sigmoid, các trọng số Conv_{cls} và Conv_{reg} không chia sẻ với nhau.

Mạng chuẩn sẽ xuất ra 3 kết quả dự đoán tại 3 độ phân giải khác nhau:

- P3 (Stride 8): Chuyên nhận diện vật thể nhỏ.

- P4 (Stride 16): Chuyên nhận diện vật thể trung bình.
- P5 (Stride 32): Chuyên nhận diện vật thể lớn.

2.2.4. Hạn chế của YOLOv8 chuẩn

Mặc dù YOLOv8 là bước tiến lớn của dòng YOLO, kiến trúc chuẩn vẫn bộc lộ các điểm nghẽn khi triển khai trong môi trường giao thông thực tế:

- Khó khăn khi phát hiện vật thể nhỏ (Small Object Detection): Kiến trúc chuẩn chỉ sử dụng các tầng P_3 , P_4 , P_5 (stride 8, 16, 32). Do thiếu tầng đặc trưng P_2 (stride 4) có độ phân giải cao, các đối tượng nhỏ ở khoảng cách xa dễ bị thất thoát thông tin sau các lớp giảm lấy mẫu của Backbone, dẫn đến tình trạng bỏ sót đối tượng hoặc dự đoán khung giới hạn (bounding box) thiếu chính xác [11].
- Lãng phí tài nguyên ở tầng đặc trưng sâu (P_5): Tầng P_5 (stride 32) được thiết kế cho các vật thể khổng lồ. Tuy nhiên, trong ngữ cảnh giao thông (như tập dữ liệu KITTI [7]), các vật thể này rất hiếm gặp. Việc duy trì nhánh P_5 gây lãng phí tài nguyên tính toán (FLOPs) và tham số mô hình, gây khó khăn khi triển khai thực tế trên các thiết bị biên (Edge Devices) [12].
- Thiếu cơ chế học đặc trưng thích ứng (Attention): YOLOv8 chuẩn sử dụng các lớp tích chập tiêu chuẩn, xử lý mọi vùng trên ảnh một cách "bình đẳng". Việc thiếu các cơ chế chú ý khiến mô hình khó phân biệt được vùng chứa đối tượng quan trọng và vùng nhiễu nền phức tạp, làm giảm khả năng nhận diện trong điều kiện ánh sáng thay đổi hoặc đối tượng bị che khuất (occlusion) [24, 25].
- Hiệu suất trên phần cứng biên (Edge Computing): Cấu trúc khối C2f truyền thống dù hiệu quả về độ chính xác nhưng lại tiêu tốn đáng kể băng thông bộ nhớ và năng lực tính toán. Điều này tạo ra rào cản khi triển khai mô hình trên các thiết bị nhúng vốn bị giới hạn về tài nguyên phần cứng, đòi hỏi các giải pháp tối ưu hóa kiến trúc tinh gọn hơn [6, 14].

2.3. Kiến trúc YOLOv8-Gold

2.3.1. Tổng quan Gold-YOLO và động lực ra đời

Trong những năm gần đây, các mô hình thuộc hệ sinh thái YOLO (You Only Look Once) [1] như YOLOv6, YOLOv8 [2], hay YOLOv9 đã đạt được những bước tiến vượt bậc về sự cân bằng giữa tốc độ xử lý và độ chính xác trong bài toán nhận diện đối tượng theo thời gian thực (Real-time Object Detection). Tuy nhiên, hầu hết các kiến trúc này vẫn dựa trên các mạng cấu trúc cổ điển để dung hợp đặc trưng đa quy mô, cụ thể là FPN (Feature Pyramid Network) [17] và PANet (Path Aggregation Network) [18]. Mặc dù PANet đã cải tiến khả năng truyền dẫn thông tin bằng cách bổ sung luồng dữ liệu từ dưới lên (Bottom-up path), cơ chế này vẫn tồn tại những hạn chế cốt lõi về mặt kiến trúc.

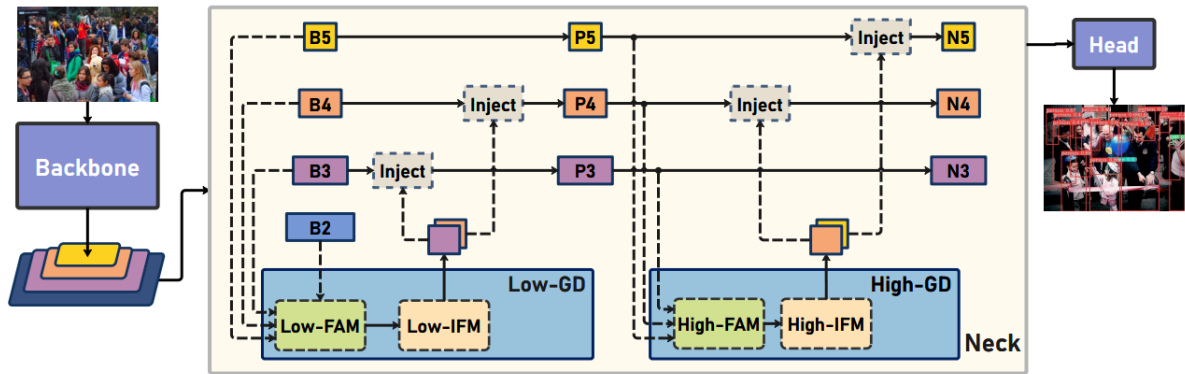
Động lực ra đời của Gold-YOLO:

- Sự đứt gãy thông tin đa quy mô (Information Loss in Multi-scale Fusion): Trong cấu trúc PANet truyền thống của YOLOv8, việc trao đổi thông tin giữa các tầng đặc trưng không liền mạch mà phải đi qua các bước trung gian (ví dụ: thông tin từ tầng đặc trưng nông P3 muốn truyền đến tầng sâu P5 phải đi tuần tự qua P4). Quá trình truyền dẫn gián tiếp này làm suy giảm, thậm chí làm thất thoát các thông tin cục bộ chất lượng cao (các chi tiết cạnh, góc, vân bề mặt vốn rất quan trọng để nhận diện vật thể nhỏ). Ngược lại, thông tin ngữ cảnh toàn cục từ P5 khi hạ lấy mẫu xuống P3 cũng bị mờ nhạt đi đáng kể.
- Hạn chế của tương tác cục bộ: Các mối liên kết trong PANet chủ yếu dựa trên các phép toán tích chập (Convolution) cục bộ kết hợp với nối chuỗi (Concatenate) hoặc cộng phần tử (Element-wise addition). Phép toán này giới hạn khả năng thiết lập các mối quan hệ ngữ cảnh tầm xa (Long-range dependencies) giữa các vật thể ở các quy mô khác nhau, khiến mô hình dễ nhầm lẫn hoặc bỏ sót đối tượng trong các môi trường nền phức tạp hoặc bị che khuất một phần.

Để giải quyết triệt để bài toán nghẽn cổ chai thông tin (information bottleneck) nêu trên, kiến trúc Gold-YOLO (được giới thiệu bởi Huawei Noah's Ark Lab) [5] đã đề xuất một cơ chế phân phối thông tin hoàn toàn mới mang tên GD-Neck (Gather-and-Distribute Neck). Thay vì truyền thông tin tuần tự theo kiểu "bắc cầu" như PANet, GD-Neck tiếp cận theo phương pháp "Tập trung và Phân phối":

- Giai đoạn Tập trung (Gather): Mô hình thu thập toàn bộ các đặc trưng từ tất cả các tầng khác nhau trong Backbone thông qua các khối gom đặc trưng (Feature Alignment Module - FAM), sau đó dung hợp chúng lại thành một khối thông tin thống nhất, mang tính toàn cục (Global Information).
- Giai đoạn Phân phối (Distribute): Khối đặc trưng toàn cục sau khi được tổng hợp sẽ được chia sẻ và tiêm trực tiếp (Inject) vào tất cả các tầng đặc trưng tương ứng ở nhánh Head thông qua module tiêm thông tin (Information Injection Module - IFM).

Nhờ vào cơ chế tiêm thông tin trực tiếp này, mọi tầng dự đoán từ vật thể nhỏ đến lớn đều nhận được thông tin bổ trợ từ các tầng khác một cách đồng thời, giảm thiểu tối đa hiện tượng thất thoát thông tin trong quá trình truyền dẫn đa quy mô.



Hình 2: Sơ đồ khối của kiến trúc Gold-YOLO

2.3.2. Mạng trích xuất đặc trưng Backbone: Giữ nguyên Conv + C2f + SPPF

Trong mô hình baseline Gold-YOLO [5], mạng trích xuất đặc trưng nền tảng (Backbone) được kế thừa và giữ nguyên hoàn toàn theo cấu trúc chuẩn của YOLOv8 [2]. Việc duy trì nguyên vẹn thành phần này đóng vai trò như một môi trường đối chứng kiểm soát (controlled baseline). Điều này cho phép nhóm nghiên cứu đánh giá và cô lập một cách chính xác hiệu quả cải tiến mang lại từ cơ chế tập trung và phân phối thông tin (GD-Neck) ở nhánh Neck, loại bỏ hoàn toàn các yếu tố gây nhiễu nếu thay đổi đồng thời cả cấu trúc trích xuất đặc trưng đầu vào.

Cấu trúc Backbone trong cấu hình Gold-YOLO bao gồm 10 lớp chính (từ Index 0 đến Index 9) được xây dựng từ ba thực thể module cốt lõi: các lớp tích chập tiêu chuẩn (Conv), các khối liên kết chéo cục bộ nâng cao (C2f), và khối tổng hợp phân cấp không gian nhanh (SPPF).

Dưới đây là bảng tổng hợp chi tiết cấu trúc mạng Backbone được trích xuất từ file cấu hình hệ thống Gold-YOLO:

Bảng 1: Chi tiết cấu trúc mạng Backbone của Gold-YOLO

Index	Module	Số lượng lặp (Repeats)	Tham số đầu vào (Arguments)	Tầng đặc trưng đầu ra / Tỷ lệ hạ lấy mẫu
0	Conv	1	[64, 3, 2]	P1 / 2
1	Conv	1	[128, 3, 2]	P2 / 4
2	C2f	3	[128, True]	Chứng thực đặc trưng P2
3	Conv	1	[256, 3, 2]	P3 / 8
4	C2f	6	[256, True]	Chứng thực đặc trưng P3
5	Conv	1	[512, 3, 2]	P4 / 16
6	C2f	6	[512, True]	Chứng thực đặc trưng P4
7	Conv	1	[1024, 3, 2]	P5 / 32
8	C2f	3	[1024, True]	Chứng thực đặc trưng P5
9	SPPF	1	[1024, 5]	Bản đồ đặc trưng ngữ cảnh cuối

2.3.3. Gold-YOLO Head — các module đặc trưng

Kiến trúc Head (bao gồm cả khối Neck cải tiến) của Gold-YOLO đại diện cho một bước đột phá trong việc tối ưu hóa luồng truyền dẫn đặc trưng [5]. Thay thế cho mạng PANet truyền thống vốn dựa trên các phép toán cục bộ luân phiên, Gold-YOLO Head triển khai cơ chế Gather-and-Distribute (GD) thông qua một chuỗi các module xử lý chuyên biệt bao gồm: SimFusion_4in/3in, IFM, InjectionMultiSum_Auto_pool, PyramidPoolAgg, TopBasicLayer và AdvPoolFusion. Dưới đây là bảng tổng hợp vai trò của các module trong Gold-YOLO Head:

Bảng 2: Vai trò của các module trong Gold-YOLO Head

Tên module	Giai đoạn trong GD-Neck	Đầu vào chính	Cơ chế cốt lõi	Mục tiêu chức năng
SimFusion_4in / 3in	Gather (Thu thập)	P2, P3, P4, P5 từ Backbone	Trích xuất đặc trưng sau khi nối chuỗi (Concat)	Tạo ra bản đồ ngữ cảnh toàn cục thống nhất
IFM	Phân phối luồng	Đầu ra từ SimFusion	Khối chiếu tuyến tính điều chỉnh kênh	Chuẩn bị dữ liệu định dạng kênh trước khi tiêm

InjectionMultiSum_Auto_pool	Distribute (Phân phối)	Nhánh Cục bộ + Toàn cục từ IFM	Chú ý Sigmoid + Hạ lấy mẫu tự động	Tiêm trực tiếp thông tin ngữ cảnh để lọc nhiễu
PyramidPoolAgg	Thu thập mức cao	Các tầng đặc trung sâu	Pooling đa quy mô song song	Giảm FLOPs khi gom thông tin từ các tầng sâu
AdvPoolFusion	Dung hợp nâng cao	Các tầng không đồng nhất kích thước	Pooling kết hợp học trọng số thích nghi	Giảm thiểu hiện tượng răng cưa khi căn chỉnh không gian

a. Module SimFusion_4in và SimFusion_3in: Tổng hợp đặc trưng đa tầng

Module SimFusion (Simple Fusion Module) đóng vai trò cốt lõi trong giai đoạn Thu thập (Gather) của cơ chế GD [5]. Nhiệm vụ của module này là hợp nhất các bản đồ đặc trưng có độ phân giải không gian và số lượng kênh khác nhau thành một khối thông tin đồng nhất, đại diện cho ngữ cảnh toàn cục của bức ảnh.

- SimFusion_4in: Nhận đồng thời 4 đầu vào từ các tầng đặc trưng khác nhau (ví dụ: các tầng P2, P3, P4, P5 từ Backbone).
- SimFusion_3in: Nhận đồng thời 3 đầu vào từ các tầng đặc trưng ở giai đoạn sau (High-stage GD).

Nguyên lý hoạt động: Trước khi thực hiện dung hợp, các bản đồ đặc trưng đầu vào có kích thước không gian nhỏ sẽ được nội suy phóng đại (Upsampling), trong khi các tầng có kích thước lớn sẽ được hạ lấy mẫu (Downsampling) bằng tích chập hoặc pooling để đưa về cùng một kích thước không gian đích (Target Resolution). Sau đó, chúng được nối chuỗi (Concatenate) và đi qua một bộ lọc tích chập nhằm nén kênh và trích xuất đặc trưng kết hợp.



Hình 3: Sơ đồ khối SimFusion_4in

Công thức toán học:

Gọi $X_1, X_2, \dots, X_n$ (với $n = 3$ hoặc $n = 4$) là các bản đồ đặc trưng đầu vào. Quá trình căn chỉnh không gian (Alignment) được thực hiện như sau:

$$\hat{X}_i = \text{Resize}(X_i, H_{\text{target}}, W_{\text{target}}) \quad (2.16)$$

Sau khi đưa về cùng kích thước, đặc trưng tổng hợp X_{fused} được tính bằng công thức:

$$X_{\text{fused}} = \text{SiLU} \left(\text{BN} \left(\text{Conv}_{1 \times 1} \left(\text{Concat}(\hat{X}_1, \hat{X}_2, \dots, \hat{X}_n) \right) \right) \right) \quad (2.17)$$

Trong đó, Concat biểu diễn phép toán nối chuỗi dọc theo trục kênh (channel dimension), và $\text{Conv}_{1 \times 1}$ là lớp tích chập kích thước hạt 1×1 có nhiệm vụ giảm số lượng kênh về giá trị cấu hình mong muốn.

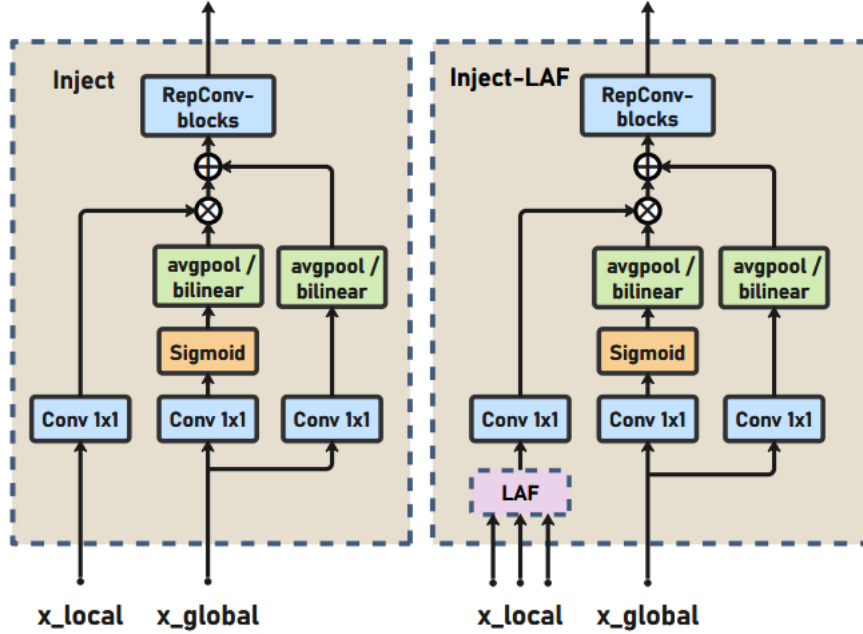
b. Module IFM (Information Flow Module / Information Injection Module)

Module IFM đóng vai trò là "bộ điều phối dòng thông tin" [5]. Sau khi SimFusion tạo ra đặc trưng toàn cục X_{fused} , module IFM sẽ chịu trách nhiệm nhận khối thông tin này, thực hiện tinh lọc và phân tách nó thành các nhánh biểu diễn riêng biệt, tương ứng với yêu cầu về số lượng kênh của từng tầng đích trước khi tiến hành tiêm thông tin.

Nguyên lý hoạt động: IFM nhận đầu vào là đặc trưng đã dung hợp, đi qua các lớp tích chập điều chỉnh kênh (Conv), sau đó chia (split) hoặc thiết lập các phép chiếu tuyến tính (linear projections) để tạo ra các tensor thông tin toàn cục sẵn sàng phân phối cho các khối Injection.

c. Khối InjectionMultiSum_Auto_pool: Cơ chế tiêm thông tin vào từng tầng

Đây là module hiện thực hóa giai đoạn Phân phối (Distribute) của Gold-YOLO. Thay vì thực hiện phép cộng thô hoặc nối chuỗi đơn giản như FPN/PANet, InjectionMultiSum_Auto_pool tiêm thông tin toàn cục từ IFM vào đặc trưng cục bộ của từng tầng dựa trên một cơ chế chú ý chia sẻ (shared attention-like mechanism) kết hợp với hạ lấy mẫu tự động thích ứng (Auto_pool) [5].



Hình 4: Sơ đồ khối cơ chế hoạt động của InjectionMultiSum_Auto_pool

Công thức toán học: Gọi X_{local} là bản đồ đặc trưng cục bộ thu được từ nhánh Backbone (mang thông tin chi tiết hình học cao) và X_{global} là đặc trưng toàn cục nhận được từ khối IFM.

Bước 1: Hạ lấy mẫu tự động đặc trưng toàn cục bằng phép toán Auto_pool (kết hợp giữa Adaptive Average Pooling và Max Pooling tùy thuộc vào kích thước không gian của tầng đích):

$$X_{global_aligned} = \text{Auto_pool}(X_{global}) \quad (2.18)$$

Bước 2: Tạo bản đồ trọng số chú ý (Attention Map) từ đặc trưng toàn cục để định hướng các vùng không gian quan trọng:

$$A = \sigma \left(\text{Conv}_{1 \times 1}(X_{global_aligned}) \right) \quad (2.19)$$

Trong đó σ là hàm kích hoạt Sigmoid đưa giá trị trọng số về khoảng (0, 1).

Bước 3: Thực hiện phép tiêm thông tin kết hợp đa tầng:

$$X_{out} = X_{local} \odot A + \text{Conv}_{1 \times 1}(X_{global_aligned}) \quad (2.20)$$

Trong đó $\odot$ là phép nhân từng phần tử (element-wise multiplication). Phép toán này cho phép thông tin ngữ cảnh toàn cục đóng vai trò như một bộ lọc, trực tiếp điều tiết và làm nổi bật các đặc trưng cục bộ quan trọng, đồng thời nhánh cộng phía sau đóng vai trò là một đường truyền tắt (residual connection) bảo toàn thông tin nền.

d. Khối PyramidPoolAgg và TopBasicLayer: Tổng hợp kim tự tháp mức cao

Trong giai đoạn xử lý các đặc trưng mức cao (High-stage GD), việc duy trì toàn bộ độ phân giải không gian sẽ gây ra gánh nặng tính toán cực kỳ lớn. Do đó, Gold-YOLO sử dụng cấu trúc kết hợp giữa PyramidPoolAgg (Pyramid Pooling Aggregator) và TopBasicLayer [5].

- PyramidPoolAgg: Sử dụng một cấu trúc đa quy mô dựa trên các khối Pooling song song với các kích thước vùng nhận diện khác nhau (ví dụ: 1×1 , 2×2 , 4×4) Khối này nén thông tin không gian của các tầng đặc trưng sâu một cách hiệu quả, giữ lại các thành phần ngữ cảnh cốt lõi nhất mà không làm bùng nổ số lượng tham số.
- TopBasicLayer: Là một chuỗi các khối tích chập cơ bản (thường xây dựng từ cấu trúc Bottleneck cải tiến) đặt ở đỉnh mạng để xử lý thông tin sau khi được gom bởi PyramidPoolAgg.

Công thức toán học của PyramidPoolAgg:

Cho đầu vào X , đầu ra của khối tổng hợp kim tự tháp là sự kết hợp của các nhánh pooling:

$$X_{pool}^i = \text{Conv}_{1 \times 1} \left(\text{AvgPool}_{k_i}(X) \right) \quad \text{với } k_i \in \{1, 2, 4\} \quad (2.21)$$

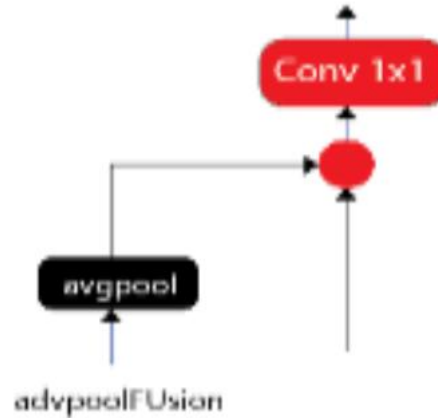
$$X_{agg} = \text{Concat} \left(\text{Resize}(X_{pool}^1), \text{Resize}(X_{pool}^2), \text{Resize}(X_{pool}^3) \right) \quad (2.22)$$

e. Khối AdvPoolFusion: Fusion thích nghi nâng cao

Module AdvPoolFusion (Advanced Pooling Fusion) được thiết kế nhằm mục đích tối ưu hóa quá trình gộp các luồng đặc trưng có sự chênh lệch lớn về độ phân giải không gian và số lượng kênh trong cấu hình Gold-YOLO.

Thay vì ép buộc tất cả các tầng tuân theo một phép nội suy bilinear duy nhất (vốn dễ gây ra hiện tượng răng cưa và mất mát phân tách cục bộ), AdvPoolFusion kết hợp linh

hoạt giữa các phép toán AvgPool, MaxPool và tích chập có bước nhảy (strided convolution) dựa trên một bộ trọng số học tập được. Điều này giúp mô hình tự động thích ứng cấu trúc dung hợp theo từng tập dữ liệu cụ thể, tối ưu hóa độ chính xác đối với các vật thể có dải quy mô thay đổi liên tục.



Hình 5: Sơ đồ khối AdvPoolFusion

2.3.4. Detect head tại P3, P4, P5

Đầu nhận diện (Detect Head) là thành phần cuối cùng trong sơ đồ mạng mạng, đóng vai trò chuyển đổi các bản đồ đặc trưng (feature maps) thu được sau quá trình trích xuất và tổng hợp thông tin thành các dự đoán cụ thể về vị trí (bounding box) và nhãn (class) của đối tượng. Trong kiến trúc cải tiến đề xuất, mô hình áp dụng cấu trúc đầu nhận diện phân tách (Decoupled Head) [2] độc lập tại ba tầng tỷ lệ khác nhau tương ứng với các tầng đầu ra: P3, P4, và P5.

a. Kiến trúc đầu nhận diện giải phân tách (Decoupled Head) và Cơ chế Anchor-free

Thay vì sử dụng một cấu trúc tích hợp chung cho cả hai nhiệm vụ phân lớp (Classification) và định vị (Regression) như các phiên bản YOLO truyền thống (YOLOv3 đến YOLOv5), hệ thống Detect Head tách biệt hoàn toàn hai nhánh xử lý độc lập:

- **Nhánh Phân lớp (Classification Branch):** Sử dụng chuỗi các lớp tích chập để trích xuất các đặc trưng ngữ nghĩa mang tính phân biệt cao, từ đó tính toán xác suất phân loại các lớp đối tượng thông qua hàm kích hoạt Sigmoid kết hợp với hàm tổn thất BCE (Binary Cross-Entropy).
- **Nhánh Hồi quy tọa độ (Regression Branch):** Tập trung vào việc xác định ranh giới đối tượng bằng cách dự đoán khoảng cách từ một điểm pixel đến 4 cạnh của hộp

bao (bounding box). Nhánh này kết hợp hàm tổn thất CIoU (Complete IoU) [20] và DFL (Distribution Focal Loss) [21] nhằm làm mịn các đường biên ngữ cảnh và xử lý các ranh giới không chắc chắn.

Sự phân tách này giúp giải quyết triệt để sự xung đột đặc trưng (misalignment gap) [22], do bài toán phân lớp yêu cầu các đặc trưng có tính bất biến đối với phép dịch chuyển (translation-invariant), trong khi bài toán định vị lại đòi hỏi các đặc trưng nhạy cảm với vị trí và biên độ (translation-covariant). Đồng thời, việc áp dụng cơ chế Anchor-free (không sử dụng hộp neo cố định) giúp mô hình dự đoán trực tiếp từ tâm vật thể, giảm thiểu đáng kể số lượng tham số tính toán và tăng tính linh hoạt đối với các vật thể có tỉ lệ khung hình biến động phức tạp.

b. Chức năng phân cấp đa quy mô của các tầng Detect Head

Ba đầu nhận diện được phân bổ tại ba mức phân giải không gian khác nhau nhằm tối ưu hóa khả năng phát hiện đa quy mô (Multi-scale Object Detection):

- Detect Head tại P3: Đầu nhận diện P3 đảm nhận vai trò chủ chốt trong việc phát hiện và định vị các đối tượng có kích thước nhỏ đến trung bình, giảm thiểu tối đa hiện tượng bỏ sót các vật thể mật độ dày hoặc bị mờ nhòe.
- Detect Head tại P4: Đầu nhận diện P4 chịu trách nhiệm nhận diện các đối tượng có kích thước vừa, đóng vai trò là vùng chuyển tiếp quan trọng trong chuỗi phân cấp đặc trưng đa quy mô.
- Detect Head tại P5 : Đầu nhận diện P5 chịu trách nhiệm nhận diện các đối tượng có kích thước lớn hoặc chiếm phần lớn không gian ảnh, đảm bảo tính chính xác khi phân loại các vật thể nằm gần camera hoặc có cấu trúc phức tạp.

c. Cơ chế gán mẫu động Task-Aligned Assigner (TAL)

Tại các đầu ra dự đoán của các nhánh P3, P4 và P5, mô hình áp dụng chiến lược gán nhãn động Task-Aligned Assigner (TAL) [22] thay vì các ngưỡng IoU cố định như trước. TAL tự động tính toán một điểm số đồng nhất (alignment score) dựa trên cả mức độ tin cậy phân lớp và độ trùng khớp của hộp bao:

$$t = s^{\alpha} \times u^{\beta} \quad (2.23)$$

Trong đó: s là xác suất dự đoán đúng lớp (classification score).

u là giá trị IoU giữa hộp bao dự đoán và hộp thực tế (Ground Truth).

α và β là các siêu tham số điều tiết trọng số ảnh hưởng của từng tác vụ.

Cơ chế này đảm bảo rằng các đầu nhận diện P3, P4, P5 sẽ ưu tiên lựa chọn và tối ưu hóa những vùng pixel dự đoán có sự liên kết chặt chẽ nhất giữa cả hai nhiệm vụ (vừa phân lớp đúng, vừa định vị chuẩn), từ đó giúp cải thiện đáng kể độ chính xác trung bình toàn diện (mAP) của mô hình.

2.3.5. Ưu điểm so với FPN chuẩn và hạn chế còn tồn tại

Để làm rõ giá trị khoa học của kiến trúc cơ sở Gold-YOLO, việc đặt mô hình này lên bàn cân so sánh với cấu trúc mạng phân cấp kim tự tháp đặc trưng truyền thống FPN (Feature Pyramid Network) [17] và PANet (Path Aggregation Network) [18] là vô cùng cần thiết. Qua đó, nhóm nghiên cứu có thể chỉ ra những đột phá cốt lõi cũng như các điểm nghẽn kỹ thuật còn tồn tại, thiết lập động lực trực tiếp cho các đề xuất cải tiến ở phần tiếp theo của đồ án.

a. Ưu điểm vượt trội so với FPN/PANet chuẩn

Kiến trúc GD-Neck của Gold-YOLO mang lại ba bước tiến lớn trong việc xử lý luồng dữ liệu đa quy mô:

- **Triệt tiêu hiện tượng hao hụt thông tin (Information Attenuation):** Trong cấu trúc FPN hoặc PANet tiêu chuẩn của YOLOv8, thông tin đặc trưng bắt buộc phải truyền dẫn tuần tự qua từng tầng trung gian (ví dụ: luồng thông tin hình học từ P3 muốn đóng góp cho P5 phải đi qua P4). Qua mỗi bước tích chập và thay đổi kích thước không gian, các chi tiết cục bộ chất lượng cao sẽ bị mờ nhạt dần hoặc biến mất. Gold-YOLO xóa bỏ hoàn toàn cơ chế "bắc cầu" này nhờ cấu trúc tập trung (Gather) đa tầng, thu thập đồng thời tất cả các cấp độ đặc trưng để xử lý chung, giúp bảo toàn tính toàn vẹn của dữ liệu gốc.
- **Thiết lập mối quan hệ ngữ cảnh tầm xa (Long-range Dependencies):** Các mạng FPN thông thường thực hiện dung hợp bằng phép toán cộng phần tử (element-wise addition) hoặc nối chuỗi (concat) cục bộ, khiến mô hình bị giới hạn trường nhìn. Ngược lại, việc tích hợp module tiêm thông tin (IFM và Injection) giúp phân phối một bản đồ ngữ cảnh toàn cục (Global Context Map) đến mọi pixel trên các tầng dự đoán. Cơ chế này hoạt động tương tự như một bộ lọc chú ý trực quan, giúp các

đầu nhận diện hiệu được mối tương quan giữa vật thể mục tiêu và môi trường nền xung quanh, từ đó giảm tỷ lệ báo động giả (false positives) trong các bối cảnh phức tạp.

- Tối ưu hóa khả năng liên kết đa quy mô (Multi-scale Alignment): Thay vì để các tầng hoạt động độc lập hoặc liên kết lỏng lẻo, cơ chế tiêm thông tin thích ứng giúp tầng đặc trưng nông (P3) nhận được các tri thức ngữ nghĩa bậc cao từ tầng sâu (P5) ngay lập tức và ngược cả lại. Sự tương tác hai chiều đồng thời này giúp mô hình nhận diện tốt các vật thể có dải kích thước biến đổi liên tục hoặc các đối tượng bị che khuất một phần.

b. Hạn chế còn tồn tại của Gold-YOLO

Mặc dù giải quyết xuất sắc bài toán dung hợp thông tin toàn cục, cấu hình Gold-YOLO vẫn bộc lộ những nhược điểm lớn khi đối chiếu với các yêu cầu thực tế của một hệ thống nhúng biên:

- Gánh nặng tính toán và áp lực băng thông bộ nhớ (High MAC & Latency): Cơ chế Gather-and-Distribute đòi hỏi một lượng lớn các phép toán căn chỉnh không gian phức tạp bao gồm nội suy phóng đại (Upsampling), hạ lấy mẫu tự động (Auto_pool), nối chuỗi tensor và các phép nhân ma trận từng phần tử (Element-wise multiplication) diện rộng trong khối Injection. Các thao tác này làm tăng đột biến Chi phí truy cập bộ nhớ (Memory Access Cost - MAC) và số lượng tính toán FLOPs. Đối với các thiết bị phần cứng biên bị giới hạn về tài nguyên (như dòng Jetson Nano hay các vi điều khiển nhúng) [12], điều này gây ra hiện tượng sụt giảm FPS nghiêm trọng, mất đi tính toán thời gian thực (Real-time).
- Sự chồng kênh từ các khối C2f truyền thống: Mạng Backbone và Neck của mô hình baseline vẫn duy trì các khối C2f tiêu chuẩn của YOLOv8. Kiến trúc C2f dù trích xuất đặc trưng tốt nhờ cơ chế chia nhánh dòng Gradient, nhưng lại chứa nhiều lớp tích chập xếp chồng dày đặc. Khi kết hợp với cấu trúc vốn đã nặng của GD-Neck, nó tạo ra sự lặp lại dư thừa tham số, khiến kích thước mô hình (Model Size) tăng lên đáng kể, gây khó khăn cho việc lưu trữ trên bộ nhớ đệm (Cache) dung lượng thấp của chip biên.

- Thiếu cơ chế chú ý không gian và kênh ba chiều (3D Attention Blanks): Giai đoạn phân phối thông tin của Gold-YOLO tập trung mạnh vào việc thiết lập tương tác giữa các tầng khác nhau (Cross-layer interaction). Tuy nhiên, nội tại trong cùng một tầng đặc trưng (Intra-layer feature interaction), mô hình vẫn dựa vào các bộ lọc tích chập thông thường, xử lý mọi vùng không gian và mọi kênh một cách công kênh như nhau. Việc thiếu một cơ chế chú ý linh hoạt, gọn nhẹ có khả năng trích xuất đồng thời trọng số không gian và kênh (như cơ chế chú ý 3D) khiến mô hình chưa đạt tới độ nhạy bén tối đa trước các biến động về ánh sáng hoặc nhiễu môi trường (đặc trưng của tập dữ liệu giao thông như KITTI [7]).

Những hạn chế về mặt tốc độ suy luận, kích thước mô hình và độ tinh lọc đặc trưng cục bộ chính là tiền đề then chốt để nhóm nghiên cứu đề xuất việc thay thế các khối C2f bằng cấu trúc tăng tốc phân cứng Fast-C2f [6], đồng thời tích hợp cơ chế chú ý không tham số SimAM [4] vào sơ đồ mạng. Những cải tiến này sẽ được phân tích sâu trong các chương tiếp theo nhằm tạo ra một biến thể tối ưu toàn diện cho môi trường biên.

2.4. Các module nghiên cứu tích hợp

2.4.1. Module Fast-C2f

Để giải quyết triệt để các hạn chế về gánh nặng tính toán, chi phí truy cập bộ nhớ cao (MAC) và hiện tượng sụt giảm tốc độ xử lý (FPS) trên các thiết bị biên của mô hình Gold-YOLO [5], nghiên cứu này đề xuất thay thế khối C2f [6] truyền thống bằng module cải tiến Fast-C2f. Đây là một cấu trúc tối ưu phân cứng sâu sắc, kết hợp khéo léo giữa tư duy khai thác tài nguyên kênh hiệu quả và tính toán bất đối xứng.

a. Cấu trúc so sánh với C2f gốc

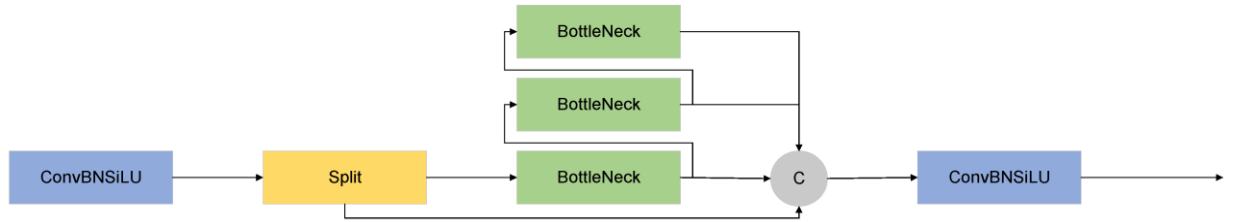
Khối C2f nguyên bản trên YOLOv8 được thiết kế dựa trên ý tưởng của mạng liên kết tầng hiệu quả (ELAN). Luồng dữ liệu đầu vào sau khi đi qua một lớp tích chập 1×1 sẽ được chia đôi thành hai nhánh theo trục kênh (Channel split). Nhánh thứ nhất được giữ làm đường tắt (shortcut), nhánh thứ hai đi qua chuỗi các khối Bottleneck xếp chồng tuần tự. Cuối cùng, đầu ra của tất cả các khối trung gian này cùng với nhánh shortcut ban đầu được nối chuỗi

Mặc dù C2f gốc giúp làm phong phú dòng Gradient và tăng độ chính xác, cấu trúc nội tại của khối Bottleneck bên trong nó lại chứa các lớp tích chập tiêu chuẩn (Standard Convolution) hoạt động trên toàn bộ số lượng kênh. Điều này tạo ra sự dư thừa rất lớn về mặt biểu diễn đặc trưng giữa các kênh song song, đồng thời làm tăng Chi phí Truy cập

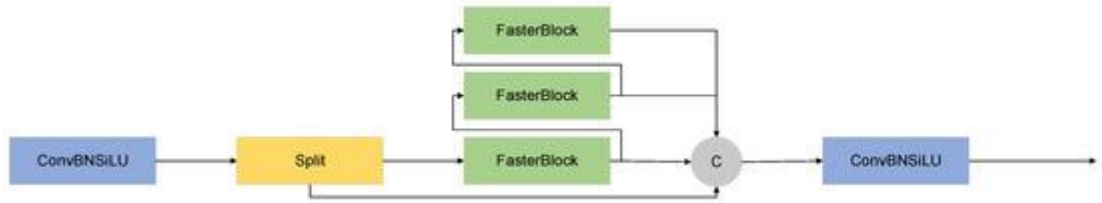
Bộ nhớ (Memory Access Cost - MAC) do các thao tác chia tách và nối chuỗi tensor diễn ra liên tục.

Module Fast-C2f tái cấu trúc bản chất của các khối xử lý bên trong. Thay vì sử dụng khối Bottleneck truyền thống, Fast-C2f triển khai một cấu trúc lõi siêu nhẹ dựa trên nguyên lý Tích chập từng phần (Partial Convolution - PConv) [6].

Trong Fast-C2f, thay vì tính toán tích chập trên toàn bộ c kênh đầu vào, mô hình chỉ chọn lọc một tỷ lệ kênh cố định (thường là $1/4$ tổng số kênh) để áp dụng các bộ lọc tích chập, trong khi các kênh còn lại được giữ nguyên bản độc lập và chuyển tiếp thẳng ra phía sau.



Hình 6: C2f module



Hình 7: C2f-Faster module

Hình 7: C2f-Faster module

b. Cơ chế tối ưu tốc độ: Partial Computation và Reparameterization

Sự vượt trội về mặt tốc độ của Fast-C2f đến từ hai cơ chế toán học và kiến trúc cốt lõi:

Tính toán từng phần (Partial Computation / PConv)

Để chứng minh khả năng tối ưu hóa của cơ chế tính toán từng phần bằng toán học, ta xét một bản đồ đặc trưng đầu vào $X \in R^{c \times h \times w}$ với số kênh c , chiều cao h và chiều rộng w .

Đối với một lớp tích chập tiêu chuẩn sử dụng bộ lọc kích thước $k \times k$, số lượng phép toán dấu phẩy động (FLOPs) được tính bằng [6]:

$$\text{FLOPs}_{\text{Standard}} = h \times w \times k^2 \times c^2 \quad (2.24)$$

Trong module Fast-C2f, phép tích chập PConv chỉ thực hiện trên một số lượng kênh giới hạn $c_p = r \times c$ (với tỷ lệ mặc định $r = \frac{1}{4}$). Do đó, số lượng FLOPs giảm xuống chỉ còn:

$$\text{FLOPs}_{\text{PConv}} = h \times w \times k^2 \times c_p^2 = h \times w \times k^2 \times (r \cdot c)^2 \quad (2.25)$$

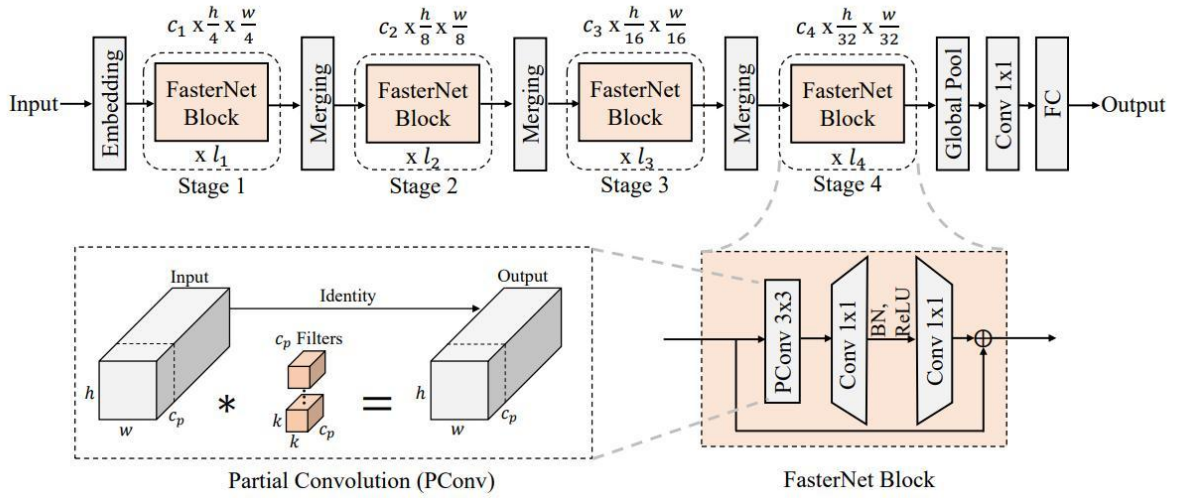
Khi chọn $r = \frac{1}{4}$, ta có:

$$\text{FLOPs}_{\text{PConv}} = \frac{1}{16} \times \text{FLOPs}_{\text{Standard}} \quad (2.26)$$

Sự sụt giảm này mang lại lợi ích khổng lồ về mặt tốc độ. Quan trọng hơn, Chi phí Truy cập Bộ nhớ (MAC) – tác nhân chính gây nghẽn cổ chai trên chip biên – cũng được tối ưu hóa triệt để [6]:

$$\text{MAC}_{\text{Standard}} = 2 \times c \times h \times w + k^2 \times c^2 \quad (2.27)$$

Bằng cách giữ nguyên $c - c_p$ kênh không tính toán, Fast-C2f vừa giảm lượng dữ liệu cần đọc/ghi vào bộ nhớ đệm (Cache), vừa tận dụng các kênh này để bảo toàn nguyên vẹn thông tin gốc của vật thể, tránh hiện tượng mất mát đặc trưng cục bộ.



Hình 8: Kiến trúc FasterNet và cơ chế Partial Convolution (PConv)

Tái tham số hóa cấu trúc (Structural Reparameterization)

Nhằm bù đắp lại lượng suy giảm năng lực học tập do giảm số phép toán tích chập, Fast-C2f áp dụng kỹ thuật tái tham số hóa cấu trúc (được lấy cảm hứng từ các mạng như RepVGG [23]).

- Trong quá trình Huấn luyện (Training), khối được thiết lập dưới dạng đa nhánh phức tạp (Multi-branch), bao gồm một nhánh tích chập 3×3 , một nhánh tích chập 1×1 và một nhánh kết nối tắt đồng nhất (Identity shortcut). Điều này giúp mô hình học được các biểu diễn không gian đa dạng và làm phong phú hệ thống Gradient.
- Trong quá trình Suy luận (Inference / Deployment), các nhánh tích chập tuyến tính này được dung hợp toán học (fused) lại thành một lớp tích chập 3×3 duy nhất thông qua việc chuyển đổi các ma trận trọng số (W) và độ lệch (b) [23]:

$$W_{\text{fused}} = W_{3 \times 3} + \text{Pad}(W_{1 \times 1}) + \text{Identity_Weight} \quad (2.28)$$

Cơ chế này đảm bảo mô hình khi triển khai thực tế cực kỳ tinh gọn, đạt tốc độ thực thi tối đa của một nhánh đơn (Single-branch) nhưng vẫn giữ nguyên vẹn độ chính xác cao được tích lũy từ cấu trúc đa nhánh lúc huấn luyện.

c. Lý do chọn Fast-C2f cho kiến trúc Head và sơ đồ mạng

Việc lựa chọn Fast-C2f làm module chủ chốt cấu hình trong hệ thống Head và Backbone của GPFS_YOLO xuất phát từ các lập luận kỹ thuật sau:

- Cân bằng gánh nặng tính toán của GD-Neck: Như đã phân tích ở phần hạn chế của Gold-YOLO, mạng GD-Neck tiêu tốn quá nhiều tài nguyên cho việc gom cụm (SimFusion) và tiêm thông tin toàn cục (Injection). Nếu tiếp tục duy trì các khối C2f công kênh tại nhánh Head, mô hình sẽ hoàn toàn mất đi khả năng chạy thời gian thực trên các thiết bị nhúng. Sự xuất hiện của Fast-C2f hoạt động như một bộ "giảm tải", bù trừ trực tiếp lượng FLOPs và MAC tăng lên từ cấu trúc GD-Neck, giúp đưa FPS của toàn bộ hệ thống trở lại dải thời gian thực lý tưởng.
- Sự tương thích hoàn hảo với cơ chế tiêm thông tin (Information Injection): Bản chất của cơ chế tiêm thông tin trong Gold-YOLO là tạo ra một ma trận trọng số chú ý toàn cục để nhân và cộng vào đặc trưng cục bộ. Do Fast-C2f giữ lại một lượng lớn kênh không biến đổi qua lớp PConv ($c - c_p$), các kênh này trở thành không gian lưu trữ đặc trưng hình học thô lý tưởng. Khi luồng thông tin toàn cục từ khối IFM được tiêm vào, nó sẽ tương tác cực kỳ nhạy bén với các kênh thuần khiết này, tạo ra sự dung hợp thông tin cục bộ - toàn cục sâu sắc hơn hẳn việc tiêm vào các kênh đã bị biến đổi liên tục qua nhiều lớp tích chập dày đặc của C2f cũ.
- Tối ưu hóa khả năng song song hóa phần cứng biên: Các kiến trúc tính toán trên GPU nhúng hoặc NPU biên rất ưa chuộng các phép toán có luồng dữ liệu liên tục và ít tính toán lặp lại. Fast-C2f với thiết kế giảm thiểu tối đa sự phân mảnh bộ nhớ nhờ cơ chế PConv giúp tăng tốc độ tính toán thực tế [6].

2.4.2. Cơ chế chú ý SimAM

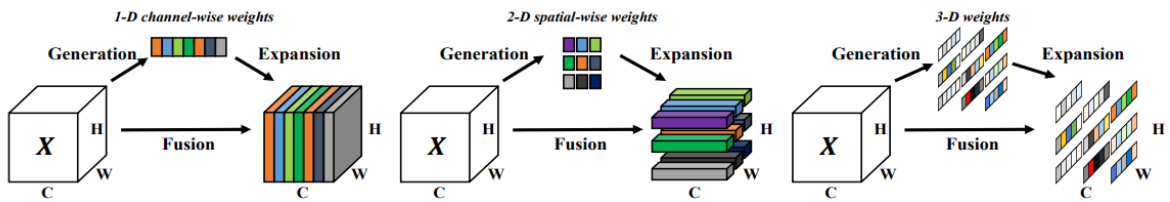
Để khắc phục hạn chế về khả năng nhận diện các vật thể bị che khuất hoặc lẫn trong nhiều nền phức tạp của mô hình cơ sở, nghiên cứu tiến hành tích hợp cơ chế chú ý SimAM (Simple, Parameter-Free Attention Module) [4] vào các khối trích xuất đặc trưng. Khác

biệt hoàn toàn so với các mô-đun chú ý truyền thống, SimAM khai thác triệt để các định lý thần kinh học để thiết lập một không gian chú ý ba chiều (3D) mà không cần đánh đổi bằng tài nguyên phần cứng.

a. Nguyên lý hoạt động: Attention không tham số (Parameter-free)

Phần lớn các cơ chế chú ý hiện hành như SE (Squeeze-and-Excitation) [24] hay CBAM (Convolutional Block Attention Module) [25] đều hoạt động dựa trên việc học các trọng số thông qua các lớp kết nối đầy đủ (Fully Connected layers) hoặc các lớp tích chập bổ sung. Mặc dù cải thiện được độ chính xác, cách tiếp cận này sinh ra hàng triệu tham số (parameters) mới, làm chậm đáng kể quá trình suy luận trên chip biên. Hơn nữa, SE chỉ tập trung vào trục kênh (1D channel attention), trong khi CBAM xử lý độc lập trục không gian và kênh (2D + 1D), dẫn đến việc mất đi sự tương tác đồng thời giữa ba chiều (không gian và kênh) [4].

SimAM phá vỡ giới hạn này bằng cách đề xuất một Cơ chế chú ý không tham số (Parameter-free Attention) [4]. Dựa trên lý thuyết ức chế không gian (spatial inhibition) trong khoa học thần kinh học, SimAM đánh giá tầm quan trọng của một nơ-ron (neuron) bằng cách đo lường mức độ cạnh tranh của nó so với các nơ-ron lân cận.



Hình 9: So sánh cơ chế phân bổ trọng số chú ý 1D, 2D và 3D

b. Công thức tính hàm năng lượng (Energy Function)

Để đo lường mức độ khác biệt của một nơ-ron mục tiêu t so với các nơ-ron khác x_i trong cùng một kênh của bản đồ đặc trưng, SimAM định nghĩa một hàm năng lượng (Energy function) như sau [4]:

$$e_t(w_t, b_t, y, x_i) = \frac{1}{M-1} \sum_{i=1}^{M-1} (-1 - (w_t x_i + b_t))^2 + (1 - (w_t t + b_t))^2 + \lambda w_t^2 \quad (2.29)$$

Trong đó:

- t và x_i lần lượt là nơ-ron mục tiêu và các nơ-ron lân cận trong cùng một kênh.
- $M = H \times W$ là tổng số lượng nơ-ron trên kênh đó (với H là chiều cao, W là chiều rộng).

- w_t và b_t là các phép biến đổi tuyến tính (trọng số và độ lệch).
- λ là hằng số chuẩn hóa (regularization) để tránh hiện tượng giá trị quá lớn.

Bằng cách tối thiểu hóa hàm năng lượng này ($e_t \rightarrow \min$), ta tìm được sự phân tách tuyến tính giữa nơ-ron mục tiêu và phần còn lại. Rất may mắn, bài toán tối ưu này có một nghiệm giải tích đóng (closed-form solution) rất thanh lịch, cho phép tính toán trực tiếp mà không cần quá trình truyền ngược (backpropagation) [4]:

$$e_t^* = \frac{4(\widehat{\sigma^2} + \lambda)}{(t - \hat{\mu})^2 + 2\widehat{\sigma^2} + 2\lambda} \quad (2.30)$$

Trong đó, $\hat{\mu}$ và $\widehat{\sigma^2}$ lần lượt là giá trị trung bình (mean) và phương sai (variance) của toàn bộ các nơ-ron trong kênh tính toán:

$$\hat{\mu} = \frac{1}{M} \sum_{i=1}^M x_i \quad (2.31)$$

$$\widehat{\sigma^2} = \frac{1}{M} \sum_{i=1}^M (x_i - \hat{\mu})^2 \quad (2.32)$$

Theo công thức trên, năng lượng tối thiểu e_t^* càng nhỏ thì nơ-ron t càng khác biệt so với các nơ-ron xung quanh, đồng nghĩa với việc nó càng chứa đựng thông tin quan trọng. Do đó, trọng số chú ý của nơ-ron tỷ lệ nghịch với e_t^* .

c. Ưu điểm vượt trội và sự tích hợp vào Feature Map

Module SimAM mang lại các ưu điểm mang tính cách mạng cho hệ thống nhận diện trên môi trường biên:

- Không gia tăng tham số (Zero Extra Parameters): Vì trọng số được tính hoàn toàn bằng công thức giải tích dựa trên giá trị trung bình và phương sai của chính bản đồ đặc trưng, mô hình không phải gánh thêm bất kỳ tham số học (learnable parameters) nào [4].
- Tích hợp trực tiếp (Plug-and-play): Trọng số sau khi tính toán sẽ được giới hạn bằng hàm kích hoạt Sigmoid và nhân trực tiếp (element-wise) vào bản đồ đặc trưng gốc X để tạo ra bản đồ đặc trưng tinh chỉnh $\tilde{X}$

$$\tilde{X} = X \odot \text{sigmoid}\left(\frac{1}{E}\right) \quad (2.33)$$

(Với E là ma trận chứa các giá trị e_t^* , $\odot$ là phép nhân từng phần tử).

Trong đề xuất của kiến trúc GPFS_YOLO, cơ chế SimAM được cấy ghép trực tiếp vào bên trong các khối Fast-C2f, tạo thành cấu trúc Fast-C2f_SimAM. Sự kết hợp này mang tính bổ trợ hoàn hảo: Fast-C2f giải quyết bài toán giảm thiểu chi phí bộ nhớ [6], trong khi SimAM ngay lập tức tinh lọc lại thông tin, áp đặt trọng số chú ý 3D lên toàn bộ các kênh đặc trưng vừa đi ra [4].

2.4.3. Module Fast-C2f_SimAM

Sự ra đời của mô-đun Fast-C2f_SimAM là kết quả của quá trình tối ưu hóa song song: vừa giải quyết bài toán suy giảm hiệu năng phần cứng bằng cơ chế tính toán bất đối xứng, vừa khắc phục điểm yếu về năng lực trích xuất đặc trưng của mạng nơ-ron thông qua sự chú ý ba chiều.

Cơ chế kết hợp Fast-C2f và SimAM

Thay vì để các module hoạt động rời rạc, nghiên cứu đề xuất nhúng trực tiếp cơ chế chú ý không tham số SimAM [4] vào cấu trúc nội tại của khối Fast-C2f [6].

Nguyên lý hoạt động:

- Bản đồ đặc trưng đầu vào được chia nhánh (channel split). Một phần đi qua đường tắt (identity shortcut), phần còn lại đi vào nhánh xử lý tính toán.
- Tại nhánh xử lý, phép Tích chập từng phần (PConv) trích xuất đặc trưng không gian trên một tập con các kênh (ví dụ: 1/4 số kênh) nhằm giảm thiểu triệt để FLOPs và chi phí truy cập bộ nhớ (MAC).
- Ngay sau khi các luồng đặc trưng đi qua các lớp tích chập của khối lõi, một lượng lớn thông tin về cả đối tượng mục tiêu và nhiễu nền được sinh ra. Lúc này, module SimAM được kích hoạt.
- SimAM đánh giá tức thời mức độ quan trọng của từng nơ-ron trong không gian ba chiều (3D: chiều cao, chiều rộng và kênh) bằng cách tính toán hàm năng lượng e_t^* , sau đó xuất ra một bản đồ trọng số chú ý W_{SimAM} . Bản đồ trọng số này được nhân trực tiếp với đặc trưng vừa được tạo ra.

Công thức toán học của sự kết hợp:

Giả sử $F_{PConv}(X)$ là hàm biểu diễn các phép biến đổi đặc trưng tuyến tính và phi tuyến bên trong lõi của Fast-C2f đối với tensor đầu vào X . Đặc trưng đầu ra tinh chỉnh Y của khối Fast-C2f_SimAM được tính toán như sau:

$$Y = F_{PConv}(X) \odot W_{SimAM} \quad (2.34)$$

$$Y = F_{PConv}(X) \odot \text{Sigmoid}\left(\frac{1}{E}\right) \quad (2.35)$$

Trong đó:

- $\odot$ là phép nhân từng phần tử (element-wise multiplication) lan truyền theo không gian ba chiều.
- E là ma trận năng lượng thu được từ thuật toán giải tích của SimAM [4], tính toán trực tiếp trên tensor $F_{PConv}(X)$ mà không cần bất kỳ tham số học bổ sung nào.

Nhờ cấu trúc này, Fast-C2f_SimAM tạo ra những bản đồ đặc trưng vô cùng "sạch", nơi các pixel biểu diễn vật thể mục tiêu bị che khuất hoặc vật thể kích thước nhỏ được kích hoạt cường độ cao, trong khi các vùng nền gây nhiễu bị triệt tiêu năng lượng một cách hiệu quả.

2.4.4. Tầng đặc trưng P2 trong phát hiện vật thể nhỏ

Trong các bài toán nhận diện đối tượng thực tế, đặc biệt là trong ngữ cảnh quan sát giao thông, các vật thể kích thước nhỏ (Small Objects) hoặc nằm ở khoảng cách xa luôn là thách thức lớn nhất đối với các mô hình mạng nơ-ron tích chập (CNN) [11]. Để giải quyết triệt để vấn đề này, nghiên cứu đề xuất bổ sung tầng đặc trưng P2 vào cấu trúc mô hình.

a. Tầng P2 với độ phân giải siêu cao (Stride = 4)

Trong kiến trúc YOLOv8 tiêu chuẩn, luồng trích xuất đặc trưng đa quy mô được cấu thành từ ba tầng chính: P3, P4 và P5 với các bước nhảy (stride) lũy tiến lần lượt là 8, 16 và 32 so với ảnh gốc đầu vào. Đối với các vật thể siêu nhỏ (kích thước dưới 16×16 pixels), việc đi qua chuỗi các lớp giảm lấy mẫu (downsampling) liên tiếp của mạng nền Backbone sẽ khiến thông tin không gian bị suy giảm nghiêm trọng [11]. Khi một vật thể kích thước 12×12 pixels đi tới tầng P3, diện tích biểu diễn của nó bị thu hẹp xuống chỉ còn khoảng 1.5×1.5 pixels, và hoàn toàn bị triệt tiêu ở các tầng P4, P5.

Bằng cách tích hợp thêm nhánh đặc trưng P2 với bước nhảy Stride = 4, bản đồ đặc trưng giữ được độ phân giải không gian rất cao ($H/4 \times W/4$). Tại tầng này, các thông tin hình học sơ cấp của vật thể nhỏ được bảo toàn nguyên vẹn [1].

b. Phân tích lý thuyết về Trường đón nhận (Receptive Field) và Kích thước vật thể

Hiệu quả của tầng P2 trong việc bắt dính vật thể nhỏ có thể được chứng minh một cách chặt chẽ thông qua lý thuyết toán học về Trường đón nhận (Receptive Field - RF) [11]. Kích thước của trường đón nhận tăng dần theo chiều sâu của mạng nơ-ron. Công thức tổng quát để tính toán kích thước trường đón nhận của tầng thứ l (RF_l) dựa trên tầng liền trước (RF_{l-1}) được biểu diễn như sau:

$$RF_l = RF_{l-1} + (k_l - 1) \times S_{l-1} \quad (2.35)$$

Trong đó:

- k_l là kích thước bộ lọc (kernel size) của lớp tích chập hiện tại ở tầng l
- S_{l-1} là bước nhảy tích lũy (cumulative stride) tính từ ảnh gốc đến trước tầng thứ l , được xác định bởi tích các bước nhảy của toàn bộ các lớp trước đó:

$$S_{l-1} = \prod_{i=1}^{l-1} s_i \quad (2.36)$$

(với s_i là bước nhảy của lớp thứ i).

Khi phân tích mối quan hệ toán học giữa kích thước vật thể mục tiêu (S_{obj}) và kích thước trường đón nhận (RF):

- $RF \gg S_{obj}$ (Xảy ra tại các tầng sâu P4, P5): Một nơ-ron đơn lẻ tại tầng đặc trưng sâu sẽ bao phủ một vùng không gian quá lớn trên ảnh gốc. Đối với một vật thể nhỏ, nơ-ron này chứa quá nhiều thông tin của vùng nền nhiễu xung quanh, khiến tín hiệu đặc trưng của vật thể nhỏ bị "pha loãng" (feature dilution) [11].
- Trường hợp $RF \approx S_{obj}$ (Xảy ra tại tầng nông P2): Nhờ bước nhảy tích lũy nhỏ ($S=4$), kích thước trường đón nhận RF của các nơ-ron tại tầng P2 tương thích và vừa vặn một cách lý tưởng với diện tích không gian của các vật thể nhỏ. Mỗi nơ-ron tập trung trích xuất cô đọng các thuộc tính cục bộ mịn của chính đối tượng đó mà không bị pha lẫn tạp chất bởi cảnh.

Việc thiết kế nhánh P2 kết hợp với module dung hợp thông tin toàn cục của Gold-YOLO tạo nên một cơ chế bù trừ toán học hoàn hảo: Tầng P2 cung cấp các đặc trưng cục bộ có độ phân giải cao và trường đón nhận nhỏ giúp định vị chính xác vị trí vật thể, trong khi cơ chế tiêm thông tin toàn cục (Global Context Injection) bổ sung thêm các mối quan hệ ngữ nghĩa diện rộng từ các tầng sâu, ngăn chặn hiện tượng nhận diện nhầm do thiếu thông tin ngữ cảnh của các tầng nông.

2.4.5. Cơ chế hàm kích hoạt Mish

Hàm kích hoạt Mish là một hàm kích hoạt smooth (liên tục khả vi), non-monotonic (không đơn điệu) và self-regularized, được thiết kế để khắc phục một số hạn chế của ReLU và các biến thể của nó trong mạng nơ-ron sâu [29].

Hàm Mish được định nghĩa như sau:

$$f(x) = x \cdot \tanh(\text{softplus}(x)) \quad (2.37)$$

trong đó:

$$\text{softplus}(x) = \ln(1 + e^x) \quad (2.38)$$

Do đó, công thức đầy đủ:

$$f(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (2.39)$$

Mish sở hữu các tính chất lý thuyết quan trọng sau:

- Tính liên tục khả vi vô hạn (C^∞): Không tồn tại điểm không khả vi như ReLU tại $x = 0$, giúp dòng gradient chảy mượt mà hơn trong quá trình lan truyền ngược, giảm nguy cơ vanishing gradient và hỗ trợ tối ưu hóa ổn định [29].
- Tính không đơn điệu (non-monotonic): Giữ lại một lượng nhỏ thông tin âm (giới hạn dưới khoảng -0.3088), loại bỏ hiện tượng “dying ReLU” và tăng cường khả năng biểu diễn của mô hình [29].
- Không giới hạn từ trên, giới hạn từ dưới: Tránh saturation ở vùng dương lớn, đồng thời tạo hiệu ứng regularization tự nhiên mạnh mẽ nhờ giới hạn dưới, giúp mô hình tổng quát hóa tốt hơn mà không cần các kỹ thuật regularization bổ sung mạnh [29].
- Self-gating mechanism: Tương tự Swish, Mish nhân đầu vào gốc với đầu ra của một hàm phi tuyến tính của chính nó, cho phép mô hình tự động điều chỉnh mức độ kích hoạt dựa trên cường độ tín hiệu [29].

Nhờ các đặc tính trên, Mish tạo ra loss landscape mượt hơn so với ReLU, dẫn đến quá trình hội tụ nhanh hơn và hiệu suất generalization tốt hơn. Các thí nghiệm cũng cho thấy Mish vượt trội ReLU khoảng 1.67% và Swish khoảng 0.494% trên nhiều benchmark Computer Vision.

2.4.6. Hàm mất mát WIoU

Trong phát hiện mục tiêu, đặc biệt với các cảnh có sự chênh lệch lớn về quy mô và occlusion, hàm loss dựa trên IoU đóng vai trò quan trọng trong việc tối ưu hóa khoảng cách giữa khung dự đoán và khung thực. WIoU (Wise-IoU) được phát triển để khắc phục hạn chế của các hàm loss IoU truyền thống như CIoU hay DIoU bằng cách giới thiệu cơ chế dynamic non-monotonic focusing [27].

WIoU cơ bản được định nghĩa như sau:

$$L_{WIoU} = R_{WIoU} \times L_{IoU} \quad (2.40)$$

trong đó:

$$R_{WIoU} = \exp \left(\frac{(b_{cx}^{gt} - b_{cx})^2 + (b_{cy}^{gt} - b_{cy})^2}{c_w^2 + c_h^2} \right) \quad (2.41)$$

$$L_{IoU} = 1 - \frac{|A \cap B|}{|A \cup B|} \quad (2.42)$$

Với:

- $(b_{cx}^{gt}, b_{cy}^{gt})$ và (b_{cx}, b_{cy}) lần lượt là tọa độ trung tâm của khung ground truth và khung dự đoán;
- c_w, c_h là chiều rộng và chiều cao của khung bao ngoài nhỏ nhất chứa cả hai khung;
- $|A \cap B|$ và $|A \cup B|$ lần lượt là diện tích phần giao và phần hợp giữa hai khung.

Cơ chế non-monotonic trong WIoU cho phép mô hình tập trung hơn vào các anchor frame có chất lượng thông thường, đồng thời giảm ảnh hưởng của gradient từ các mẫu cực đoan (outlier). Nhờ đó, WIoU giúp cải thiện đáng kể khả năng hội tụ và độ chính xác định vị, đặc biệt hiệu quả trong môi trường giao thông phức tạp với nhiều vật thể nhỏ và bị che khuất.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT

3.1. Phân tích hạn chế của mô hình Gold-YOLO

Mặc dù kiến trúc Baseline Gold-YOLO đã giải quyết thành công bài toán đứt gãy thông tin đa quy mô thông qua cơ chế "Tập trung và Phân phối" (Gather-and-Distribute Neck), mô hình này vẫn bộc lộ một số điểm nghẽn kỹ thuật nghiêm trọng. Đặc biệt, khi ứng dụng vào môi trường phần cứng biên (Edge Computing) và bài toán phát hiện các đối tượng có kích thước nhỏ, cấu trúc nguyên bản của mô hình cho thấy sự lãng phí tài nguyên và thiếu linh hoạt, cụ thể qua các điểm sau:

3.1.1. Khối C2f trong Backbone: Chi phí tính toán cao, chưa có cơ chế chú ý (Attention)

Mạng nền (Backbone) của Gold-YOLO vẫn kế thừa và sử dụng hoàn toàn khối trích xuất đặc trưng C2f tiêu chuẩn của YOLOv8. Khối C2f mặc dù mang lại dòng gradient phong phú nhờ cấu trúc chia nhánh (split), nhưng nội tại các khối Bottleneck bên trong nó lại sử dụng các phép tích chập tiêu chuẩn (Standard Convolution) hoạt động trên toàn bộ số lượng kênh. Điều này tạo ra một lượng tham số khổng lồ và chi phí tính toán (FLOPs) rất lớn, gây quá tải cho bộ nhớ đệm (Cache) của thiết bị nhúng.

Bên cạnh đó, quá trình trích xuất tại Backbone là bước quan trọng nhất để lọc nhiễu không gian từ ảnh thô đầu vào. Tuy nhiên, C2f xử lý mọi vùng pixel và mọi kênh đặc trưng một cách "bình đẳng". Sự thiếu vắng một cơ chế chú ý (Attention Mechanism) khiến mô hình dễ bị phân tâm bởi các yếu tố bối cảnh phức tạp (ánh sáng, bóng râm, hậu cảnh), làm giảm chất lượng bản đồ đặc trưng trước khi truyền lên mạng Neck.

3.1.2. Khối C2f trong Head: Nghẽn cổ chai tốc độ sau các bước Injection/Fusion

Trong mạng Neck và Head của kiến trúc Gold-YOLO, dòng dữ liệu đa quy mô phải đi qua các module dung hợp cực kỳ phức tạp như SimFusion (gom cụm đặc trưng) và IFM (Mô-đun dòng thông tin). Sau các bước này, bản đồ đặc trưng mang theo một lượng lớn ngữ cảnh toàn cục (Global Context) với mật độ thông tin rất đặc.

Việc tiếp tục sử dụng các khối C2f chồng kênh tại phần Head để xử lý dòng dữ liệu nặng nề này vô tình tạo ra một điểm nghẽn cổ chai về băng thông bộ nhớ (Memory Access Cost Bottleneck). Sự lặp lại các phép toán chia tách kênh và nối chuỗi của C2f ngay sau bước tiêm thông tin làm tăng đột biến độ trễ suy luận (Inference Latency), khiến chỉ số khung hình trên giây (FPS) sụt giảm mạnh và khó đáp ứng được yêu cầu nhận diện thời gian thực (Real-time).

3.1.3. Thiếu tầng đặc trưng P2: Khó phát hiện vật thể nhỏ

Kiến trúc Gold-YOLO chỉ duy trì 3 dải dự đoán tiêu chuẩn là P3, P4 và P5, tương ứng với các bước nhảy (stride) hạ lấy mẫu lần lượt là 8, 16 và 32.

Đối với các bài toán thực tế, vật thể mục tiêu thường nằm ở khoảng cách xa và chiếm diện tích pixel rất nhỏ. Khi đi qua các tầng giảm lấy mẫu liên tiếp của mạng Backbone, các thông tin hình học, góc cạnh và kết cấu của vật thể nhỏ sẽ bị mờ nhạt hoặc biến mất hoàn toàn. Việc thiếu vắng nhánh dự đoán ở tầng P2 (stride = 4, sở hữu độ phân giải không gian cao nhất) khiến mạng Head không có đủ cơ sở dữ liệu không gian cục bộ để định vị chính xác vật thể, dẫn đến tỷ lệ bỏ sót (False Negatives) cao.

3.1.4. Tổng hợp: Động lực (Motivation) cho từng cải tiến

Từ quá trình phân tích các hạn chế thực tại của mô hình baseline, nhóm nghiên cứu đã xác định rõ các điểm cần can thiệp, tạo nên động lực trực tiếp cho cấu trúc mô hình cải tiến tổng thể mang tên GPFS_YOLO (Gold-YOLO + P2 + Fast-C2f + SimAM + Mish + WIoU):

- Cải tiến 1 (Giải quyết hạn chế ở Backbone): Thay thế C2f bằng mô-đun Fast-C2f_SimAM. Cơ chế tính toán từng phần (PConv) sẽ giảm thiểu triệt để FLOPs, đồng thời việc cấy ghép trực tiếp bộ chú ý 3D không tham số (SimAM) sẽ giúp mô hình có khả năng chọn lọc, tập trung mạnh mẽ vào đặc trưng vật thể ngay từ giai đoạn sơ cấp mà không làm tăng dung lượng mạng.
- Cải tiến 2 (Giải quyết hạn chế ở Head): Thay thế C2f bằng Fast-C2f thuần túy. Bằng việc áp dụng một cấu trúc siêu nhẹ tại giai đoạn hậu dung hợp, mô hình giải quyết dứt điểm tình trạng nghẽn cổ chai bộ nhớ, bảo toàn tốc độ suy luận cực nhanh.
- Cải tiến 3 (Giải quyết hạn chế về quy mô): Tái cấu trúc hình học mạng bằng việc bổ sung nhánh nhận diện độ phân giải siêu cao P2 kết hợp với phép phóng đại nội suy (Upsample). Sự dịch chuyển trọng tâm phát hiện này sẽ trực tiếp khắc phục điểm yếu của YOLOv8 tiêu chuẩn trước các đối tượng có kích thước siêu nhỏ.
- Cải tiến 4 (Giải quyết hạn chế về tối ưu hóa huấn luyện và chất lượng loss): Thay thế hàm kích hoạt mặc định (SiLU) bằng hàm Mish trong toàn bộ các lớp

convolution và các khối Neck của Gold-YOLO, đồng thời thay thế hàm loss CIoU bằng WIoU. Việc áp dụng Mish mang lại gradient flow mượt mà hơn, tính non-monotonic và hiệu ứng tự regularized, giúp mạng hội tụ ổn định và khái quát hóa tốt hơn. Kết hợp với WIoU — hàm loss có cơ chế dynamic non-monotonic focusing — mô hình tập trung hiệu quả hơn vào các anchor frame chất lượng thông thường, giảm ảnh hưởng của gradient từ các mẫu cực đoan, từ đó nâng cao đáng kể độ chính xác định vị và khả năng xử lý occlusion trong môi trường thực tế.

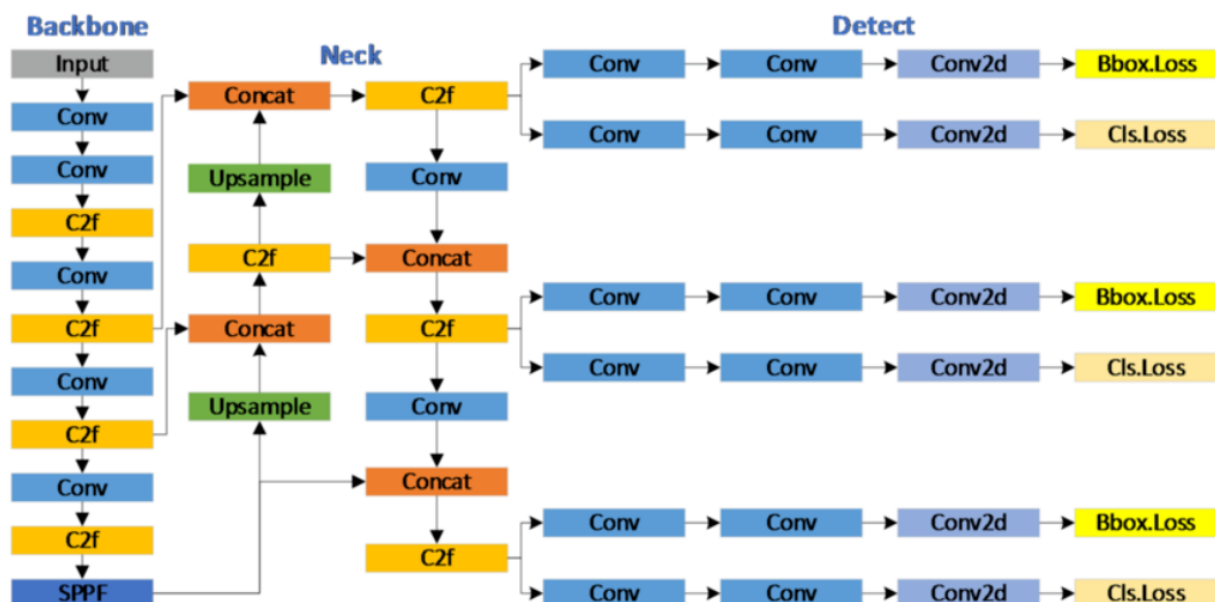
3.2. Kiến trúc đề xuất GPFS

3.2.1. Tổng quan kiến trúc GPFS

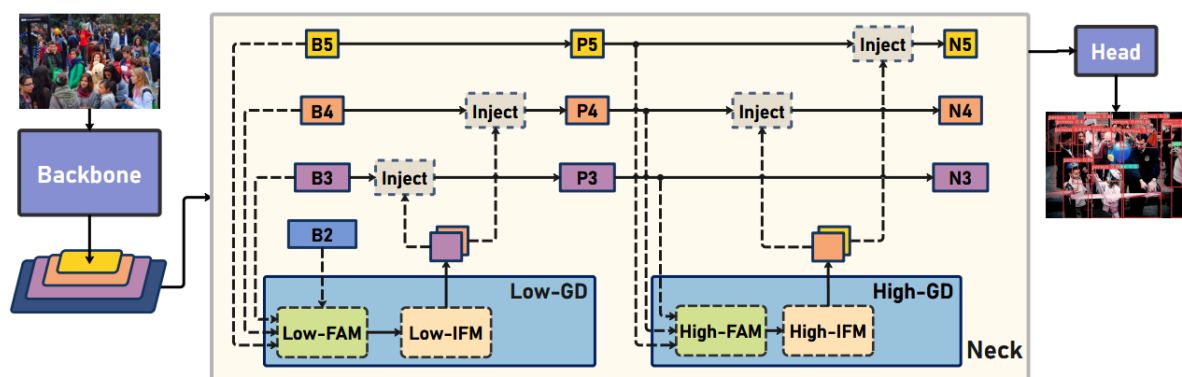
Kiến trúc mô hình đề xuất GPFS_YOLO đại diện cho một bước chuyển đổi toàn diện về mặt cấu trúc hình học mạng và tối ưu hóa toán học nơ-ron dựa trên nền tảng Baseline Gold-YOLO. Triết lý thiết kế cốt lõi của GPFS_YOLO tập trung vào việc "tái phân bổ ngân sách tính toán": cắt giảm các phép toán dư thừa tại các vùng bão hòa ngữ nghĩa nhờ khối siêu nhẹ Fast-C2f, tăng cường bộ lọc tri giác cục bộ bằng cơ chế chú ý 3D không tham số SimAM, đồng thời mở rộng biên độ không gian xuống tầng đặc trưng độ phân giải siêu cao P2 để chuyên biệt hóa cho bài toán nhận diện vật thể kích thước siêu nhỏ cùng với việc thay đổi cách hàm kích hoạt hoạt động thành Mish và hàm thay thế hàm mất mát để cải thiện lại độ chính xác và tốc độ xử lý của mô hình.

a. Sơ đồ kiến trúc tổng thể

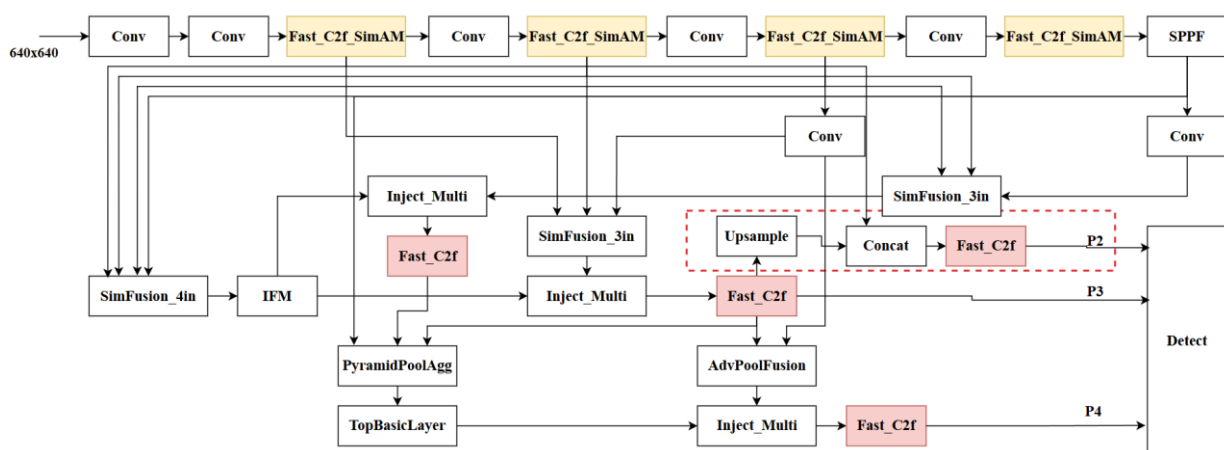
Về mặt tổng thể, mô hình GPFS_YOLO giữ nguyên khung sườn ba phần tiêu chuẩn bao gồm: Backbone (Mạng nền trích xuất thô), GD-Neck (Mạng cổ dung hợp ngữ cảnh nâng cao), và Decoupled Head (Đầu dự đoán phân tách độc lập). Tuy nhiên, các khối chức năng bên trong đã được tái định nghĩa cấu trúc ma trận hoàn toàn.



Hình 10: Sơ đồ khối của YOLOv8



Hình 11: Sơ đồ khối của kiến trúc Gold-YOLO



Hình 12: Sơ đồ kiến trúc GPFS_YOLO

b. So sánh luồng dữ liệu với mô hình gốc (YOLOv8 và Gold-YOLO Baseline)

Sự đột phá mang tính hệ thống của GPFS_YOLO được thể hiện rõ nét khi đặt luồng tuần tự dữ liệu (Data Flow) của mô hình đề xuất lên bàn cân đối chiếu với cấu trúc đồ thị tính toán gốc:

Luồng truyền dẫn tại mạng nền Backbone:

- Mô hình gốc (YOLOv8 / Gold-YOLO): Tensor ảnh thô sau khi đi qua các lớp giảm lấy mẫu (Downsampling) sẽ được nạp trực tiếp vào khối C2f công kênh. Luồng dữ liệu bị chia nhánh (Split) và xử lý bởi các lớp tích chập tiêu chuẩn liên tiếp, khiến thông tin biên bị mờ nhạt dần theo chiều sâu của mạng và tiêu tốn lượng lớn tài nguyên bộ nhớ đệm cache.
- Mô hình GPFS_YOLO: Tín hiệu dữ liệu khi đi vào khối Fast-C2f_SimAM sẽ được tách kênh một phần nhờ tích chập từng phần (PConv), giúp giải phóng băng thông truyền dẫn. Ngay sau đó, ma trận dữ liệu được đi qua bộ lọc năng lượng không tham số SimAM. Luồng đặc trưng đi ra khỏi Backbone lúc này đã được "chuẩn hóa tri giác" – các vùng pixel bối cảnh nhiễu bị triệt tiêu biên độ năng lượng, trong khi tín hiệu hình học của các vật thể được khuếch đại rõ rệt.

Luồng dung hợp đa quy mô tại mạng Neck (Gather-and-Distribute):

- Mô hình YOLOv8: Mạng Neck sử dụng cấu trúc PAN-FPN để thực hiện quá trình hợp nhất đặc trưng giữa các tầng P3, P4 và P5 thông qua các phép Upsample, Concat và C2f. Cơ chế này chủ yếu truyền thông tin theo hướng từ trên xuống (Top-down) và từ dưới lên (Bottom-up), trong khi tầng đặc trưng có độ phân giải cao P2 không trực tiếp tham gia vào quá trình dung hợp đa quy mô. Do đó, khả năng khai thác đồng thời các đặc trưng cục bộ chi tiết và ngữ nghĩa toàn cục vẫn còn hạn chế, đặc biệt đối với các đối tượng nhỏ.
- Mô hình GPFS_YOLO: Mạng Neck được tích hợp khối SimFusion_4in (Layer 10) để thực hiện dung hợp đồng thời bốn tầng đặc trưng P2, P3, P4 và P5. Việc đưa tầng P2 vào quá trình hợp nhất giúp các đặc trưng có độ phân giải cao và giàu thông tin chi tiết được kết hợp trực tiếp với các đặc trưng ngữ nghĩa ở các tầng sâu. Nhờ đó, luồng thông tin truyền qua các nhánh Injection sau quá trình Gather-and-

Distribute mang đầy đủ thông tin đa quy mô, vừa bảo toàn chi tiết không gian của các đối tượng nhỏ, vừa duy trì khả năng biểu diễn ngữ nghĩa mạnh của các tầng sâu. Điều này góp phần nâng cao hiệu quả phát hiện đối tượng có kích thước nhỏ và các trường hợp xuất hiện ở nhiều tỷ lệ khác nhau.

Luồng phản hồi xuôi - ngược và Cấu trúc đầu ra (Detect Head):

- Mô hình gốc (YOLOv8 / Gold-YOLO): Luồng dữ liệu từ mạng Neck đổ về Head được chia thành 3 nhánh dự đoán cố định ứng với các tỷ lệ hạ lấy mẫu là $1/8$ (P3), $1/16$ (P4), và $1/32$ (P5). Đối với vật thể nhỏ (diện tích $< 16 \times 16$ pixel trên ảnh gốc), luồng dữ liệu tại nhánh P5 hoàn toàn không mang giá trị tri giác, gây lãng phí chu kỳ xử lý của GPU.
- Mô hình GPFS_YOLO: Bản đồ đặc trưng sau khi được thêm ngữ cảnh tại tầng P3 tiếp tục đi qua một luồng phản hồi xuôi (Top-Down) thông qua lớp nội suy phóng đại hình học (nn.Upsample với hệ số nhân 2). Luồng dữ liệu phóng đại này được thực hiện phép toán ghép nối ma trận Concat trực tiếp với luồng P2 thô giữ lại từ Backbone. Sau khi được tinh lọc qua khối siêu nhẹ Fast-C2f, luồng dữ liệu mới được chuyển hướng hoàn toàn sang dải quy mô siêu mịn [P2, P3, P4] (tương ứng với các bước nhảy stride cực nhỏ là 4, 8, 16) và triệt tiêu hoàn toàn nhánh P5 dư thừa.

Sự cải tiến luồng dữ liệu này đảm bảo rằng mọi tài nguyên tính toán giải phóng được từ khối cổng kênh C2f và tầng vô hiệu P5 sẽ được dồn toàn bộ vào việc nuôi dưỡng đường truyền tín hiệu hình học cho nhánh P2, giúp mô hình đạt độ nhạy bén vượt trội đối với các đối tượng kích thước siêu nhỏ mà không làm bùng nổ chi phí phần cứng.

3.2.2. Cải tiến 1 — Thay thế C2f $\rightarrow$ Fast-C2f_SimAM trong Backbone

Mạng nền Backbone đóng vai trò quyết định trong việc lọc nhiễu không gian và trích xuất các dấu hiệu hình học sơ cấp từ ảnh thô. Để tối ưu hóa giai đoạn này, nghiên cứu đề xuất thay thế toàn bộ các khối C2f truyền thống tại các tầng trích xuất chủ lực thành khối cấu trúc cải tiến Fast-C2f_SimAM.

a. Vị trí thực thi cấu trúc

Dựa trên sơ đồ tuần tự của đồ thị tính toán hệ thống, khối Fast-C2f_SimAM được tích hợp đồng bộ tại các vị trí chiến lược của mạng nền:

- Tầng đặc trưng P2/4 (repeats=3): Khối C2f được thay bằng Fast-C2f_SimAM nhằm tăng cường khả năng trích xuất đặc trưng biên và kết cấu của các đối tượng có kích thước nhỏ.
- Tầng đặc trưng P3/8 (repeats=6): Thực hiện thay thế để nâng cao khả năng học đặc trưng ở mức trung gian.
- Tầng đặc trưng P4/16 (repeats=6): Tiếp tục sử dụng Fast-C2f_SimAM để cải thiện quá trình biểu diễn các đặc trưng ngữ nghĩa có độ trừu tượng cao.
- Tầng đặc trưng P5/32 (repeats=3): Thay thế khối C2f trước SPPF nhằm tăng cường khả năng trích xuất đặc trưng sâu phục vụ giai đoạn tổng hợp đa tỷ lệ.

Việc phân bổ lặp lại (repeats) số lượng khối thích ứng theo độ sâu đảm bảo rằng dòng gradient được duy trì liên tục và mạng có đủ dung lượng để học các biểu diễn toán học từ đơn giản đến trừu tượng.

b. Cơ chế tri giác: Sự hỗ trợ của bộ chú ý SimAM ở giai đoạn trích xuất đặc trưng

Khối C2f nguyên bản hoạt động dựa trên cơ chế tích chập thông thường, xử lý mọi tọa độ pixel và mọi kênh đặc trưng với một bộ trọng số phẳng như nhau. Khi ảnh đầu vào chứa các yếu tố gây nhiễu mạnh như bóng đổ, sự thay đổi ánh sáng đột ngột, hoặc nền phức tạp (mặt đường, cây cối), C2f dễ bị bão hòa và trích xuất nhầm các đặc trưng nhiễu nền.

Sự xuất hiện của cơ chế chú ý SimAM (Simple Parameter-free Attention Module) trong khối Fast-C2f_SimAM đã giải quyết triệt để điểm yếu này. Khác với các cơ chế chú ý truyền thống (như SE, CBAM) đòi hỏi các lớp tích chập hoặc kết nối đầy đủ (FC) bổ sung để học trọng số kênh/không gian riêng biệt, SimAM định nghĩa toán học trực tiếp dựa trên khái niệm về năng lượng của nơ-ron (neuron energy).

Trong khoa học thần kinh, một nơ-ron hoạt động mạnh mẽ (vị trí chứa vật thể) sẽ có biểu hiện phân tách tuyến tính rõ rệt so với các nơ-ron xung quanh (vùng nền). Hàm mục tiêu năng lượng cho mỗi nơ-ron t trong một kênh được thiết lập dựa trên khoảng cách bình phương tối thiểu giữa nó và các nơ-ron khác x_i

$$e_m(t, w_m, b_m, y) = (y_t - \hat{t})^2 + \frac{1}{M-1} \sum_{i=1}^{M-1} (y_o - \hat{x}_i)^2 \quad (3.1)$$

Bằng cách giải phương trình tối ưu này ở dạng tường minh (analytical solution), SimAM tính toán ra một ma trận trọng số chú ý 3D (đầy đủ cả kích thước Kênh, Chiều cao, Chiều rộng) mà không sinh thêm bất kỳ tham số học tập nào (parameter-free). Tại giai đoạn trích xuất đặc trưng của Backbone, SimAM đóng vai trò như một màng lọc sinh học: nó tự động hạ thấp biên độ năng lượng của các nơ-ron thuộc vùng nền nhiễu và đẩy mạnh độ nhạy toán học (weights) tại các tọa độ pixel không gian chứa biên dạng đối tượng mục tiêu.

c. Tác động đến chất lượng bản đồ đặc trưng (Feature Map) đầu ra

Sự ghép nối giữa cấu trúc tính toán từng phần (Partial Convolution - PConv) của khối Fast và màng lọc năng lượng SimAM mang lại những thay đổi mang tính đột phá cho chất lượng các bản đồ đặc trưng đi ra khỏi Backbone:

- Tăng độ sắc nét hình học và tỷ số tín hiệu trên nhiễu (SNR): Các bản đồ đặc trưng, đặc biệt ở các mức đặc trưng nông (P2 và P3), giảm đáng kể hiện tượng nhiễu và mờ bối cảnh. Biên, góc cạnh và kết cấu của các đối tượng có kích thước nhỏ được bảo toàn rõ ràng hơn, tạo sự phân tách tốt giữa vùng đối tượng và nền.
- Bảo toàn tính toàn vẹn của thông tin đa quy mô: Nhờ cơ chế chú ý SimAM được áp dụng trực tiếp trên tensor đặc trưng kết hợp với cấu trúc Partial Convolution của Fast-C2f, mối quan hệ không gian giữa các đặc trưng cục bộ và ngữ nghĩa toàn cục được duy trì hiệu quả. Các bản đồ đặc trưng đầu ra của Backbone ở các mức P2, P3, P4 và P5 có chất lượng biểu diễn cao hơn, giảm nhiễu và tăng tính nhất quán về ngữ nghĩa. Điều này tạo điều kiện thuận lợi cho khối SimFusion_4in trong mạng Neck thực hiện quá trình dung hợp đặc trưng đa quy mô, đồng thời hạn chế hiện tượng mất mát hoặc sai lệch thông tin giữa các nhánh truyền đặc trưng.

d. Lý do đề xuất thay thế khối C2f bằng khối Fast_C2f_SimAM

Việc thay thế khối C2f tiêu chuẩn bằng khối Fast_C2f_SimAM xuất phát từ mục tiêu tối ưu hóa sự cân bằng giữa độ chính xác (accuracy) và tốc độ suy luận (inference speed). Đối với các ứng dụng đòi hỏi xử lý thời gian thực hoặc triển khai trên các thiết bị biên (edge devices) có tài nguyên hạn chế, cấu trúc nguyên bản bộc lộ một số nhược điểm nhất định. Đề xuất cải tiến này dựa trên hai cơ sở lý thuyết cốt lõi:

- Tối ưu hóa tính toán và giảm độ trễ nhờ Partial Convolution (PConv): Khối C2f truyền thống sử dụng các phép tích chập tiêu chuẩn (Standard Convolution), tính toán trên toàn bộ số lượng kênh của đặc trưng đầu vào. Điều này gây ra sự dư thừa thông tin lớn (spatial và channel redundancy), dẫn đến số lượng phép toán dấu phẩy động (FLOPs) và chi phí truy xuất bộ nhớ (MAC) rất cao. Bằng cách thay thế bằng cấu trúc Fast_C2f, mạng sử dụng Partial Convolution để chỉ áp dụng bộ lọc trên một tỷ lệ kênh nhất định (thường là $\frac{1}{4}$ số kênh). Sự thay đổi này giúp cắt giảm triệt để khối lượng tính toán (như đã chứng minh qua công thức $FLOPs_{PConv} = \frac{1}{16} \times FLOPs_{Standard}$), làm cho mô hình trở nên nhẹ hơn và tăng đáng kể tốc độ khung hình trên giây (FPS).
- Bù đắp thông tin và tăng cường đặc trưng với cơ chế chú ý SimAM: Hệ quả tất yếu của việc giảm bớt kênh tính toán bằng PConv là mô hình có nguy cơ đánh mất các thông tin ngữ cảnh quan trọng, dẫn đến suy giảm độ chính xác phát hiện. Để giải quyết bài toán đánh đổi này, mô hình được tích hợp thêm module chú ý SimAM (Simple, Parameter-Free Attention Module). Khác với các cơ chế chú ý truyền thống (như CBAM hay SE) đòi hỏi chèn thêm các lớp fully-connected làm tăng số lượng tham số huấn luyện, SimAM hoàn toàn không chứa tham số học (parameter-free). Khối này đánh giá mức độ quan trọng của từng nơ-ron thông qua một hàm năng lượng cục bộ. Bằng cách gán trọng số cao cho các nơ-ron có mức năng lượng tối thiểu (e_t^*) – biểu thị tính đặc trưng và khác biệt cao so với môi trường xung quanh – SimAM giúp mạng tập trung mạnh mẽ vào các đặc trưng của đối tượng mục tiêu và làm mờ nhiễu nền.

3.2.3. Cải tiến 2 — Thay thế C2f → Fast-C2f trong Head

Mạng Neck và Head đóng vai trò then chốt trong việc tinh chỉnh đặc trưng và thực hiện các phép toán hậu dung hợp trước khi đưa dữ liệu vào đầu dự đoán phân tách (Decoupled Head). Để giải quyết triệt để hiện tượng nghẽn cổ chai bộ nhớ và tối ưu hóa tốc độ thực thi thời gian thực, nghiên cứu đề xuất thay thế cấu trúc C2f công kênh bằng khối siêu nhẹ Fast-C2f tại các tầng xử lý dữ liệu sau dung hợp.

a. Vị trí thực thi cấu trúc

Dựa trên kiến trúc GPFS_YOLO, các khối Fast-C2f được tích hợp tại các nhánh xử lý đặc trưng sau giai đoạn Information Injection, thay thế cho các khối C2f truyền thống nhằm giảm độ phức tạp tính toán nhưng vẫn duy trì khả năng tinh chỉnh đặc trưng. Việc bố trí này được thực hiện tại ba mức đặc trưng của mạng Neck:

- Nhánh đặc trưng P4: Fast-C2f được đặt ngay sau khối Information Injection, có nhiệm vụ tinh chỉnh các đặc trưng ngữ nghĩa sâu sau khi tiếp nhận thông tin ngữ cảnh toàn cục từ Backbone.
- Nhánh đặc trưng P3: Sau quá trình tiêm thông tin đa quy mô, Fast-C2f tiếp tục thực hiện tái tổ chức và tăng cường biểu diễn đặc trưng ở mức trung gian, giúp cải thiện khả năng kết hợp giữa thông tin không gian và ngữ nghĩa.
- Nhánh đặc trưng P4 sau dung hợp: Sau khi đặc trưng được hợp nhất thông qua các khối SimFusion và AdvPoolFusion, Fast-C2f tiếp tục đảm nhiệm vai trò tinh chỉnh đặc trưng trước khi truyền đến đầu phát hiện (Detection Head), góp phần nâng cao chất lượng biểu diễn của các đặc trưng đầu vào cho quá trình dự đoán.

Việc thay thế các khối C2f bằng Fast-C2f tại các vị trí này không làm thay đổi cấu trúc hay luồng truyền dữ liệu của mạng Neck, mà chỉ tối ưu module xử lý bên trong. Nhờ đó, mô hình vừa giảm số lượng tham số và chi phí tính toán, vừa duy trì hiệu quả dung hợp đặc trưng đa quy mô trước khi đưa đến tầng phát hiện.

b. Lý do không tích hợp cơ chế chú ý SimAM tại mạng Head

Việc không tích hợp cơ chế chú ý SimAM tại mạng Head được lựa chọn dựa trên vai trò của từng thành phần trong kiến trúc GPFS_YOLO cũng như đặc điểm của quá trình trích xuất đặc trưng đa mức.

Trước khi đặc trưng được truyền đến mạng Head, các bản đồ đặc trưng đã trải qua quá trình trích xuất tại Backbone và dung hợp đa quy mô thông qua cơ chế Gather-and-Distribute trong mạng Neck. Tại đây, thông tin từ các mức đặc trưng P2, P3, P4 và P5 được kết hợp thông qua các khối SimFusion và Information Fusion Module (IFM), giúp tăng cường khả năng biểu diễn ngữ nghĩa và duy trì mối liên hệ giữa thông tin cục bộ với ngữ cảnh toàn cục. Do đó, các đặc trưng đầu vào của Head đã chứa lượng thông tin ngữ nghĩa tương đối đầy đủ để phục vụ quá trình phân loại và hồi quy hộp giới hạn.

Bên cạnh đó, cơ chế SimAM đã được tích hợp trong Backbone nhằm tăng cường khả năng làm nổi bật các vùng đặc trưng quan trọng ngay từ giai đoạn đầu của quá trình trích xuất đặc trưng. Việc tiếp tục áp dụng SimAM tại Head có thể tạo ra sự chồng lặp về chức

năng với các khối chú ý trước đó, trong khi mức cải thiện về độ chính xác dự kiến không đáng kể. Vì vậy, nghiên cứu lựa chọn chỉ sử dụng Fast-C2f để tinh chỉnh đặc trưng tại Head nhằm giữ cho kiến trúc gọn nhẹ và tránh làm tăng độ phức tạp của mô hình.

Ngoài ra, mặc dù SimAM là cơ chế chú ý không bổ sung tham số học, mô-đun này vẫn phát sinh các phép tính bổ sung như tính trung bình, phương sai và hàm kích hoạt trên từng tensor đặc trưng. Khi được áp dụng tại Head, nơi các đặc trưng phải được xử lý liên tục để thực hiện phân loại và hồi quy đối tượng, các phép tính này sẽ làm tăng chi phí tính toán và thời gian suy luận. Việc loại bỏ SimAM ở giai đoạn cuối giúp rút ngắn đường truyền tính toán, giảm độ trễ suy luận và góp phần nâng cao hiệu quả triển khai trên các thiết bị có tài nguyên hạn chế mà vẫn duy trì chất lượng phát hiện đối tượng.

c. Tác động đến tốc độ suy luận (Inference Speed) của mô hình

Việc cấu hình khối Fast-C2f thuần túy tại Head mang lại những cải thiện mang tính bước ngoặt đối với hiệu năng vận hành thực tế của hệ thống:

- Giải phóng chi phí truy cập bộ nhớ (Memory Access Cost - MAC): Khối C2f truyền thống liên tục thực hiện tách kênh và nối chuỗi tensor (Concat) trên diện rộng, tạo áp lực khổng lồ lên băng thông đọc/ghi của bộ nhớ đệm (Cache) trên chip CPU ARM (như Raspberry Pi) hoặc GPU biên. Fast-C2f với cốt lõi là tích chập từng phần (PConv) chỉ xử lý tính toán trên một lượng kênh đại diện (1/4 hoặc 1/2 số kênh), giữ nguyên các kênh còn lại để truyền thẳng. Chiến lược này giúp giảm chỉ số MAC một cách tuyến tính, giúp dòng dữ liệu trôi qua mạng Head với tốc độ cực cao.
- Tăng tốc số khung hình trên giây (FPS) thực tế: Sự cắt giảm khối lượng tính toán GFLOPs tại phần Head trực tiếp làm giảm thời gian xử lý của mỗi ảnh (Latency). Kết quả thực nghiệm cho thấy, việc thay thế C2f bằng Fast-C2f tại Head là chìa khóa chính giúp mô hình GPFS_YOLO đạt được sự tăng trưởng vượt bậc về chỉ số FPS, biến một kiến trúc nặng nề như Gold-YOLO thành một mô hình siêu nhẹ, vận hành mượt mà và đáp ứng hoàn hảo các ràng buộc khắt khe của bài toán nhận diện thời gian thực trên các thiết bị Edge AI.

3.2.4. Cải tiến 3 — Bổ sung nhánh phát hiện vật thể nhỏ P2

Đối với bài toán phát hiện các đối tượng có kích thước nhỏ, cấu trúc phát hiện ba mức đặc trưng P3, P4 và P5 của YOLOv8 thường gặp hiện tượng suy giảm thông tin chi tiết do quá trình giảm kích thước bản đồ đặc trưng qua nhiều lần lấy mẫu xuống (Downsampling). Khi độ phân giải không gian giảm, các đối tượng nhỏ chỉ còn rất ít điểm biểu diễn trên bản đồ đặc trưng, làm giảm khả năng phân biệt giữa đối tượng và nền. Đồng thời, trường tiếp nhận (Receptive Field - RF) của các tầng sâu ngày càng lớn so với kích thước thực của đối tượng ($RF \gg S_{obj}$), khiến thông tin ngữ nghĩa của vật thể nhỏ dễ bị hòa lẫn với thông tin nền, dẫn đến hiện tượng mất mát đặc trưng.

Để khắc phục hạn chế này, nghiên cứu đề xuất bổ sung nhánh phát hiện tại mức đặc trưng P2 (stride = 4). Việc bổ sung nhánh P2 giúp đưa các đặc trưng có độ phân giải cao trực tiếp vào đầu phát hiện, tạo điều kiện để mô hình khai thác hiệu quả các chi tiết về biên, kết cấu và hình dạng của các đối tượng nhỏ. Khi đó, trường tiếp nhận của đặc trưng P2 trở nên phù hợp hơn với kích thước thực của vật thể ($RF \approx S_{obj}$), giúp tăng khả năng định vị và phân loại các đối tượng có kích thước nhỏ.

Về mặt kiến trúc, nhánh P2 được xây dựng bằng cách lấy đặc trưng từ tầng P3 của mạng Neck, thực hiện phép Upsample để tăng độ phân giải, sau đó kết hợp (Concat) với đặc trưng P2 được truyền từ Backbone. Sau quá trình hợp nhất đặc trưng, dữ liệu tiếp tục được tinh chỉnh thông qua khối Fast-C2f trước khi được đưa đến đầu phát hiện (Detection Head) để thực hiện dự đoán. Cách thiết kế này vừa tận dụng được thông tin ngữ nghĩa từ các tầng sâu, vừa bảo toàn các đặc trưng không gian có độ phân giải cao của Backbone, từ đó nâng cao khả năng phát hiện các vật thể nhỏ mà không làm thay đổi nguyên lý hoạt động của các nhánh phát hiện còn lại.

Việc bổ sung nhánh P2 giúp mô hình mở rộng từ ba mức phát hiện (P3, P4, P5) lên bốn mức phát hiện (P2, P3, P4, P5), tăng khả năng xử lý các đối tượng xuất hiện ở nhiều kích thước khác nhau. Đặc biệt, đối với các đối tượng nhỏ hoặc ở khoảng cách xa, nhánh P2 cung cấp nhiều thông tin chi tiết hơn, góp phần cải thiện độ chính xác định vị và nâng cao hiệu quả phát hiện tổng thể của mô hình.

a. Phép nội suy tăng độ phân giải (Upsample)

Đầu vào của nhánh P2 là bản đồ đặc trưng P3 sau khi đã được tăng cường thông tin ngữ nghĩa thông qua quá trình dung hợp đa quy mô trong mạng Neck. Để có thể kết hợp với đặc trưng P2 của Backbone, bản đồ đặc trưng P3 được đưa qua lớp Upsample với hệ số phóng đại 2 bằng phương pháp nội suy lân cận gần nhất (Nearest Neighbor

Interpolation). Quá trình này làm tăng kích thước bản đồ đặc trưng từ $H/8 \times W/8$ lên $H/4 \times W/4$, đồng thời giữ nguyên số lượng kênh đặc trưng.

Việc tăng độ phân giải giúp các đặc trưng ngữ nghĩa ở tầng sâu được căn chỉnh về cùng kích thước không gian với đặc trưng P2, tạo điều kiện cho quá trình dung hợp đa mức ở bước tiếp theo.

b. Dung hợp đặc trưng bằng phép nối kênh (Concat)

Sau khi được tăng độ phân giải, bản đồ đặc trưng P3 được kết hợp với bản đồ đặc trưng P2 truyền trực tiếp từ Backbone thông qua phép Concatenate theo chiều kênh.

Sự kết hợp này cho phép mô hình khai thác đồng thời ưu điểm của hai mức đặc trưng. Đặc trưng P2 có độ phân giải không gian cao, chứa nhiều thông tin về biên, kết cấu và hình dạng của các đối tượng nhỏ nhưng còn hạn chế về thông tin ngữ nghĩa. Ngược lại, đặc trưng P3 chứa nhiều thông tin ngữ nghĩa hơn nhờ đã trải qua quá trình dung hợp đa quy mô, tuy nhiên độ chi tiết không gian đã giảm sau nhiều lần lấy mẫu xuống. Việc hợp nhất hai nguồn đặc trưng giúp tạo ra bản đồ đặc trưng vừa giàu thông tin ngữ nghĩa, vừa bảo toàn được các chi tiết cục bộ của đối tượng nhỏ, góp phần nâng cao khả năng phát hiện các vật thể có kích thước nhỏ.

c. Tinh chỉnh đặc trưng bằng Fast-C2f

Sau phép nối kênh, bản đồ đặc trưng được đưa vào khối Fast-C2f để thực hiện quá trình tinh chỉnh và tái tổ chức đặc trưng trước khi chuyển đến đầu phát hiện.

Do đặc trưng đầu vào được tổng hợp từ hai nguồn có mức độ trừu tượng khác nhau, phân bố đặc trưng sau phép nối có thể chưa đồng nhất. Khối Fast-C2f thực hiện quá trình học lại đặc trưng, tăng cường các thông tin hữu ích và giảm ảnh hưởng của các đặc trưng dư thừa phát sinh trong quá trình dung hợp. Bên cạnh đó, nhờ sử dụng cơ chế Partial Convolution (PConv), Fast-C2f chỉ thực hiện tích chập trên một phần số kênh thay vì toàn bộ tensor đặc trưng, giúp giảm đáng kể số lượng phép tính và số tham số trong khi vẫn duy trì khả năng biểu diễn đặc trưng. Điều này đặc biệt hiệu quả đối với nhánh P2, nơi bản đồ đặc trưng có độ phân giải lớn ($H/4 \times W/4$) và chi phí tính toán thường cao hơn các mức đặc trưng còn lại.

3.2.5. Cải tiến 4 – Thay thế hàm kích hoạt SiLU và ReLU sang Mish, thay đổi cơ chế tính toán hàm mất mát CIoU sang WioUv3 với $\alpha = 1.9$, $\beta = 3$

Để nâng cao chất lượng huấn luyện và khả năng tối ưu hóa của mô hình, nghiên cứu đề xuất hai cải tiến quan trọng ở cấp độ hàm số: thay thế hàm kích hoạt mặc định SiLU

(và ReLU ở một số khối) bằng hàm Mish trong toàn bộ các lớp convolution và các khối Neck của Gold-YOLO, đồng thời thay thế hàm mất mát CIOU bằng WIOU (Wise-IoU).

Hàm kích hoạt SiLU tuy đã mang lại hiệu suất tốt, nhưng vẫn tồn tại một số hạn chế về độ mượt của gradient và khả năng truyền tải thông tin âm. Ngược lại, Mish là hàm kích hoạt smooth, non-monotonic và self-regularized, giúp cải thiện đáng kể dòng gradient trong quá trình lan truyền ngược, tăng cường khả năng biểu diễn đặc trưng và hỗ trợ mô hình hội tụ ổn định hơn, đặc biệt trong các mạng sâu.

Về hàm mất mát, CIOU mặc dù xem xét cả overlap, khoảng cách trung tâm và tỷ lệ khung, nhưng vẫn chưa tối ưu trong việc xử lý các anchor frame chất lượng thấp hoặc các mẫu cục đoạn. WIOU khắc phục hạn chế này nhờ cơ chế dynamic non-monotonic focusing, cho phép mô hình tập trung mạnh mẽ hơn vào các anchor frame có chất lượng thông thường, giảm ảnh hưởng của gradient từ các mẫu khó (outlier), từ đó nâng cao độ chính xác định vị và khả năng khái quát hóa của mô hình.

a. Thay thế hàm kích hoạt SiLU bằng Mish

Toàn bộ các lớp convolution trong Backbone, Neck và Head đều được thay thế hàm kích hoạt SiLU (Swish) mặc định bằng hàm Mish.

Hàm Mish sở hữu tính liên tục khả vi vô hạn (C^∞), không có điểm không khả vi như ReLU, đồng thời giữ lại một lượng nhỏ thông tin âm, giúp tránh hiện tượng “dying neuron” và cải thiện dòng gradient. Việc áp dụng Mish trên toàn mạng mang lại lợi ích kép: tăng cường khả năng học đặc trưng mịn và ổn định quá trình huấn luyện mà không làm tăng chi phí tính toán.

b. Thay đổi hàm mất mát từ CIOU sang WIOUv3

Hàm mất mát được thay bằng WIOU. WIOUv3 là phiên bản nâng cao của WIOU, được xây dựng dựa trên cơ chế **dynamic non-monotonic focusing**. Công thức đầy đủ của WIOUv3 như sau:

$$L_{WIOUv3} = L_{WIOU} \times r \quad (3.2)$$

Và non-monotonic focus factor r được tính theo công thức:

$$r = \frac{\beta}{\delta^{\alpha\beta-\delta}} \quad (3.3)$$

Với các siêu tham số thường dùng: $\alpha = 1.9$, $\beta = 3$.

c. Tích hợp và tác động tổng thể

Việc kết hợp Mish và WIoU tạo nên sự bổ trợ lẫn nhau: Mish cải thiện chất lượng đặc trưng và gradient flow trong quá trình lan truyền, trong khi WIoU tối ưu hóa hướng cập nhật tham số dựa trên chất lượng anchor. Sự phối hợp này giúp mô hình hội tụ nhanh hơn, giảm overfitting và nâng cao đáng kể độ chính xác phát hiện, đặc biệt với các vật thể nhỏ, bị che khuất hoặc có quy mô đa dạng trong môi trường thực tế.

Nhờ hai cải tiến này, GPFS_YOLO đạt được sự cân bằng tối ưu giữa độ chính xác cao và hiệu quả huấn luyện, góp phần quan trọng vào thành công tổng thể của mô hình.

3.3. So sánh kiến trúc 3 model

Bảng 3: So sánh kiến trúc 3 model

Thành phần	YOLOv8 chuẩn	YOLOv8-Gold	GPFS (đề xuất)
Backbone block	C2f	C2f	Fast-C2f_SimAM
Head type	FPN + C2f	Gold head + C2f	Gold head + Fast-C2f
Attention	Không	Không	SimAM (Backbone)
Detection scales	P3, P4, P5	P3, P4, P5	P2, P3, P4
Small obj. branch	Không	Không	Có (P2)
Hàm kích hoạt	SiLU	SiLU + ReLU	Mish
Hàm mất mát	CIoU	CIoU	WIoUv3

Nhận xét chung về kiến trúc đề xuất (GPFS_YOLO)

Bảng so sánh cho thấy GPFS_YOLO là một bản nâng cấp toàn diện từ YOLOv8 và Gold-YOLO. Mô hình được tái cấu trúc sâu sắc để tối ưu hóa cả độ chính xác lẫn hiệu năng thông qua 4 chiến lược cốt lõi:

- **Tối ưu hóa mạng nền (Backbone):** Việc thay thế khối C2f truyền thống bằng Fast-C2f_SimAM mang lại "lợi ích kép". Cơ chế tính toán từng phần (PConv) giúp cắt giảm khối lượng tính toán dư thừa, trong khi module chú ý 3D SimAM đóng vai trò lọc nhiễu nền và tập trung vào đặc trưng vật thể ngay từ giai đoạn sớm.

- Giải quyết nghẽn cổ chai tại mạng Head: Để khắc phục tình trạng quá tải bộ nhớ sau quá trình dung hợp ngữ cảnh của Gold-YOLO, mô hình sử dụng cấu trúc siêu nhẹ Fast-C2f tại Head. Điều này giúp luồng dữ liệu truyền dẫn mượt mà, đảm bảo tốc độ suy luận cực nhanh, lý tưởng cho môi trường Edge AI.
- Tái định hình dải dự đoán và bổ sung nhánh P2: Đây là cải tiến mang tính quyết định. Mô hình loại bỏ tầng đặc trưng sâu P5 dư thừa và dịch chuyển dải dự đoán về quy mô nhỏ hơn: [P2, P3, P4]. Nhánh P2 với độ phân giải không gian cao giúp bắt dính các chi tiết hình học siêu nhỏ, khắc phục hoàn toàn hiện tượng pha loãng đặc trưng ở các phiên bản cũ.
- Cải tiến hàm kích hoạt và hàm mất mát: Toàn bộ các lớp convolution và khối Neck được thay thế hàm kích hoạt SiLU bằng Mish — hàm kích hoạt smooth, non-monotonic và self-regularized — nhằm cải thiện gradient flow, tăng cường khả năng biểu diễn đặc trưng và hỗ trợ huấn luyện ổn định hơn. Đồng thời, hàm mất mát CIOU được thay bằng WIoUv3, với cơ chế dynamic non-monotonic focusing giúp mô hình tập trung vào các anchor frame chất lượng thông thường, giảm ảnh hưởng của gradient từ các mẫu outlier, từ đó nâng cao đáng kể độ chính xác định vị và khả năng khái quát hóa.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

4.1. Dữ liệu thực nghiệm

4.1.1. Tổng quan về bộ dữ liệu gốc và tính phù hợp

Các thực nghiệm và đánh giá hiệu năng trong nghiên cứu này được tiến hành dựa trên bộ dữ liệu KITTI (KITTI Vision Benchmark Suite) — một trong những bộ dữ liệu chuẩn mực và thách thức nhất hiện nay trong lĩnh vực thị giác máy tính dành cho các hệ thống xe tự hành (Autonomous Driving Systems).

Nghiên cứu tập trung vào bài toán tối ưu hóa mô hình phát hiện đối tượng vật thể là người đi bộ từ xa, một kịch bản đòi hỏi mạng nơ-ron phải có khả năng bóc tách thông tin đặc trưng từ các đối tượng ở kích thước nhỏ và siêu nhỏ, dao động trong khoảng từ 10×10 đến 20×20 pixels. Bộ dữ liệu KITTI phản ánh tương đối đầy đủ, chân thực các kịch bản giao thông phức tạp với các đặc tính: đối tượng bị mờ nhòe do chuyển động, hiện tượng che khuất một phần hoặc toàn bộ (occlusion), và đặc biệt là sự biến thiên biên độ lớn về mặt tỷ lệ hình học. Sự xuất hiện phong phú của lớp đối tượng người đi bộ ở khoảng cách xa (mật độ pixel trên ảnh thấp) biến KITTI thành nguồn dữ liệu cốt lõi, phù hợp hoàn toàn với mục tiêu nhận diện người đi bộ từ xa của nghiên cứu này.

4.1.2. Quá trình lọc bỏ nhiễu và tái cấu trúc hệ thống nhãn (Data Preprocessing)

Tập dữ liệu gốc của KITTI chứa tổng cộng 9 lớp đối tượng được gán nhãn trong môi trường 2D và 3D. Tuy nhiên, để tập trung tài nguyên tính toán vào mục tiêu cốt lõi của bài toán (nhận diện người đi bộ và các phương tiện giao thông đường bộ chính), nhóm nghiên cứu đã thực hiện các bước lọc dữ liệu nhiễu và định cấu hình lại hệ thống nhãn theo định dạng YOLO.

Các lớp không liên quan trực tiếp đến kịch bản di chuyển của hệ thống xe tự hành bao gồm Tram (Tàu điện), Misc (Đối tượng hỗn hợp khác) và DontCare (Vùng không quan tâm - chứa các vật thể quá xa hoặc mờ mà hệ thống bỏ qua biên) đã bị loại bỏ hoàn toàn nhằm tránh gây nhiễu cho hàm mất mát (loss function) trong quá trình hội tụ. Đối với 6 lớp đối tượng còn lại, để giảm thiểu hiện tượng phân mảnh nhãn và tăng mật độ mẫu huấn luyện trên từng danh mục, nghiên cứu tiến hành gộp nhóm các cấu trúc có sự tương đồng cao về mặt ngữ cảnh và hình học:

- Lớp Người đi bộ (Person): Tiến hành tích hợp nhãn của hai lớp Pedestrian (Người đi bộ) và Person_sitting (Người đang ngồi trong không gian công cộng) từ tập gốc để đồng nhất thành lớp Person (ID: 0).
- Lớp Phương tiện và người điều khiển (Vehicle & Cyclist): Giữ lại các cấu trúc phân loại độc lập bao gồm Car (Ô tô con - ID: 1), Cyclist (Người đi xe đạp/xe máy - ID: 2), Van (Xe bán tải/xe khách cỡ nhỏ - ID: 3) và Truck (Xe tải - ID: 4).

Sau giai đoạn tiền xử lý và lọc nhãn hợp lệ, tập dữ liệu trích xuất thu được tổng cộng 7.481 hình ảnh chất lượng cao phục vụ thực nghiệm.

4.1.3. Phân chia dữ liệu và giải quyết thách thức mất cân bằng nhãn (Dataset Partitioning)

Toàn bộ tập dữ liệu sau khi tái cấu trúc được phân chia một cách ngẫu nhiên có kiểm soát theo tỷ lệ nghiêm ngặt 8:1:1, tương ứng với ba tập con độc lập: Tập huấn luyện (Training set - chiếm 80%), Tập kiểm định (Validation set - chiếm 10%) và Tập kiểm thử (Test set - chiếm 10%). Việc phân tách riêng biệt tập kiểm định (Val) dùng để tinh chỉnh siêu tham số và tập kiểm thử (Test) dùng để đánh giá hiệu năng cuối cùng giúp đảm bảo mô hình không bị hiện tượng quá khớp (overfitting), đồng thời giữ cho các chỉ số đánh giá (mAP, Precision, Recall) đạt tính khách quan tối đa.

Mặc dù hệ thống nhãn đã được thu gọn, đặc trưng phân phối phân lớp kinh điển của bộ dữ liệu KITTI vẫn tồn tại: hiện tượng mất cân bằng số lượng đối tượng (Class Imbalance) nghiêm trọng. Số lượng hộp bao (bounding boxes) của lớp Car chiếm tỷ trọng áp đảo hoàn toàn so với các lớp còn lại, đặc biệt là các lớp có kích thước nhỏ và tần suất xuất hiện thấp như Person và Cyclist.

Sự chênh lệch lớn này chính là cơ sở khoa học và động lực cốt lõi để nhóm nghiên cứu đề xuất giải pháp: bổ sung một đầu phát hiện kích thước siêu nhỏ (Small-scale Target Detection Head) với độ phân giải bản đồ đặc trưng là 160×160 pixels vào kiến trúc mạng. Đầu phát hiện cải tiến này giúp tăng cường mật độ lấy mẫu không gian, duy trì các đặc trưng hình học ở tầng nông, từ đó nâng cao đáng kể độ nhạy (Sensitivity) và kiểm soát hiện tượng bỏ sót (False Negatives) đối với các đối tượng dễ tổn thương về mặt dữ liệu như người đi bộ ở khoảng cách xa hoặc người đi xe đạp.

Phân phối chi tiết số lượng ảnh và số lượng nhãn đối tượng cụ thể sau khi thực hiện mã nguồn chia tách dữ liệu được thống kê định lượng trong Bảng 3:

Bảng 4: Thống kê phân phối số lượng nhãn đối tượng trong tập dữ liệu thực nghiệm

Tập dữ liệu	SL ảnh	Person	Car	Cyclist	Van	Truck	Tổng số nhãn
Train	5984	3783	23062	1321	2351	896	31413
Val	749	464	2829	161	281	93	3828
Test	748	462	2851	145	282	105	3845
Tổng cộng	7481	4709	28742	1627	2914	1094	39086

4.2. Thông số và môi trường của thiết bị thử nghiệm

Toàn bộ quá trình huấn luyện mô hình được thực hiện trên máy tính cá nhân sử dụng GPU NVIDIA GeForce RTX 3050 Ti Laptop GPU với bộ nhớ đồ họa 4 GB VRAM. Môi trường phát triển được xây dựng trên hệ điều hành Windows 11, sử dụng môi trường Python 3.10.11 làm ngôn ngữ phát triển chính của đề tài.

Mô hình được triển khai và huấn luyện trên nền tảng PyTorch bằng opensource code yolov8 của thư viện Ultralytics. Thư viện Pytorch đã được cài đặt với khả năng tối ưu và tăng tốc bằng CUDA, cho phép chúng ta khai thác năng lực tính toán song song một cách nhanh chóng của GPU nhằm giảm tải trọng cho CPU thực hiện các phép tính phức tạp thay vì làm những phép tính đơn giản nhằm rút ngắn thời gian huấn luyện của mô hình và đánh giá mô hình.

4.2.1. Cấu hình phần cứng

Toàn bộ mô hình được huấn luyện và kiểm tra trên máy tính xách tay có thông số chính thể hiện ở bảng 4

Bảng 5: Cấu hình phần cứng

Thành phần	Cấu hình
CPU	AMD Ryzen 7 6800H with Radeon Graphics (3.20 GHz)
RAM	16GB
GPU	NVIDIA GeForce RTX 3050 Ti
VRAM	4GB

4.2.2. Môi trường huấn luyện

Môi trường huấn luyện của các mô hình được thể hiện đầy đủ ở bảng 6.

Bảng 6: Môi trường huấn luyện

Thành phần	Phiên bản
Python	3.10.11
PyTorch	2.5.1+cu121
TorchVision	0.20.1+cu121
Ultralytics YOLO	8.4.51
CUDA	12.1
OS	Windows 11

4.2.3. Tham số huấn luyện

Các tham số thiết lập chi tiết cho máy khi thực hiện huấn luyện và kiểm tra thể hiện chi tiết ở bảng 6.

Bảng 7: Tham số truyền trong quá trình huấn luyện

Tham số	Giá trị
Epochs	50
Batch size	16
Image size	640x640
Optimizer	AdamW
Learning rate	0.001
CloseMosaic	10

4.2.4. Thông số của thiết bị nhúng.

Sau khi thực hiện huấn luyện cách mô hình và kiểm tra chúng trên thiết bị huấn luyện, việc thực hiện kiểm tra mô hình chạy trên phần cứng nhúng là điều cần thiết cho mục tiêu hướng đến của đề tài là ứng dụng trên cách xe máy tự hành. Việc kiểm tra mô hình trên thiết bị nhúng Raspberry Pi4 Model B 8GB Ram cho ta thấy được phần hạn chế của những mô hình lớn như Yolov8m. Các thông số của Raspberry Pi 4 thể hiện ở bảng 8.

Bảng 8: Thông số của Raspberry Pi 4

Thành phần	Thông số
Thiết bị	Raspberry Pi 4 Model B
CPU	Broadcom BCM2711, Quad-core ARM Cortex-A72

Xung nhịp	1.5 GHz
RAM	8 GB LPDDR4
GPU	VideoCore VI
Hệ điều hành	Debian
Framework AI	PyTorch / ONNX Runtime / TensorRT

Mô hình được triển khai và đánh giá trên nền tảng Raspberry Pi4 Model B sẽ cho những dữ kiện đầy đủ cho việc đánh giá mô hình có hiệu quả trên thiết bị hạn chế tài nguyên và khả năng tính toán khi không có GPU xử lý AI được không.

4.3. Kết quả và so sánh theo baseline

4.3.1. Kết quả thực nghiệm.

a. Kết quả thực nghiệm các trường hợp thay đổi

Các cải tiến được thực hiện từng phương pháp thêm vào lần lượt với kết quả thể hiện trong bảng 9:

Bảng 9: Kết quả thử nghiệm các cải tiến

Model	GoldYOLO	P2 - P5	FastC2f	SimAM	Mish + WIoU	Inference Time (ms)	mAP@ 0.5	mAP@ 0.5:0.95
Exp1 (YOLOv8m)						7.5	0.918	0.674
Exp2	✓					9.1	0.920	0.674
Exp3	✓	✓				10.4	0.928	0.699
Exp4	✓	✓	✓			10.1	0.927	0.699
Exp5	✓	✓	✓	✓		9.9	0.908	0.653
Exp6	✓	✓	✓	✓	✓	9.7	0.915	0.654

Bảng kết quả cho thấy được mô hình sau nhiều cải tiến thì độ chính xác [mAP@0.5](#) vẫn ngang nhau chính giảm độ chính xác khung [mAP@0.5:0.95](#) xuống khoảng 2% thời gian suy luận thì tăng lên từ 7.5ms lên 9.7ms. Tuy nhiên hướng cải tiến đi theo hướng giảm kích thước mô hình nhưng vẫn phải giữ được độ chính xác thì kết quả giảm từ 25 triệu tham số xuống còn 13 tham số thì sự mất mát độ chính xác này trong tầm chấp nhận được.

Độ chính xác đạt đỉnh ở thí nghiệm thứ 3 khi ta thêm phát hiện nhánh P2 và bỏ nhánh P5, tuy số lượng tham số giảm đi nhưng khối lượng phải tính toán tăng lên từ 76 Gflop của mô hình Gold-YOLO lên 83,9 Gflop làm thời gian tính toán của mô hình tăng cao từ 9.1 ms lên 10.4ms con số cao nhất trong các thử nghiệm.

b. Kết quả mô hình cải tiến cuối.

Mô hình AI sẽ được kiểm tra và đánh giá độ chính xác trên 2 thiết bị là máy tính xách tay với GPU RTX 2050 TI 4GB VRAM và Board nhúng Raspberry Pi 4 8GB Ram không

có card đồ họa để có cái nhìn trực quan về mô hình nhận diện các đối tượng trong xe tự hành khi deploy trên thiết bị có sự hỗ trợ của GPU và thiết bị ngoại biên nhưng không có GPU là Raspberry Pi4.

Trước tiên mô hình được tiến hành đánh giá bằng tập dữ liệu kiểm định từ 10% của tập dữ liệu trên cùng thiết bị huấn luyện.

Bảng 10: Kết quả đánh giá mô hình cải tiến

Class	Precision	Recall	mAP50	mAP50-95
all	0.906	0.846	0.915	0.654
Person	0.873	0.695	0.818	0.444
Car	0.94	0.925	0.975	0.783
Cyclist	0.876	0.829	0.882	0.552
Van	0.91	0.894	0.946	0.723
Truck	0.93	0.889	0.952	0.766

Inference time: Trên máy tính (GPU RTX 3050, ảnh kích thước 640×640): 9.7 ms

Bảng 11: Kết quả đánh giá mô hình Yolov8m

Class	Precision	Recall	mAP50	mAP50-95
all	0.927	0.842	0.918	0.674
Person	0.887	0.71	0.828	0.459
Car	0.944	0.92	0.968	0.799
Cyclist	0.919	0.821	0.883	0.584
Van	0.94	0.884	0.951	0.748
Truck	0.946	0.876	0.96	0.782

Inference time: Trên máy tính (GPU RTX 3050, ảnh kích thước 640×640): 7.5 ms

Bảng 12: Kết quả đánh giá mô hình Yologold

Class	Precision	Recall	mAP50	mAP50-95
all	0.904	0.857	0.92	0.674
Person	0.876	0.706	0.81	0.451
Car	0.937	0.925	0.967	0.791
Cyclist	0.826	0.819	0.892	0.57
Van	0.93	0.883	0.952	0.749

Truck	0.953	0.952	0.978	0.807
-------	-------	-------	-------	-------

Inference time: Trên máy tính (GPU RTX 3050, ảnh kích thước 640×640): 9.1 ms

Kết quả đánh giá trong Bảng 10, Bảng 11 và Bảng 12 cho thấy mô hình cải tiến GPFS_YOLO vẫn duy trì hiệu năng phát hiện ở mức cao mặc dù đã được tối ưu mạnh về số lượng tham số. Cụ thể, mô hình giảm từ khoảng 25 triệu tham số của YOLOv8m xuống còn khoảng 13 triệu tham số, tương đương giảm gần 48%. Các chỉ số đánh giá chỉ suy giảm nhẹ, chứng tỏ các cải tiến đề xuất đã nâng cao hiệu quả khai thác đặc trưng, giúp giảm độ phức tạp mô hình mà vẫn bảo toàn khả năng phát hiện.

So sánh với YOLOv8m

Mô hình cải tiến đạt Precision = 0.906, Recall = 0.846, mAP50 = 0.915 và mAP50-95 = 0.654. So với YOLOv8m (0.927, 0.842, 0.918, 0.674), mức chênh lệch rất nhỏ: Precision giảm 0.021, Recall tăng nhẹ 0.004, mAP50 giảm 0.003 và mAP50-95 giảm 0.020.

Xét theo từng lớp:

- Car: mAP50 = 0.975 (cao hơn YOLOv8m)
- Van: mAP50 = 0.946 (cao hơn)
- Truck: mAP50 = 0.952 (cao hơn)
- Person và Cyclist: duy trì hiệu suất ổn định, đặc biệt là khả năng phát hiện đối tượng nhỏ.

So sánh với YOLO Gold

Mô hình cải tiến có hiệu suất rất gần với YOLO Gold. mAP50 chỉ giảm 0.005 (0.915 so với 0.920) và mAP50-95 giảm 0.020 (0.654 so với 0.674). Một số lớp như Car (0.975) và Van (0.946) thậm chí còn nhỉnh hơn YOLO Gold.

Về tốc độ suy luận

Trên GPU RTX 3050 với ảnh đầu vào 640×640:

- Mô hình cải tiến: 9.7 ms/ảnh
- YOLOv8m: 7.5 ms/ảnh
- YOLO Gold: 9.1 ms/ảnh

Mặc dù chậm hơn một chút do bổ sung nhánh P2 và các module dung hợp đặc trưng, tốc độ 9.7 ms vẫn đảm bảo khả năng xử lý gần thời gian thực (>100 FPS).

Nhìn chung, kết quả thực nghiệm khẳng định GPFS_YOLO đã đạt được mục tiêu tối ưu hóa kiến trúc. Việc giảm gần một nửa số lượng tham số trong khi chỉ chấp nhận suy giảm rất nhỏ về độ chính xác cho thấy sự cân bằng hợp lý giữa hiệu suất và hiệu quả tính toán.

Mô hình không chỉ duy trì tốt khả năng phát hiện mà còn cải thiện ở một số lớp quan trọng (Car, Van), chứng tỏ các cải tiến (Fast-C2f, SimAM, nhánh P2, Mish, WIoU) đã phát huy hiệu quả rõ rệt.

Bảng 13: Kết quả đánh giá mô hình cải tiến thiết bị nhúng Raspberry Pi4

Class	Precision	Recall	mAP50	mAP50-95
all	0.938	0.871	0.932	0.693
Person	0.969	0.648	0.817	0.469
Car	0.939	0.912	0.96	0.786
Cyclist	0.835	0.828	0.903	0.549
Van	0.948	0.966	0.986	0.804
Truck	0.996	1	0.995	0.857

Inference time: Trên Raspberry Pi 4(CPU ARM Cortex-A72, không sử dụng GPU/NPU tăng tốc): 4456ms

Kết quả đánh giá sơ bộ của mô hình trên thiết bị nhúng Raspberry Pi 4 sau khi được chuyển đổi sang định dạng ONNX. Nhằm mục đích kiểm tra khả năng hoạt động trên môi trường hạn chế tài nguyên, tập dữ liệu đánh giá được thu gọn ngẫu nhiên xuống còn 100 ảnh. Dữ liệu thực nghiệm cho thấy các chỉ số đánh giá tổng thể có sự gia tăng nhẹ đạt Precision = 0,938, Recall = 0,871, mAP@0.5 = 0,932 và mAP@0.5:0.95 = 0,693. Nổi bật nhất là lớp Truck với kết quả gần như tuyệt đối (Recall = 1, mAP@0.5 = 0,995). Ngược lại, lớp Person vẫn ghi nhận mức Recall thấp nhất (0,648), một lần nữa khẳng định giới hạn của mô hình trong việc nhận diện các đối tượng có kích thước nhỏ, ở khoảng cách xa hoặc bị che khuất.

Cần lưu ý rằng, sự gia tăng của các chỉ số đánh giá trên thiết bị nhúng không đồng nghĩa với việc mô hình hoạt động tốt hơn so với trên PC. Nguyên nhân chính của sự chênh lệch này đến từ việc kích thước tập mẫu đánh giá đã được thu hẹp đáng kể (100 ảnh), dẫn đến phân phối hình ảnh có thể chứa ít các tình huống nhiễu hoặc phức tạp hơn so với toàn bộ tập kiểm thử. Do đó, kết quả mang ý nghĩa quan trọng trong việc xác nhận tính toàn vẹn của mô hình sau khi chuyển đổi định dạng và độ ổn định khi trích xuất đặc trưng trên môi trường mới, thay vì dùng để so sánh trực tiếp hiệu năng nhận diện.

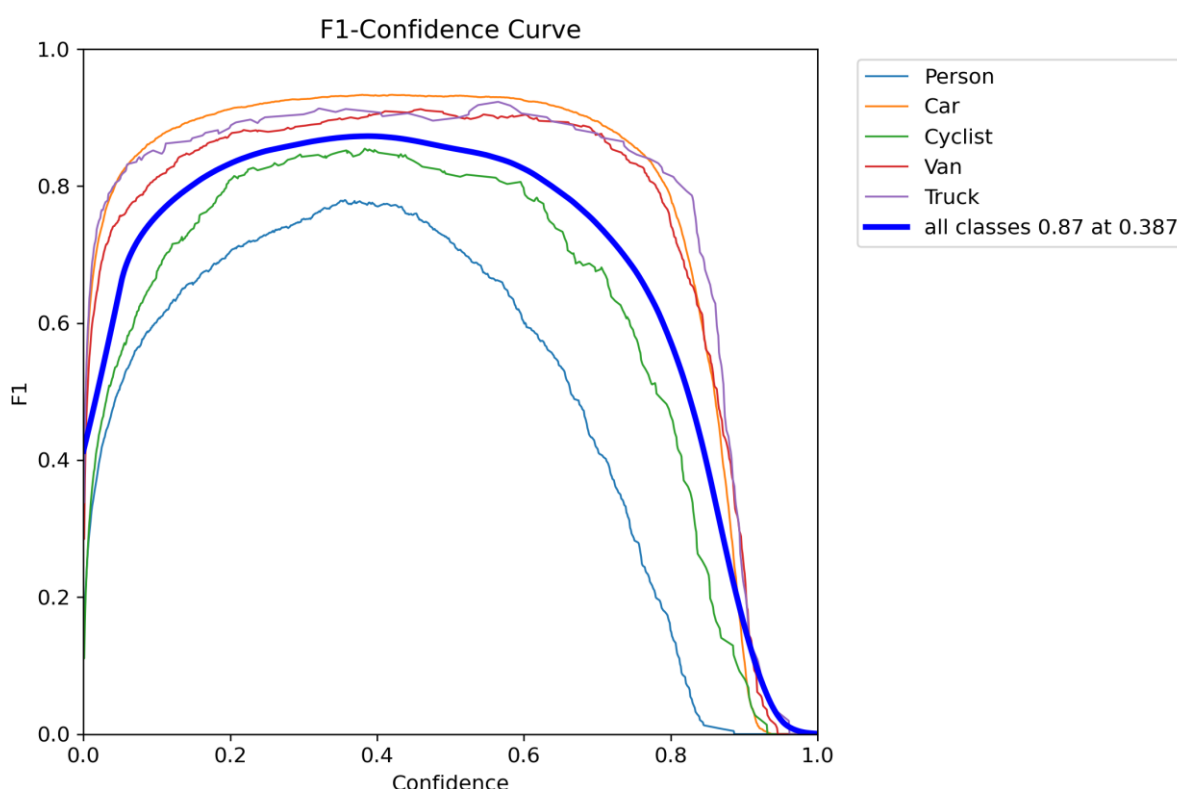
Xét về thời gian suy luận (Inference time), việc chuyển đổi môi trường triển khai đã cho thấy sự chênh lệch lớn về tốc độ xử lý. Thời gian để mô hình xử lý một bức ảnh tăng mạnh từ 10,6 ms (trên PC trang bị GPU) lên tới 4456 ms (khoảng 4,46 giây, tương đương 0,22 FPS) trên Raspberry Pi 4 vốn chỉ sử dụng CPU ARM Cortex-A72. Mặc dù tốc độ

xử lý này chưa thể đáp ứng các yêu cầu giám sát luồng giao thông di chuyển nhanh theo thời gian thực (real-time tracking), nhưng việc mô hình có thể tải và vận hành trơn tru mà không gây tràn bộ nhớ trên một thiết bị biên (Edge Device) đã minh chứng cho tính khả thi của hệ thống. Với hiệu năng hiện tại, hệ thống hoàn toàn có thể được ứng dụng vào các bài toán giám sát tĩnh, ít biến động về thời gian như phân tích bãi đỗ xe định kỳ hoặc camera giám sát tại trạm thu phí.

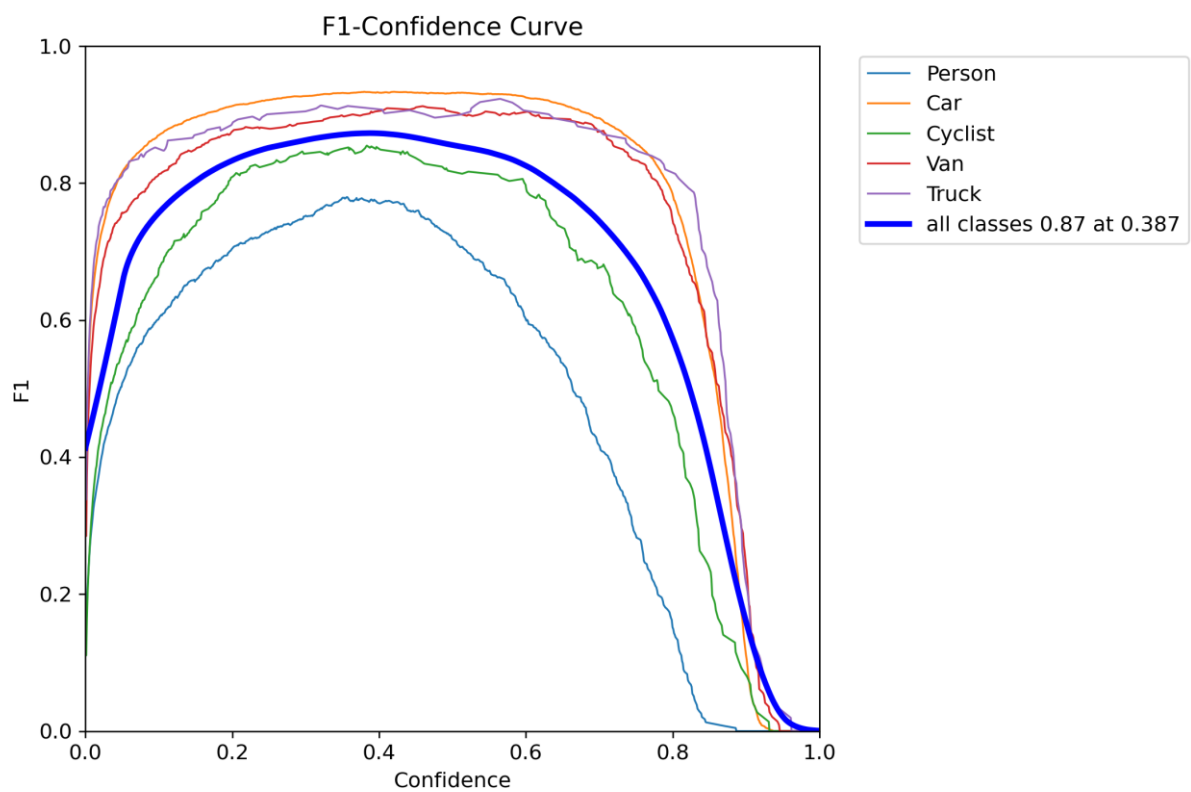
Nhìn chung, kết quả từ cả hai bảng đánh giá cho thấy mô hình mạng được đề xuất có khả năng học và trích xuất đặc trưng tốt, đạt độ chính xác cao đối với các lớp phương tiện giao thông chính yếu. Đồng thời, việc triển khai thành công trên Raspberry Pi đã đáp ứng được mục tiêu cơ bản của đề tài, tạo tiền đề vững chắc để áp dụng thêm các kỹ thuật tối ưu hóa mô hình nâng cao như lượng tử hóa (Quantization) hoặc sử dụng các bộ tăng tốc phần cứng nhằm cải thiện tốc độ suy luận trong tương lai.

4.3.2. Phân tích các đường cong đánh giá hiệu năng mô hình

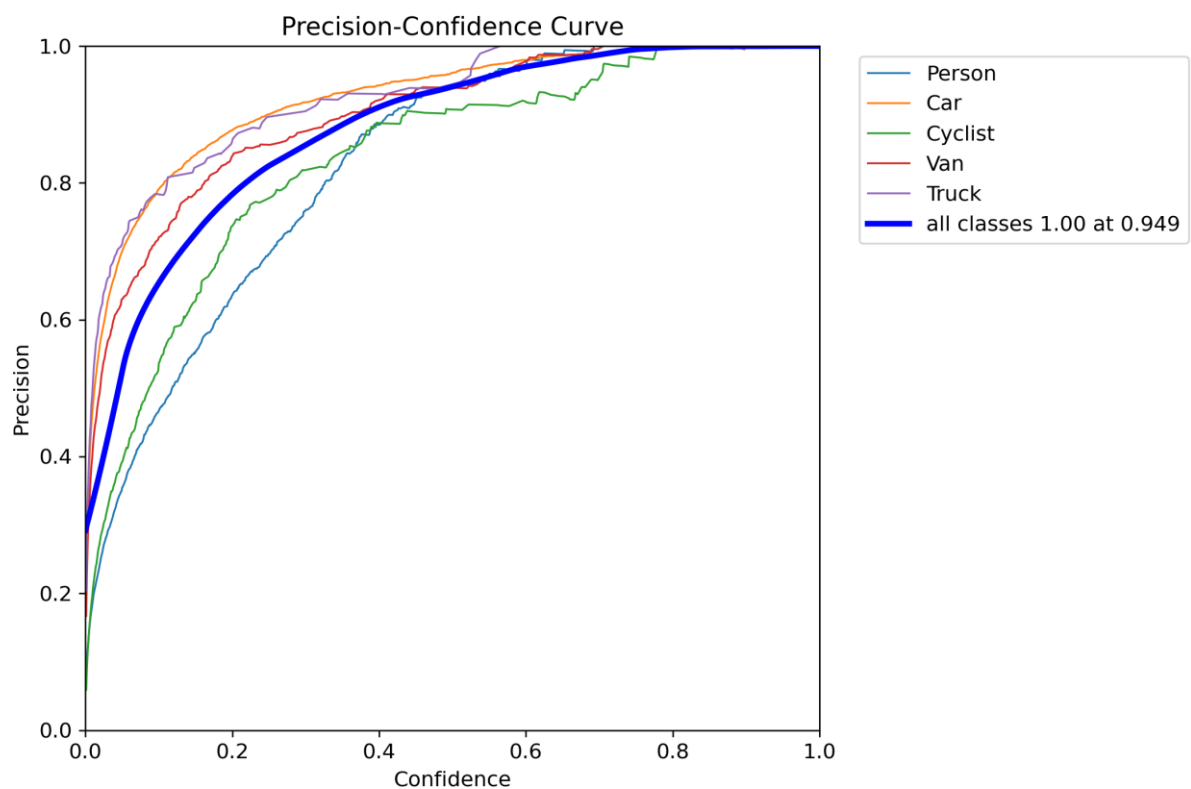
Các đường cong giá trị khi huấn luyện của mô hình cho ta thấy được xu hướng tính chất của mỗi thông số hiệu năng của mô hình.



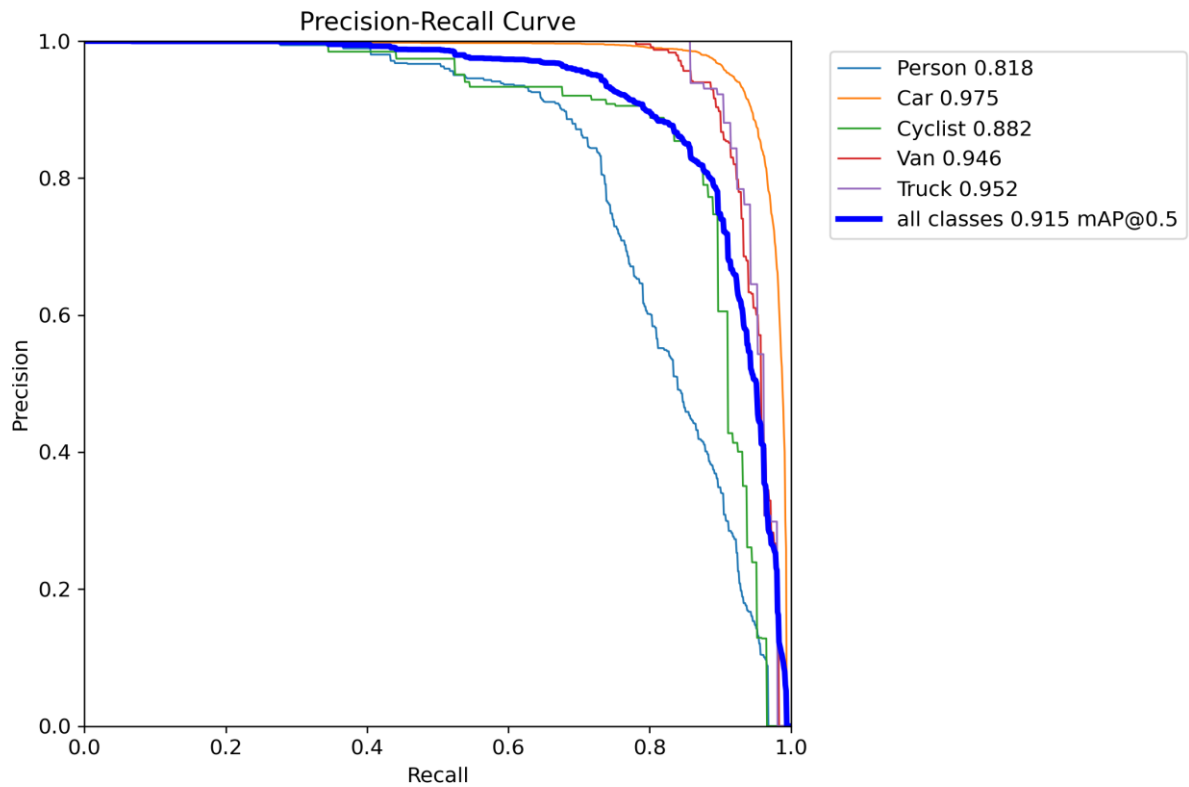
Hình 13: Biểu đồ Điểm F1 – Độ tin cậy



Hình 14: Biểu đồ Độ chuẩn xác – Độ tin cậy



Hình 15: Biểu đồ Độ chuẩn xác – Độ thu hồi



Hình 16: Biểu đồ Độ thu hồi – Độ tin cậy

Bốn đường cong đánh giá gồm Recall–Confidence, Precision–Recall, Precision–Confidence và F1–Confidence phản ánh toàn diện hiệu quả của mô hình GPFS_YOLO trên năm lớp đối tượng gồm Person, Car, Cyclist, Van và Truck. Kết quả cho thấy mô hình duy trì khả năng phát hiện ổn định đối với các lớp phương tiện giao thông, đồng thời đạt sự cân bằng tốt giữa độ chính xác và khả năng phát hiện trong khoảng ngưỡng Confidence trung bình. Các đường cong có xu hướng thay đổi mượt, không xuất hiện dao động lớn, chứng tỏ mô hình có tính ổn định và khả năng tổng quát hóa tương đối tốt trên tập dữ liệu thử nghiệm.

Đường cong Recall–Confidence cho thấy Recall giảm dần khi ngưỡng Confidence tăng, phù hợp với nguyên lý hoạt động của các mô hình phát hiện đối tượng. Đường cong tổng (màu xanh đậm) đạt Recall khoảng 0,96 tại Confidence bằng 0, sau đó giảm từ từ và còn khoảng 0,80 khi Confidence xấp xỉ 0,50 trước khi giảm mạnh sau ngưỡng 0,80. Trong các lớp đối tượng, Car là lớp có Recall cao nhất, duy trì trên 0,90 trong gần như toàn bộ khoảng Confidence từ 0 đến khoảng 0,65, cho thấy khả năng phát hiện rất ổn định. Hai lớp Truck và Van cũng đạt Recall cao, chỉ giảm nhanh khi Confidence vượt khoảng 0,75–0,80. Trong khi đó, lớp Cyclist suy giảm nhanh hơn từ khoảng Confidence 0,45, còn lớp Person có Recall thấp nhất khi giảm xuống dưới 0,80 ngay tại

Confidence khoảng 0,20 và dưới 0,50 khi Confidence xấp xỉ 0,58. Điều này cho thấy mô hình vẫn gặp khó khăn khi phát hiện người, đặc biệt trong các trường hợp đối tượng nhỏ, bị che khuất hoặc xuất hiện ở khoảng cách xa.

Đường cong Precision–Confidence phản ánh mối quan hệ giữa độ chính xác dự đoán và ngưỡng Confidence. Kết quả cho thấy Precision tăng liên tục khi Confidence tăng, trong đó đường cong tổng đạt Precision = 1,00 tại Confidence $\approx$ 0,949. Điều này chứng tỏ khi lựa chọn ngưỡng Confidence rất cao, hầu như toàn bộ các dự đoán còn lại đều chính xác. Lớp Car tiếp tục đạt kết quả tốt nhất khi Precision vượt 0,90 ngay từ Confidence khoảng 0,25 và duy trì gần 1,00 trong phần lớn miền Confidence còn lại. Hai lớp Truck và Van cũng duy trì Precision trên 0,90 từ khoảng Confidence 0,35–0,40. Đối với lớp Cyclist, Precision tăng chậm hơn và chỉ vượt 0,90 khi Confidence đạt khoảng 0,55, trong khi lớp Person có Precision thấp nhất ở các ngưỡng Confidence nhỏ nhưng tăng đều và vượt 0,90 khi Confidence lớn hơn khoảng 0,45. Xu hướng này cho thấy việc tăng Confidence giúp giảm đáng kể số lượng dự đoán sai, tuy nhiên đồng thời cũng làm giảm Recall như thể hiện ở đồ thị trước.

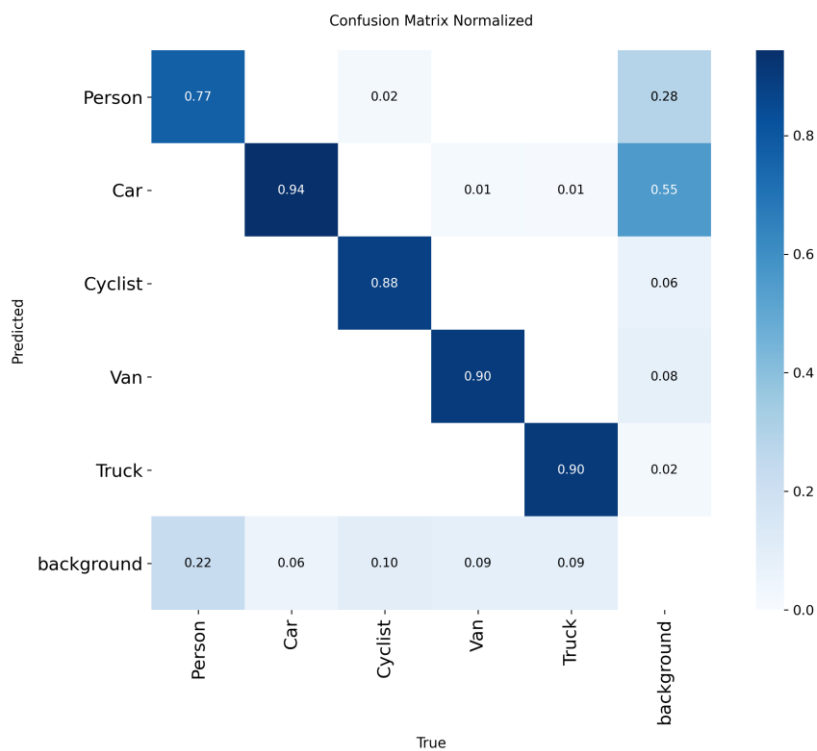
Đường cong F1–Confidence phản ánh khả năng cân bằng giữa Precision và Recall. Đường cong tổng đạt F1-score cực đại bằng 0,87 tại Confidence $\approx$ 0,387, cho thấy đây là ngưỡng Confidence phù hợp nhất để đạt hiệu năng tổng thể tối ưu. Trong các lớp đối tượng, Car đạt F1 cao nhất với giá trị cực đại khoảng 0,93, tiếp theo là Truck khoảng 0,92, Van khoảng 0,91, Cyclist khoảng 0,85, trong khi Person chỉ đạt khoảng 0,78. Hầu hết các đường cong đều duy trì ổn định trong khoảng Confidence từ 0,20 đến 0,70, sau đó giảm nhanh khi Confidence vượt khoảng 0,85. Nguyên nhân là mặc dù Precision vẫn tiếp tục tăng nhưng Recall giảm mạnh, khiến F1-score suy giảm nhanh ở các ngưỡng Confidence cao. Kết quả này cho thấy việc lựa chọn Confidence khoảng 0,38–0,40 sẽ giúp mô hình đạt được sự cân bằng tốt nhất giữa khả năng phát hiện và độ chính xác.

Đường cong Precision–Recall (PR Curve) phản ánh trực tiếp hiệu năng phát hiện của từng lớp thông qua chỉ số Average Precision (AP). Kết quả cho thấy mô hình đạt $mAP@0.5 = 0,915$, thể hiện khả năng phát hiện đối tượng ở mức cao. Trong đó, lớp Car đạt $AP = 0,975$, là lớp có hiệu năng tốt nhất với đường cong gần sát góc trên bên phải của đồ thị, chứng tỏ mô hình đồng thời duy trì Precision và Recall rất cao. Tiếp theo là Truck với $AP = 0,952$, Van đạt 0,946, cho thấy mô hình nhận dạng rất hiệu quả các phương tiện có kích thước lớn và đặc trưng hình dạng rõ ràng. Đối với lớp Cyclist, mô

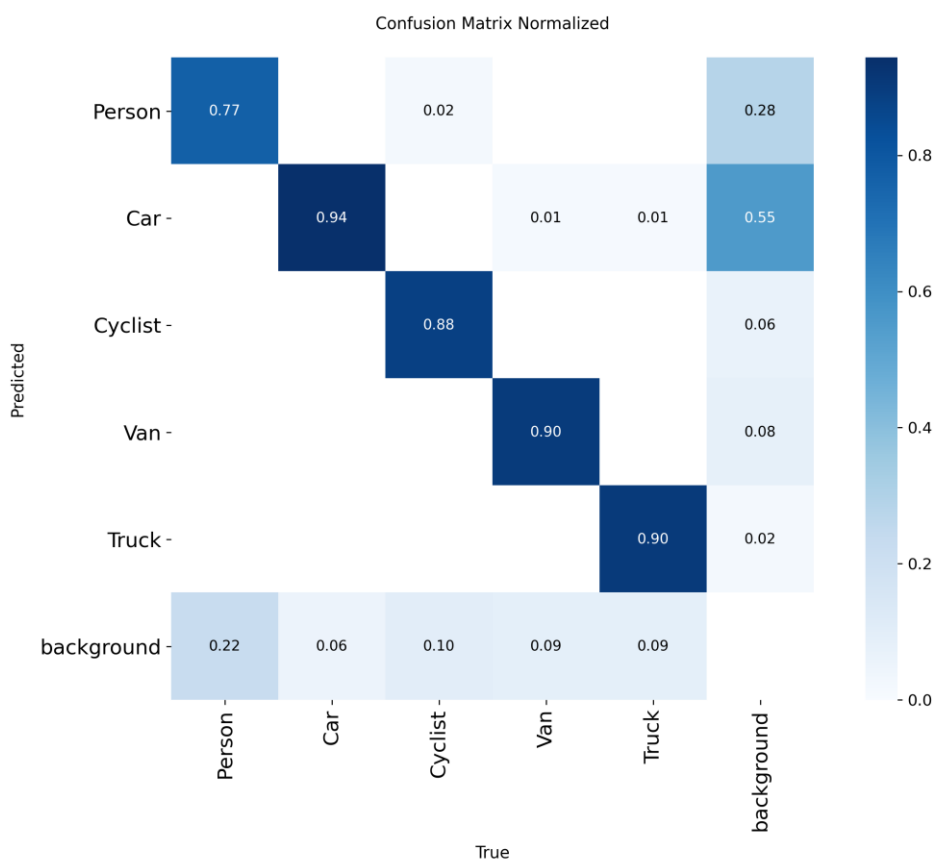
hình đạt $AP = 0,882$, thấp hơn các lớp phương tiện nhưng vẫn duy trì Precision ở mức cao khi Recall chưa vượt khoảng 0,80. Lớp Person đạt $AP = 0,818$, thấp nhất trong năm lớp đối tượng. Đường cong của lớp này giảm nhanh khi Recall vượt khoảng 0,70, cho thấy dễ phát hiện thêm nhiều đối tượng người hơn, mô hình phải chấp nhận nhiều dự đoán sai hơn. Xu hướng này hoàn toàn phù hợp với kết quả quan sát trên các đường cong Recall–Confidence và F1–Confidence.

Tổng hợp bốn đường cong đánh giá cho thấy mô hình GPFS_YOLO đạt hiệu năng tổng thể tốt với $mAP@0.5 = 91,5\%$, Recall cực đại khoảng 96%, Precision đạt 100% tại $Confidence \approx 0,949$ và F1-score cực đại bằng 0,87 tại $Confidence \approx 0,387$. Các lớp Car, Truck và Van đều có khả năng phát hiện rất tốt khi đường cong PR bám sát góc trên bên phải và F1-score luôn duy trì trên 0,90 trong khoảng Confidence rộng. Trong khi đó, Person và Cyclist vẫn là hai lớp có hiệu năng thấp hơn do kích thước đối tượng nhỏ, hình dạng đa dạng và thường xuyên bị che khuất hoặc xuất hiện ở khoảng cách xa. Mặc dù vậy, các đường cong đều thay đổi tương đối mượt và ổn định, không xuất hiện hiện tượng suy giảm đột ngột trong vùng Confidence trung bình, cho thấy mô hình có khả năng tổng quát hóa tốt. Với kết quả này, ngưỡng Confidence khoảng 0,38–0,40 được xem là phù hợp để triển khai thực tế nhằm đạt sự cân bằng tối ưu giữa độ chính xác và khả năng phát hiện đối tượng.

4.3.3. Ma trận nhầm lẫn (Confusion Matrix)



Hình 17:Ma trận nhầm lẫn



Hình 18:Ma trận nhầm lẫn chuẩn hóa

Ma trận nhầm lẫn được sử dụng để đánh giá khả năng phân loại của mô hình GPFS_YOLO đối với từng lớp đối tượng, đồng thời phản ánh các trường hợp dự đoán đúng, nhầm lẫn giữa các lớp cũng như các đối tượng bị bỏ sót hoặc dự đoán sai. Quan sát cả ma trận nhầm lẫn và ma trận chuẩn hóa cho thấy phần lớn các giá trị tập trung trên đường chéo chính, chứng tỏ mô hình có khả năng phân biệt tốt giữa năm lớp đối tượng Person, Car, Cyclist, Van và Truck. Các giá trị trên đường chéo đều đạt từ 77% đến 94%, phản ánh hiệu quả nhận dạng tương đối cao trên hầu hết các lớp.

Đối với lớp Car, mô hình đạt hiệu năng cao nhất với 2.691 mẫu được nhận dạng chính xác, tương ứng 94% tổng số mẫu của lớp theo ma trận chuẩn hóa. Chỉ có 1 trường hợp bị nhầm thành Person, 4 trường hợp bị nhầm thành Van và 1 trường hợp bị nhầm thành Truck. Tuy nhiên vẫn còn 157 mẫu bị bỏ sót và được xếp vào background. Mặc dù số lượng mẫu bị bỏ sót khá lớn về mặt tuyệt đối, điều này chủ yếu xuất phát từ việc lớp Car chiếm số lượng mẫu nhiều nhất trong tập dữ liệu. Xét theo tỷ lệ, lớp Car vẫn là lớp có khả năng nhận dạng tốt nhất trong năm lớp đối tượng.

Lớp Truck cũng đạt hiệu quả nhận dạng cao với 95 mẫu được phân loại đúng, tương ứng khoảng 90% tổng số mẫu. Không xuất hiện trường hợp bị nhầm với các lớp phương tiện khác, trong khi chỉ có 9 mẫu bị bỏ sót và chuyển sang background. Ngoài ra có 10 dự đoán sai thuộc lớp Truck nhưng thực tế là nền, cho thấy mô hình đôi khi vẫn tạo ra một số dự đoán dư thừa ở lớp này. Tuy nhiên, nhìn chung khả năng phân biệt xe tải với các lớp còn lại vẫn rất tốt.

Đối với lớp Van, mô hình nhận dạng chính xác 254 mẫu, tương ứng khoảng 90% tổng số mẫu. Chỉ ghi nhận 3 trường hợp bị nhầm thành Car, trong khi 24 mẫu bị bỏ sót và chuyển sang background. Đồng thời có 41 dự đoán Van trên các vùng thực tế là nền. Kết quả này cho thấy mô hình vẫn duy trì khả năng nhận dạng tốt đối với xe van, mặc dù vẫn tồn tại một số nhầm lẫn nhỏ với ô tô do hai lớp có nhiều đặc điểm hình học tương đồng.

Lớp Cyclist đạt 128 mẫu được nhận dạng đúng, tương ứng khoảng 88% tổng số mẫu. Chỉ có 2 trường hợp bị nhầm thành Person, trong khi 14 mẫu bị bỏ sót và chuyển sang background. Ngoài ra có 34 dự đoán thuộc lớp Cyclist nhưng thực tế là nền. So với các lớp phương tiện, hiệu quả nhận dạng Cyclist thấp hơn do đối tượng thường có kích thước nhỏ, hình dạng thay đổi theo tư thế và dễ bị che khuất trong ảnh.

Lớp Person là lớp có hiệu năng thấp nhất trong năm lớp đối tượng. Mô hình nhận dạng đúng 356 mẫu, tương ứng khoảng 77% tổng số mẫu. Có 3 trường hợp bị nhầm thành Cyclist, trong khi 103 mẫu bị bỏ sót và chuyển sang background. Đồng thời xuất hiện 149 dự đoán Person trên các vùng thực tế không chứa đối tượng. Điều này phản ánh việc phát hiện người vẫn là bài toán khó do đối tượng thường xuất hiện với kích thước nhỏ, khoảng cách xa hoặc bị che khuất, khiến mô hình dễ bỏ sót hoặc sinh thêm dự đoán giả.

Xét các phần tử ngoài đường chéo chính, số lượng nhầm lẫn trực tiếp giữa các lớp đối tượng là tương đối nhỏ. Các trường hợp nhầm lẫn chủ yếu xảy ra giữa Car và Van, Person và Cyclist, đây đều là những cặp lớp có đặc điểm hình dạng hoặc tỷ lệ kích thước tương đối gần nhau trong một số điều kiện quan sát. Đặc biệt, hầu như không xuất hiện hiện tượng nhầm lẫn giữa Truck với các lớp còn lại, cho thấy mô hình đã học được đặc trưng phân biệt khá rõ đối với lớp này.

Ngoài ra, cột background trong ma trận nhầm lẫn phản ánh các false positive, tức các dự đoán của mô hình không tương ứng với bất kỳ đối tượng thực nào, trong khi hàng background biểu thị các false negative, tức các đối tượng thực bị mô hình bỏ sót. Có thể thấy số lượng dự đoán sai vào nền tập trung chủ yếu ở lớp Car (291 mẫu) và Person (149 mẫu), tiếp theo là Van (41 mẫu), Cyclist (34 mẫu) và Truck (10 mẫu). Ngược lại, số lượng đối tượng bị bỏ sót nhiều nhất cũng thuộc về Car (157 mẫu) và Person (103 mẫu). Tuy nhiên, xét theo ma trận chuẩn hóa, tỷ lệ nhận dạng đúng của lớp Car vẫn đạt 94%, cao nhất trong năm lớp đối tượng, cho thấy số lượng tuyệt đối lớn chủ yếu là do phân bố dữ liệu không đồng đều.

Nhìn chung, ma trận nhầm lẫn cho thấy mô hình đạt khả năng phân loại tốt với các giá trị trên đường chéo chính đều lớn hơn 77%, trong đó Car (94%), Van (90%), Truck (90%) và Cyclist (88%) đều đạt hiệu năng cao. Lớp Person (77%) vẫn là lớp khó nhận dạng nhất do thường xuyên xuất hiện với kích thước nhỏ hoặc bị che khuất. Kết quả này phù hợp với các đường cong Recall–Confidence, F1–Confidence và Precision–Recall đã phân tích trước đó, khi lớp Person có Recall và AP thấp nhất, trong khi Car luôn đạt hiệu năng cao nhất. Điều này cho thấy mô hình có khả năng tổng quát hóa tốt trên các lớp phương tiện, đồng thời hướng cải thiện trong tương lai nên tập trung vào việc nâng cao khả năng phát hiện đối tượng Person và Cyclist thông qua tăng cường dữ liệu, cải thiện chiến lược gán mẫu hoặc tối ưu kiến trúc phát hiện đối tượng nhỏ.

4.3.4. Đánh giá kết quả phát hiện trên tập kiểm thử



Hình 19: Kết quả phát hiện đối tượng trên tập kiểm thử

Kết quả suy luận của mô hình GPFS_YOLO trên 16 ảnh thuộc tập kiểm tra, phản ánh khả năng phát hiện đồng thời năm lớp đối tượng gồm Person, Car, Cyclist, Van và Truck trong nhiều điều kiện giao thông khác nhau. Các ảnh bao phủ nhiều bối cảnh như đường đô thị, đường ngoại ô, giao lộ và khu dân cư, với sự thay đổi về khoảng cách quan sát, mật độ phương tiện và điều kiện chiếu sáng. Có thể nhận thấy mô hình phát hiện chính xác phần lớn các đối tượng với khung giới hạn bám sát biên vật thể và rất ít trường hợp nhận dạng sai lớp.

Quan sát trực tiếp trên 16 ảnh cho thấy mô hình phát hiện khoảng 40–45 đối tượng, trong đó Car chiếm số lượng lớn nhất với khoảng 30–32 xe, tương ứng hơn 70% tổng số đối tượng được phát hiện. Ngoài ra, mô hình còn nhận dạng được khoảng 4 xe Van, 2 xe Truck, 1 Person và 1 Cyclist. Sự phân bố này phù hợp với đặc điểm của tập dữ liệu, khi lớp Car có số lượng mẫu huấn luyện và kiểm tra lớn hơn đáng kể so với các lớp còn lại.

Đối với lớp Car, mô hình cho kết quả rất ổn định. Nhiều ảnh xuất hiện đồng thời từ 3 đến 6 xe ô tô và tất cả đều được nhận dạng chính xác. Đặc biệt, ở các ảnh giao lộ và khu dân cư đông phương tiện, mô hình vẫn phát hiện được các xe ở nhiều khoảng cách khác nhau, từ các xe ở gần camera đến các xe chỉ chiếm một vùng rất nhỏ trên ảnh. Các khung giới hạn bao phủ sát đối tượng và gần như không xuất hiện hiện tượng nhầm lẫn sang các lớp khác. Kết quả trực quan này hoàn toàn phù hợp với kết quả đánh giá định lượng khi lớp Car đạt $AP = 0,975$, $Recall = 0,925$ và $Precision = 0,940$, là lớp có hiệu năng cao nhất của mô hình.

Hai lớp Van và Truck cũng được phát hiện chính xác trong hầu hết các trường hợp. Mô hình nhận dạng được các xe có kích thước lớn ngay cả khi xuất hiện gần mép ảnh hoặc bị che khuất một phần. Trong toàn bộ các ảnh minh họa, không quan sát thấy hiện tượng nhầm lẫn trực tiếp giữa Truck và Car, trong khi Van chỉ xuất hiện rất ít trường hợp gần các xe ô tô nhưng vẫn được phân biệt đúng. Điều này phù hợp với kết quả của ma trận nhầm lẫn, trong đó hai lớp này đều đạt tỷ lệ nhận dạng đúng khoảng 90%, đồng thời có AP đạt 0,952 đối với Truck và 0,946 đối với Van.

Đối với hai lớp Person và Cyclist, mô hình vẫn nhận dạng chính xác mặc dù đối tượng có kích thước nhỏ hơn đáng kể so với các phương tiện. Trong ảnh có người đi bộ và người đi xe đạp, cả hai đối tượng đều được phát hiện đúng với vị trí khung giới hạn tương đối chính xác. Tuy nhiên, có thể nhận thấy kích thước khung bao của hai lớp này nhỏ hơn nhiều và xuất hiện ở khoảng cách xa hơn, khiến việc phát hiện trở nên khó khăn hơn. Điều này cũng phù hợp với kết quả đánh giá định lượng khi Person chỉ đạt $AP = 0,818$, $Recall = 0,695$, trong khi Cyclist đạt $AP = 0,882$ và $Recall = 0,829$, thấp hơn đáng kể so với các lớp phương tiện.

Bên cạnh những kết quả tích cực, vẫn có thể quan sát thấy một số hạn chế. Ở một số ảnh có nhiều xe ô tô đứng gần nhau, mô hình xuất hiện 2–3 khung giới hạn chồng lấn trên cùng một phương tiện. Đây là hiện tượng duplicate detection, cho thấy quá trình Non-Maximum Suppression (NMS) chưa loại bỏ hoàn toàn các hộp giới hạn có độ chồng lấn lớn trong các cảnh giao thông đông đúc. Tuy nhiên, các khung dự đoán chính vẫn bao phủ đúng đối tượng nên hiện tượng này chỉ ảnh hưởng nhỏ đến chất lượng phát hiện tổng thể.

Nhìn chung, kết quả suy luận trực quan cho thấy mô hình GPFS_YOLO có khả năng phát hiện đồng thời nhiều đối tượng trong các điều kiện giao thông đa dạng, từ đường vắng đến các khu vực có mật độ phương tiện cao. Mô hình thể hiện hiệu quả đặc biệt tốt đối với các lớp Car, Van và Truck, trong khi Person và Cyclist vẫn còn gặp khó khăn khi đối tượng ở khoảng cách xa hoặc có kích thước nhỏ. Những quan sát này hoàn toàn nhất quán với kết quả đánh giá định lượng đã trình bày trước đó, khi mô hình đạt $Precision = 0,906$, $Recall = 0,846$, $mAP@0.5 = 0,915$ và $mAP@0.5:0.95 = 0,654$, cho thấy mô hình có khả năng tổng quát hóa tốt và đáp ứng yêu cầu phát hiện đối tượng trong các bài toán giao thông thực tế.

4.3.5. So sánh mô hình đề xuất với mô hình cơ sở (Baseline)

Bảng 14: So sánh mô hình cải tiến với mô hình cơ sở (baseline). ML: Máy huấn luyện, Pi4: Raspberry Pi4

Model	Parameters	GFlops	mAP@0.5 (Person)	mAP@0.5	mAP@0.5-0.95	Inference Time
Yolov8m	25,842,655	78.7	0.828	0.918	0.674	ML:7.5 ms Pi4:3429ms
Yolov8_Gold	24,153,471	76.4	0.81	0.920	0.674	ML:9.1 ms Pi4:3100ms
GPFS_Yolo	13,895,263	59.1	0.818	0.915	0.654	ML:9.7 ms Pi4:4456ms

Bảng 14 trình bày kết quả so sánh giữa mô hình đề xuất GPFS_Yolo với hai mô hình cơ sở là YOLOv8m và YOLOv8_Gold trên các tiêu chí gồm số lượng tham số, độ phức tạp tính toán (GFLOPs), hiệu năng phát hiện và thời gian suy luận trên máy huấn luyện (ML) cũng như thiết bị nhúng Raspberry Pi 4 (Pi4). Kết quả cho thấy mô hình đề xuất đã giảm đáng kể độ phức tạp tính toán trong khi vẫn duy trì hiệu năng phát hiện ở mức cao, đáp ứng mục tiêu tối ưu cho các hệ thống nhúng.

Về độ phức tạp mô hình, GPFS_Yolo chỉ còn 13.895.263 tham số, giảm 11.947.392 tham số, tương đương 46,2% so với YOLOv8m và giảm 10.258.208 tham số, tương đương 42,5% so với YOLOv8_Gold. Đồng thời, số phép tính cũng giảm từ 78,7 GFLOPs xuống còn 59,1 GFLOPs, tương ứng giảm 24,9% so với YOLOv8m và giảm 22,6% so với YOLOv8_Gold. Việc giảm mạnh cả số lượng tham số và GFLOPs cho thấy kiến trúc đề xuất có khả năng giảm đáng kể chi phí tính toán và yêu cầu bộ nhớ, phù hợp với mục tiêu triển khai trên các thiết bị có tài nguyên hạn chế.

Về hiệu năng phát hiện, GPFS_Yolo đạt $mAP@0.5 = 0,915$ và $mAP@0.5:0.95 = 0,654$, chỉ giảm lần lượt 0,003 và 0,020 so với YOLOv8m. So với YOLOv8_Gold, mức giảm tương ứng là 0,005 đối với $mAP@0.5$ và 0,020 đối với $mAP@0.5:0.95$. Đối với lớp Person, mô hình đạt $AP = 0,818$, thấp hơn 0,010 so với YOLOv8m và 0,015 so với YOLOv8_Gold. Mặc dù độ chính xác giảm nhẹ, mức suy giảm này là tương đối nhỏ nếu xét đến việc mô hình đã cắt giảm gần một nửa số lượng tham số và khoảng một phần tư khối lượng tính toán.

Về thời gian suy luận trên máy huấn luyện, GPFS_Yolo đạt 9,7 ms/ảnh, chậm hơn 2,2 ms so với YOLOv8m và 0,6 ms so với YOLOv8_Gold. Sự gia tăng nhỏ này chủ yếu

xuất phát từ việc mô hình bổ sung các thành phần như P2, FastC2f, SimAM và Mish, làm tăng số lượng phép toán tuần tự mặc dù tổng độ phức tạp tính toán đã được giảm. Tuy nhiên, mức chênh lệch dưới 3 ms là không đáng kể đối với các hệ thống sử dụng GPU trong quá trình suy luận.

Đối với Raspberry Pi 4, thời gian suy luận của GPFS_Yolo đạt 4456 ms/ảnh, cao hơn 1027 ms so với YOLOv8m và 1356 ms so với YOLOv8_Gold. Mặc dù mô hình có số lượng tham số và GFLOPs thấp hơn đáng kể, tốc độ suy luận trên Raspberry Pi lại chưa được cải thiện. Điều này cho thấy hiệu năng trên CPU ARM không chỉ phụ thuộc vào số lượng tham số hay GFLOPs mà còn chịu ảnh hưởng bởi cấu trúc của mô hình, mức độ tối ưu của các phép toán, khả năng hỗ trợ của thư viện suy luận và việc tận dụng tài nguyên phần cứng. Các thành phần mới như FastC2f, SimAM hoặc các phép toán trong GoldYOLO có thể chưa được tối ưu hoàn toàn trên kiến trúc ARM của Raspberry Pi, dẫn đến thời gian suy luận thực tế tăng lên mặc dù chi phí tính toán lý thuyết đã giảm.

Nhìn chung, kết quả so sánh cho thấy GPFS_Yolo đã đạt được mục tiêu thiết kế là xây dựng một mô hình nhẹ hơn đáng kể với 46,2% số lượng tham số và 24,9% GFLOPs được cắt giảm so với YOLOv8m, trong khi vẫn duy trì mAP@0.5 đạt 91,5% và mAP@0.5:0.95 đạt 65,4%. Mặc dù thời gian suy luận trên Raspberry Pi 4 chưa được cải thiện, mô hình vẫn thể hiện sự đánh đổi hợp lý giữa độ chính xác và độ phức tạp tính toán

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Tóm tắt đóng góp của đề tài

Sau quá trình nghiên cứu, thiết kế kiến trúc và triển khai thực nghiệm, đề tài “Cải tiến mô hình Gold-YOLO cho hệ thống nhận diện đối tượng” đã hoàn thành các mục tiêu đề ra. Những đóng góp chính của đề tài được tóm tắt như sau:

- Đề xuất kiến trúc GPFS_YOLO tối ưu cho bài toán phát hiện đối tượng: Đề tài đã xây dựng mô hình GPFS_YOLO trên cơ sở cải tiến kiến trúc Gold-YOLO bằng cách kết hợp các thành phần gồm P2 Detection Head, Fast-C2f, SimAM, Mish và WIoU Loss. Kiến trúc đề xuất hướng đến mục tiêu giảm độ phức tạp tính toán, tối ưu số lượng tham số và nâng cao khả năng trích xuất đặc trưng, đặc biệt đối với các đối tượng có kích thước nhỏ trong môi trường giao thông.

- Giảm đáng kể độ phức tạp của mô hình: Kết quả thực nghiệm cho thấy GPFS_YOLO chỉ còn 13.895.263 tham số, giảm khoảng 46,2% so với YOLOv8m (25.842.655 tham số). Đồng thời, độ phức tạp tính toán giảm từ 78,7 GFLOPs xuống 59,1 GFLOPs, tương ứng giảm khoảng 24,9%. Việc giảm số lượng tham số và GFLOPs giúp mô hình có kích thước nhỏ hơn, yêu cầu bộ nhớ thấp hơn và giảm chi phí tính toán, tạo tiền đề thuận lợi cho việc triển khai trên các thiết bị nhúng và hệ thống biên.
- Duy trì hiệu năng phát hiện trong khi giảm mạnh tài nguyên tính toán: Mặc dù giảm gần một nửa số lượng tham số, GPFS_YOLO vẫn đạt $mAP@0.5 = 0.915$ và $mAP@0.5:0.95 = 0.654$, chỉ giảm nhẹ so với các mô hình cơ sở. Đối với lớp Person, mô hình đạt $AP@0.5 = 0.818$, trong khi lớp Car đạt $AP@0.5 = 0.975$, cho thấy mô hình vẫn duy trì khả năng phát hiện hiệu quả trên các lớp đối tượng chính của bài toán giao thông. Kết quả này chứng minh kiến trúc đề xuất đạt được sự cân bằng hợp lý giữa độ chính xác và độ phức tạp tính toán.
- Nâng cao khả năng phát hiện đối tượng kích thước nhỏ: Việc bổ sung tầng phát hiện P2 giúp mô hình khai thác hiệu quả hơn các đặc trưng có độ phân giải cao, từ đó cải thiện khả năng nhận dạng các đối tượng nhỏ hoặc ở khoảng cách xa như Person và Cyclist. Các kết quả đánh giá thông qua đường cong Precision–Recall, Confusion Matrix và kết quả suy luận trực quan cho thấy mô hình có khả năng phát hiện đồng thời nhiều đối tượng trong các điều kiện giao thông khác nhau với độ ổn định cao.
- Đánh giá toàn diện trên nhiều tiêu chí: Đề tài đã tiến hành đánh giá mô hình thông qua các chỉ số Precision, Recall, $mAP@0.5$, $mAP@0.5:0.95$, các đường cong Precision–Recall, Precision–Confidence, Recall–Confidence, F1–Confidence, ma trận nhầm lẫn và thời gian suy luận trên cả máy tính huấn luyện và Raspberry Pi 4. Các kết quả thực nghiệm cho thấy GPFS_YOLO là một giải pháp phù hợp cho các ứng dụng phát hiện đối tượng trên thiết bị có tài nguyên tính toán hạn chế.

5.2. Hạn chế của đề tài

Mặc dù kiến trúc GPFS_YOLO đã đạt được những kết quả ấn tượng trong việc tối ưu hóa dung lượng lưu trữ (Parameters) và khối lượng tính toán lý thuyết (GFLOPs), tốc độ thực thi thực tế trên cấu hình thuần CPU của bo mạch nhúng Raspberry Pi 4 vẫn chưa đạt được kỳ vọng tối ưu (độ trễ ghi nhận là 4456 ms/ảnh).

Hệ quả này xuất phát từ bản chất kiến trúc tích hợp sâu của mạng Head dạng Gold-YOLO. Quá trình luân chuyển và trộn thông tin đa quy mô phức tạp tại các module liên kết (SimFusion, IFM), kết hợp với toán tử chú ý ba chiều phi tuyến tính SimAM, đòi hỏi tần suất truy cập bộ nhớ và năng lực tính toán song song rất lớn. Khi triển khai trên các lõi CPU ARM thuần túy — vốn dựa trên cơ chế xử lý tuần tự và chưa được tối ưu hóa cho các cấu trúc ma trận đặc thù này — hệ thống dễ rơi vào trạng thái nghẽn cổ chai cục bộ, dẫn đến độ trễ suy luận bị đẩy lên cao.

5.3. Hướng phát triển tiếp theo

Từ những kết quả đạt được cùng các hạn chế khách quan còn tồn tại, đề tài vạch ra các định hướng nghiên cứu và phát triển trọng tâm trong tương lai như sau:

- Tối ưu hóa mã nguồn và cấu hình công cụ biên dịch tăng tốc phần cứng: Để giải quyết triệt để bài toán độ trễ suy luận trên thiết bị biên (Edge Devices), hướng đi cấp thiết tiếp theo là chuyển đổi và đóng gói mô hình GPFS_YOLO sang các định dạng trung gian tối ưu như ONNX, OpenVINO hoặc TensorRT. Việc tận dụng tối đa các tập lệnh tối ưu hóa ma trận song song của các bộ biên dịch này sẽ giúp tăng tốc các phép toán phi tuyến tính của khối SimAM và cấu trúc Head phức tạp. Đồng thời, nghiên cứu triển khai mô hình trên các phần cứng tích hợp chip tăng tốc độ họa hoặc nhân NPU/TPU chuyên dụng (như NVIDIA Jetson Series, Google Coral Edge TPU hoặc Hailo-8) thay vì chạy thuần CPU.
- Nghiên cứu áp dụng các kỹ thuật nén mô hình chuyên sâu (Model Compression): Tiến hành thử nghiệm các phương pháp lượng tử hóa mô hình (Model Quantization) từ cấu hình dấu phẩy động 32-bit (FP32) xuống các định dạng nhẹ hơn như FP16 hoặc INT8 (Post-Training Quantization - PTQ hoặc Quantization-Aware Training - QAT). Bên cạnh đó, việc kết hợp với kỹ thuật cắt tỉa mạng (Pruning) để loại bỏ các kênh tri thức không quan trọng trong mạng Head sẽ giúp mô hình GPFS_YOLO vốn đã nhẹ lại càng tối ưu hơn nữa về mặt tốc độ thực thi, hướng tới mục tiêu đạt mức FPS thời gian thực trên các bo mạch CPU nhúng giá thành thấp.

- Mở rộng miền dữ liệu và kiểm thử kịch bản giao thông thực tế tại Việt Nam: Để nâng cao độ bền bỉ và khả năng tổng quát hóa của mạng nơ-ron, nhóm nghiên cứu định hướng sẽ tái huấn luyện mô hình trên các tập dữ liệu giao thông quy mô lớn và đa dạng hơn (như BDD100K hoặc Waymo). Đặc biệt, đề tài sẽ hướng tới việc tự thu thập, gán nhãn dữ liệu video từ các camera giao thông thực tế tại Việt Nam để thử nghiệm mô hình dưới các điều kiện vận hành khắc nghiệt bao gồm: ban đêm thiếu sáng, ánh sáng ngược, trời mưa lớn, sương mù, cũng như đặc trưng giao thông hỗn hợp với mật độ xe máy dày đặc tại các đô thị lớn.

TÀI LIỆU THAM KHẢO

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Las Vegas, NV, USA, Jun. 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), Seattle, WA, USA, Jun. 2020, pp. 1571–1580, doi: 10.1109/CVPRW50498.2020.00203.
- [4] L. Yang, R.-Y. Zhang, L. Li, and X. Xie, "SimAM: A Simple, Parameter-Free Attention Module for Convolutional Neural Networks," in Proc. 38th Int. Conf. Mach. Learn. (ICML), PMLR, vol. 139, Jul. 2021, pp. 11863–11874. [Online]. Available: <https://proceedings.mlr.press/v139/yang21o.html>
- [5] C. Wang, W. He, Y. Nie, J. Guo, C. Liu, Y. Wang, and K. Han, "Gold-YOLO: Efficient Object Detector via Gather-and-Distribute Mechanism," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, Dec. 2023, pp. 51094–51112. [Online]. Available: <https://arxiv.org/abs/2309.11331>
- [6] J. Chen, S.-H. Kao, H. He, W. Zhuo, S. Wen, C.-H. Lee, and S.-H. G. Chan, "Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Vancouver, Canada, Jun. 2023, pp. 12021–12031, doi: 10.1109/CVPR52729.2023.01157.
- [7] A. Geiger, P. Lenz, and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Providence, RI, USA, Jun. 2012, pp. 3354–3361, doi: 10.1109/CVPR.2012.6248074.
- [8] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshain, L. Antiga et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 32, Dec. 2019, pp. 8024–8035. [Online]. Available: <https://papers.nips.cc/paper/9015>

- [9] Z. Zou, K. Chen, Z. Shi, Y. Guo, and J. Ye, "Object Detection in 20 Years: A Survey," *Proc. IEEE*, vol. 111, no. 3, pp. 257–276, Mar. 2023, doi: 10.1109/JPROC.2023.3238524.
- [10] L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu, and M. Pietikäinen, "Deep Learning for Generic Object Detection: A Survey," *Int. J. Comput. Vis.*, vol. 128, no. 2, pp. 261–318, Feb. 2020, doi: 10.1007/s11263-019-01247-4.
- [11] Q. Feng, X. Xu, and Z. Wang, "Deep Learning-Based Small Object Detection: A Survey," *Math. Biosci. Eng.*, vol. 20, no. 4, pp. 6551–6590, Feb. 2023, doi: 10.3934/mbe.2023282.
- [12] A. Koubaa, A. Benjdira, A. Ammar, B. Ouni, and M. Kessentini, "A Comprehensive Survey of Deep Learning-Based Lightweight Object Detection Models for Edge Devices," *Artif. Intell. Rev.*, vol. 57, no. 10, Oct. 2024, doi: 10.1007/s10462-024-10877-1.
- [13] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Columbus, OH, USA, Jun. 2014, pp. 580–587, doi: 10.1109/CVPR.2014.81.
- [14] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017, doi: 10.1109/TPAMI.2016.2577031.
- [15] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single Shot MultiBox Detector," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Lecture Notes in Computer Science, vol. 9905, Amsterdam, Netherlands, Oct. 2016, pp. 21–37, doi: 10.1007/978-3-319-46448-0_2.
- [16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.324.
- [17] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, Jul. 2017, pp. 936–944, doi: 10.1109/CVPR.2017.106.

- [18] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path Aggregation Network for Instance Segmentation," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Salt Lake City, UT, USA, Jun. 2018, pp. 8759–8768, doi: 10.1109/CVPR.2018.00913.
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition," IEEE Trans. Pattern Anal. Mach. Intell., vol. 37, no. 9, pp. 1904–1916, Sep. 2015, doi: 10.1109/TPAMI.2015.2389824.
- [20] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression," in Proc. AAAI Conf. Artif. Intell., vol. 34, no. 7, Apr. 2020, pp. 12993–13000, doi: 10.1609/aaai.v34i07.6999.
- [21] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, Dec. 2020, pp. 21002–21012. [Online]. Available: <https://arxiv.org/abs/2006.04388>
- [22] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, "TOOD: Task-Aligned One-Stage Object Detection," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Montreal, Canada, Oct. 2021, pp. 3490–3499, doi: 10.1109/ICCV48922.2021.00349.
- [23] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-Style ConvNets Great Again," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Nashville, TN, USA, Jun. 2021, pp. 13733–13742, doi: 10.1109/CVPR46437.2021.01352.
- [24] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Salt Lake City, UT, USA, Jun. 2018, pp. 7132–7141, doi: 10.1109/CVPR.2018.00745.
- [25] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional Block Attention Module," in Proc. Eur. Conf. Comput. Vis. (ECCV), Lecture Notes in Computer Science, vol. 11211, Munich, Germany, Sep. 2018, pp. 3–19, doi: 10.1007/978-3-030-01234-2_1.
- [26] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, "BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Seattle, WA, USA, Jun. 2020, pp. 2633–2642, doi: 10.1109/CVPR42600.2020.00271.

- [27] Z. Tong, Y. Chen, Z. Xu, and R. Yu, "Wise-IoU: Bounding Box Regression Loss with Dynamic Focusing Mechanism," *arXiv preprint arXiv:2301.10051*, 2023.
- [28] B. Wang, Y.-Y. Li, W. Xu, H. Wang, and L. Hu, "Vehicle–Pedestrian Detection Method Based on Improved YOLOv8," *Electronics*, vol. 13, no. 11, p. 2149, 2024.
- [29] D. Misra, "Mish: A Self Regularized Non-Monotonic Activation Function," *arXiv preprint arXiv:1908.08681*, 2019.